

Alexandria Chemistry Toolkit

Release 1.2b

David van der Spoel **Paul J. van Maaren**
Mohammad M. Ghahremanpour

Nov 14, 2026

CONTENTS:

1 Preface **1**
- 1.1 Before you begin… . . . 1
- 1.2 About the Alexandria Chemistry Toolkit . . . 1
- 1.3 Motivation for the ACT . . . 1

2 User Manual **3**
- 2.1 Installation of the ACT . . . 3
  - 2.1.1 Prerequisites . . . 3
  - 2.1.2 Conda Environment . . . 4
    - 2.1.2.1 Building on a cluster . . . 4
  - 2.1.3 Running the Installation . . . 6
    - 2.1.3.1 GPU support with SYCL . . . 8
  - 2.1.4 Troubleshooting Installation . . . 10
  - 2.1.5 Testing the ACT . . . 10
    - 2.1.5.1 Testing the GPU code . . . 11
- 2.2 Verifying changes to ACT . . . 12
  - 2.2.1 Running the test set fully . . . 12
  - 2.2.2 The regression set . . . 14
  - 2.2.3 Test coverage . . . 15
  - 2.2.4 Cleaning up the code with clang-tidy . . . 15
  - 2.2.5 Code testing on GitHub . . . 16
- 2.3 Using the ACT . . . 17
  - 2.3.1 ACT File formats . . . 19
  - 2.3.2 Creating a new force field file from scratch . . . 19
  - 2.3.3 Train your first force field . . . 20
  - 2.3.4 Is my training hitting a wall? . . . 24
  - 2.3.5 Running training using parallel processing . . . 25
- 2.4 Getting the best performance from ACT . . . 25
  - 2.4.1 Building ACT for speed . . . 25
  - 2.4.2 How train_ff uses the processes . . . 26
  - 2.4.3 Processor or GPU? . . . 26
  - 2.4.4 Training on GPUs . . . 27
  - 2.4.5 Measuring and comparing the run time . . . 28
- 2.5 MD Simulations with Alexandria Force Fields . . . 28
  - 2.5.1 Using ACT simulate . . . 28
  - 2.5.2 The ACT-OpenMM interface . . . 29
    - 2.5.2.1 MD simulations using OpenMM . . . 29
    - 2.5.2.2 Energy Minimization using OpenMM . . . 30
- 2.6 Predicting Molecular Properties . . . 30

2.6.1 Electrostatic Potential . . . 30
2.6.2 Electrostatic Moments . . . 31
2.6.3 Polarizability . . . 32
2.6.4 Normal Mode Analysis . . . 32
2.6.4.1 Infrared Spectra . . . 33
2.6.4.2 Thermochemistry . . . 34
2.6.5 Second Virial Coefficient . . . 35

3 Reference Manual **38**
3.1 Energy Function . . . 38
3.1.1 Non-bonded interactions . . . 38
3.1.1.1 Coulomb Interaction using Point charges . . . 38
3.1.1.2 Coulomb Interaction using Gaussian charges . . . 38
3.1.1.3 Coulomb Interaction using Slater charges . . . 38
3.1.1.4 Long range Coulomb interactions . . . 39
3.1.1.5 Polarization . . . 39
3.1.1.6 Induction correction . . . 39
3.1.1.7 Pauli Repulsion and London Dispersion . . . 40
3.1.1.8 Treatment of long range dispersion . . . 42
3.1.1.9 Combination Rules for Van der Waals Potentials . . . 43
3.1.2 Bonded interactions . . . 45
3.1.2.1 Harmonic potential . . . 45
3.1.2.2 Morse potential . . . 45
3.1.2.3 Wei-Hua potential . . . 45
3.1.2.4 Angle potential . . . 46
3.1.2.5 Angle potential for linear compounds . . . 46
3.1.2.6 Out-of-plane vibrations . . . 46
3.1.2.7 Torsion potential . . . 46
3.1.2.8 Proper dihedral . . . 46
3.1.3 Special potentials . . . 47
3.1.4 Virtual Sites . . . 47
3.1.5 Total energy . . . 47
3.2 Shell Relaxation and Integration . . . 47
3.2.1 Relaxation of shells . . . 48
3.2.1.1 Shell relaxation in training . . . 50
3.2.2 Integration in simulate . . . 50
3.2.2.1 Constraints . . . 51
3.3 Force Field Training Algorithms . . . 53
3.3.1 MCMC . . . 53
3.3.1.1 Metropolis Criterion . . . 53
3.3.1.2 Multiple MCMC runs . . . 54
3.3.2 Genetic Algorithm . . . 55
3.3.2.1 Initialization . . . 56
3.3.2.2 Deviation from data . . . 56
3.3.2.3 Sorting . . . 57
3.3.2.4 Penalties . . . 57
3.3.2.5 Selection probabilities . . . 57
3.3.2.6 Rank . . . 58
3.3.2.7 Fitness . . . 58
3.3.2.8 Boltzmann . . . 58
3.3.2.9 Elitism . . . 59

3.3.2.10 Selection . . . 59
3.3.2.11 Crossover . . . 59
3.3.2.12 Mutation . . . 60
3.3.2.13 Termination . . . 60
3.3.2.14 Fitness files . . . 60
3.3.3 HYBRID . . . 60
3.4 Fitness function . . . 61
3.4.1 Supported training targets . . . 61
3.4.2 Parameter mutability . . . 62
3.4.3 Bounds Penalty . . . 63
3.5 Sensitivity Analysis . . . 63
3.5.1 Running Sensitivity Analysis . . . 64
3.5.2 Algorithm . . . 64
3.5.3 Two-dimensional analysis . . . 65
3.5.4 Convergence during training . . . 66
3.5.5 Output Format . . . 66
3.5.6 Interpretation . . . 68
3.6 Force field design . . . 69
3.6.1 Physical Background . . . 69
3.6.2 Determining atom types . . . 70
3.6.3 Determining partial charges . . . 73
3.6.4 Charge Equilibration with Shells or Virtual Sites . . . 74
3.7 Force Field Training Targets . . . 75
3.7.1 Algorithm to compute energy components in ACT . . . 76
3.7.2 Monomer Energies and Forces . . . 77
3.7.3 Other Properties . . . 77
3.8 Parameters for Force Field Training . . . 77
3.9 Training Data . . . 78
3.9.1 Using existing data . . . 78
3.9.2 Alexandria Library . . . 79
3.9.3 Donchev data set . . . 79
3.9.4 Non-covalent interaction atlas . . . 79
3.9.5 ACT data . . . 79
3.9.6 Generating SAPT data . . . 79
3.9.7 Generating single molecule data . . . 80
3.9.8 Conversion to ACT molprop files . . . 80

4 Interactive Notebooks **81**
4.1 Charge Methods . . . 81
4.1.1 Electronegativity equalization method . . . 81
4.1.2 Split charge equilibration . . . 83
4.2 Atom Typing using RDKit . . . 87

5 Developer Manual **88**

Bibliography **89**

CHAPTER

ONE

PREFACE

1.1 Before you begin...

This manual is a work in progress, covering the physical background of the force field development in the ACT, which is short for Alexandria Chemistry Toolkit, from potential energy functions to the molecular properties supported by the ACT, algorithms used for training, how to generate training data, installation and usage.

A pdf version of this manual is available.

Throughout this document we highlight *alexandria commands* like this. There are also hyperlinks in the document, both for the references and in the running text, often linking to Wikipedia.

The authors welcome suggestions for additions and improvements as well as corrections for factual errors. Please send your comments by e-mail to david.vanderspoel at icm.uu.se.

We kindly request that you cite the paper about ACT [1] and other relevant works if you use the software for a scientific publication of your own.

The ACT can be downloaded as free and open source software from Github.

1.2 About the Alexandria Chemistry Toolkit

The Alexandria Chemistry Toolkit (ACT) allows for global optimization of force fields for molecular simulation. That means that, in principle, a whole force field can be derived at once by training parameters to reproduce results from *ab initio* or density functional theory as well as experimental data when available.

The ACT software consists, at the time of writing about 107,000 lines of C++ code, of which about 35,000 lines were generated by the Mathematica ™ software (Section *Coulomb Interaction using Slater charges*). In addition about 7,500 lines of Python code are part of the ACT.

The ACT was developed in the lab of David van der Spoel at Uppsala University, Sweden. A list of contributors is given on the github page of the project.

1.3 Motivation for the ACT

The fundamental motivation for the development of the ACT are given in these reviews [2, 3, 4, 5, 6, 7, 8]. Milestones on the way to a new family of physics-based force fields are given below.:

- A long series of benchmarks of force fields [9, 10, 11, 12, 13, 14, 15].
- Databases of quantum-chemical data produced as part of the Alexandria project: enthalpies of formation and thermochemistry [16] applied to force fields here [13].

- The Alexandria Library with more quantum-chemistry data [17, 18].
- A phase-transferable force field for alkali halides [19] with applications towards prediction of melting points and conductivities for alkali halide liquids and mixtures [20, 21, 22, 23].
- A new potential for treating the angle in linear molecules [24].
- New models for noble gases including accurate simulations of the melting point of solid noble gases [25].
- Use of symmetry-adapted perturbation theory for a study of exchange around a water molecule [26].
- An evaluation of potentials for chemical bonds in molecules [27].

The remainder of this document contains first the User Manual, describing installation and practical usage of the ACT including the properties that can be predicted by the ACT software and Alexandria force fields.

The second part is the Reference Manual which presents, somewhat rudimentary at the time of writing, the physical foundation of the ACT models and the algorithms for training force fields.

Happy force field designing!

CHAPTER

TWO

USER MANUAL

2.1 Installation of the ACT

The Alexandria Chemistry Toolkit (ACT) relies on a number of libraries. Even though we tried to keep it to a minimum, some more or less standard libraries are needed. ACT should compile fine on any UNIX (including MacOs) or Linux machine. %Most of the libraries can be installed using Anaconda or even Miniconda which has the advantage of running in user-space entirely, that is you do not super-user access to install it.

2.1.1 Prerequisites

The following software packages are required for the ACT to work:

- C and C++ compilers supporting C++20 at least. On Linux a GNU c++ version newer than 12.0 is recommended.
- Some version of a library that supports the message passing interface (MPI) for parallel programming. A popular version is OpenMPI.
- The cmake tools (at least version 3.13.0) are needed for compiling the code.
- For linear algebra operations we use the Eigen library, version 5 or better.
- The RDKit library (at least version 2025.09.4)
- The Boost developer library of version 1.86.0 (matching RDKit)
- The LibXml2 is needed for processing the XML data files used by the ACT.
- The nlohmann/json library is required for JSON I/O and can be installed via conda as `nlohmann_json`.
- Python, version 3.8 or better, and a number of Python libraries, namely, NumPy, Matplotlib, and PubChemPy.

The following libraries are optional only, but may be useful for developing the ACT:

- The Class Library for Numbers is used in an optional part of the code (Slater-distributed charges [17]) and can be omitted.
- The SQLite database engine is needed to process experimental data as well as quantum chemistry data from the Alexandria Library [17], available from Zenodo.

For developers, the following additional packages are needed:

- doxygen, it is used for generating software documentation,

- graphviz can be optionally added for generating graphs and tree-structures in the doxygen documentation,
- pygments, for source code listings,
- sphinx, which is used for building the manual, with
- sphinxcontrib-bibtex, for the references and
- nbsphinx for including Jupyter Notebooks.

2.1.2 Conda Environment

There are multiple ways to fulfill the prerequisites. The simplest way that should suffice on a single computer (i.e. not a cluster), is to first install miniconda on your computer, download the ACT conda environment file Yml and create and activate a new conda environment.:

```
conda create -n ACT
conda activate ACT
conda config --add channels anaconda
conda config --add channels conda-forge
conda install librdkit-dev>=2025.09.4 libboost-devel>=1.86.0 cmake eigen>=5␣
↪libxml2 numpy matplotlib pubchempy pillow nlohmann_json
```

This should install the libraries mentioned above (note: it will take some time!). If you are installing ACT on a high-performance computing cluster, there likely is support for compilers and a MPI library already. If not, then add the *openmpi* package to your conda install line. Most Linux installations come bundled with the GNU compiler suite (GCC) and for macOS the Xcode package can be downloaded free of charge from Xcode. If you do not have a compiler, add *gcc* to the conda install line:

```
conda install gcc=14 gxx=14 openmpi=5
```

For developers, please additionally install these packages:

```
conda install doxygen graphviz pygments sphinx sphinxcontrib-bibtex nbsphinx
```

Attention

If you are installing ACT in a cluster we recommend to use the cluster-provided compilers and in particular the MPI library since it may be tuned to make optimal use of the communication hardware. High-performance computer centers typically provide compilers libraries using some kind of module system.

2.1.2.1 Building on a cluster

On a cluster, use the MPI library of the cluster, which its administrators built for its network and queue system, and conda only for the libraries that the cluster does not provide, such as RDKit. Leave *openmpi* out of the conda environment, or tell *cmake* which MPI to use with the compiler wrappers of the module, for instance:

```
module load <compiler and MPI module of the cluster>
conda activate ACT
cmake .. -DMPI_C_COMPILER=$(which mpicc) -DMPI_CXX_COMPILER=$(which mpicxx) \
```

(continues on next page)

(continued from previous page)

```
        -DCMAKE_PREFIX_PATH=${CONDA_PREFIX} -DCMAKE_INSTALL_PREFIX=$HOME/
↪tools
```

The C and C++ compilers may be those of the cluster or those of conda. The compilers of conda link against a system root of their own, in which the linker also looks for the libraries that other libraries depend on. The MPI library of a cluster depends on libraries outside it, for instance those of the network, and of CUDA if it is GPU aware. For an MPI library outside the conda environment, *cmake* therefore gives the linker the library directories of the system, of the MPI library, and of *LIBRARY_PATH* and *LD_LIBRARY_PATH*, which modules set. The output of *cmake* tells when it does so.

Start the programs with the launcher of the queue system, which places the processes on the nodes and cores of the job, for instance with Slurm:

```
srun --mpi=pmi2 alexandria train_ff ...
```

Which *–mpi* option works depends on the MPI library, *srun –mpi=list* lists those that the cluster supports. The tests that use MPI are started with the *mpiexec* of the MPI library, unless *-DMPIEXEC_EXECUTABLE=srun* is given to *cmake*.

For example, on the GH200 nodes of arrhenius (NAISS), with the GPU toolchain of the cluster (GCC 14.3, MPICH 4.3.2, CUDA 13.0) and the libraries and AdaptiveCpp from conda, in a job on such a node since the login node has another processor:

```
module load GPU/buildenv-gcccuda/2026.03-cu13.0
export PATH=<conda environment>/bin:$PATH
cmake .. -DACT_DOUBLE=OFF -DACT_GPU=SYCL -DCMAKE_PREFIX_PATH=<conda␣
↪environment> \
        -DAdaptiveCpp_DIR=<AdaptiveCpp environment>/lib/cmake/AdaptiveCpp \
        -DMPI_C_COMPILER=$(which mpicc) -DMPI_CXX_COMPILER=$(which mpicxx)
```

and the programs are started with *srun –mpi=pmi2*.

On a Cray system, such as dardel (PDC), the compiler wrappers *cc* and *CC* of the Cray programming environment include the MPI library of the cluster already. Give them as the compilers and *cmake* finds MPI through them, nothing else is needed. For example, on the AMD MI250X nodes of dardel, with the Cray compilers, the libxml2 of the cluster and the other libraries and AdaptiveCpp from conda:

```
cmake .. -DACT_DOUBLE=OFF -DACT_GPU=SYCL -DCMAKE_C_COMPILER=cc -DCMAKE_CXX_
↪COMPILER=CC \
        -DCMAKE_PREFIX_PATH=<conda environment> \
        -DAdaptiveCpp_DIR=<conda environment>/lib/cmake/AdaptiveCpp \
        -DLIBXML2_LIBRARY=/pdc/software/eb/software/libxml2/2.12.3/lib/
↪libxml2.so \
        -DLIBXML2_INCLUDE_DIR=/pdc/software/eb/software/libxml2/2.12.3/
↪include/libxml2
```

The programs are started with plain *srun*. This libxml2 needs the ICU libraries of the cluster, so load them with *module load icu/74.1* in the job script as well.

2.1.3 Running the Installation

Once you have download ACT either as a release version or by cloning the git repository, enter the directory where the ACT source code is installed, and issue the following commands:

```
mkdir build-Release
cd build-Release
cmake -DCMAKE_INSTALL_PREFIX=$HOME/tools ..
make -j 8 install
```

where the *CMAKE_INSTALL_PREFIX* flag points to the location where ACT will be installed and the *-j 8* flag instructs the make command to utilize 8 cores to speed up compilation.

The programs and python scripts find the data files of ACT, in *share/act* of the installation, by themselves, also when started from the build directory, in which case they use those in *share* of the source code. To use other data files, set the environment variable *ACTDATA* to their directory; files that are not there are still taken from the installation. All that remains is to add the directory of the programs to your *PATH*, or to run the following command, which does that as well:

```
source $HOME/tools/bin/ACTRC
```

Put either in your .bash_profile (or equivalent, for remote machines) or .bashrc (or equivalent, for local machines), and restart the shell or log in again. Then you can run the alexandria executable using:

```
alexandria -h
```

To make sure you do have the correct commands in your path, please try the command:

```
which alexandria
```

which should give you something like:

```
% which alexandria
~/tools/bin/alexandria
```

There are some building options available that are mainly of use for developers (see *cmake flags available to build the alexandria program.*). These options have to be specified using:

```
-DOPTION=Value
```

where *Value* can be *ON* or *OFF* or something more option specific.

Table 2.1: cmake flags available to build the alexandria program.

Flag	Description
CMAKE_BUILD_TYPE	Build Type: either Release (default) Debug (for use with a debugger) or ASAN (Adress Sanitizer, for uncovering memory leaks and debugging crashes).
CMAKE_INSTALL_PREFIX	Path where to install the ACT, see above
CMAKE_PREFIX_PATH	Path where cmake can look for libraries. Multiple paths can be specified, separated by semicolons, for instance -DCMAKE_PREFIX_PATH=${CONDA_PREFIX}/lib
ACT_DOUBLE	Double precision, default ON. Single precision (OFF) is

continues on next page

Table 2.1 – continued from previous page

Flag	Description
	faster and is needed to use a GPU. Before, this option was called GMX_DOUBLE; the old names of this and the next two options still work, with a warning.
ACT_MPI	Build with MPI, needed to train with several processes, default ON.
ACT_OPENMP	OpenMP multithreading, default OFF. ACT parallelizes with MPI processes, see ACT_OPENMP_SIMD for the vectorization of loops.
ACT_CLN	Install ACT using the Class Library for Numbers for high precision calculations, used for Slater integrals. Default OFF, activated when ON.
ACT_BUILD_MANUAL	Whether or not to provision for building the manual. Requires installing developer tools, default OFF.
CMAKE_C_FLAGS	By default nothing is needed, but sometimes this linker flag helps to overcome problems: "-L${CONDA_PREFIX}/lib -Wl,-rpath,${CONDA_PREFIX}/lib"
CMAKE_CXX_FLAGS	See previous entry, you likely need to add both.
ACT_SIMD	Explicit SIMD instructions for the energy and force routines: AUTO (default), NONE, ARM_NEON, AVX2 (with FMA, for instance Intel Haswell and AMD Zen and later) or AVX_512. AUTO selects the best SIMD for the processor that cmake runs on, or AVX2 if the compiler cannot produce AVX-512 code. NONE turns explicit SIMD off. Only the explicit SIMD routines are compiled for this SIMD, and they are only used if the processor running the program supports it. Otherwise the routines vectorized by the compiler are used, such that the program also runs on processors with less SIMD.
ACT_PAIR_UNROLL	Unroll factor, 1 to 16, of the loops over conformations of the explicit SIMD routines for pair interactions, such as Coulomb, van der Waals and bonds. Empty (default) uses 4 for each SIMD. Changing it compiles only one file again and is meant for finding the fastest value on a computer, see below.
ACT_OPENMP_SIMD	Let the compiler vectorize the loops over conformations marked with OpenMP SIMD directives, using -fopenmp-simd. This does not turn on OpenMP multithreading. Default ON.
ACT_VECTOR_MATH	Use a vector math library for mathematical functions such as exp and erf in the loops vectorized by the compiler: the system math library on macOS, libmvec or SVML otherwise. Without such a library these loops are not vectorized. Default ON.
ACT_COUNT_FLOPS	Count the floating point operations, such that the timings at the end of the log file report the GFlop/s of the parts of the program and the fraction of the peak performance. Default OFF, since counting costs up to 10% of the time of molecular dynamics. Meant for developers.
ACT_GPU	GPU support: NONE (default) or SYCL, which needs

continues on next page

Table 2.1 – continued from previous page

Flag	Description
	AdaptiveCpp, see below.

The SIMD (single instruction, multiple data) instructions of the processor are used to compute the energies and forces of several conformations of a molecule at once, which speeds up training considerably. At the start of each program the log file reports the SIMD of the processor, the SIMD the compiler could use for the rest of the code and whether explicit SIMD is used, for instance:

```
Explicit SIMD:      AVX2, 4 conformations at once, pair loops unrolled 4␣
↪times, in use
```

For most users the defaults are best. To build for a cluster with processors different from the one where you compile, set *ACT_SIMD* to the SIMD of the compute nodes. Since the explicit SIMD routines are only used if the processor supports them, a program built with AVX_512 still runs correctly on a processor with only AVX2, albeit more slowly. The rest of the code is vectorized by the compiler for the instructions given in *CMAKE_CXX_FLAGS*, for instance *-march=native* to use all instructions of the processor where you compile. Setting the environment variable *ACT_NO_EXPLICIT_SIMD* when running a program turns off the explicit SIMD routines, which can be used to check the results or the speed of the different routines:

```
ACT_NO_EXPLICIT_SIMD=1 alexandria train_ff ...
```

The log file also reports the peak performance of a core, estimated from the SIMD instructions of the processor (floating point operations per cycle) and its maximum clock frequency, which the timings at the end of the log file use to report the efficiency of the potentials, if ACT was configured with *-DACT_COUNT_FLOPS=ON*. For a processor that ACT does not recognize, the environment variable *ACT_PEAK_GFLOPS* sets the peak of a core in GFlop/s in double precision.

The loops over conformations of the explicit SIMD routines for pair interactions are unrolled, such that the processor has the instructions for several SIMD vectors of conformations at once. Which unroll factor is fastest depends on the processor, since a larger factor helps when the loop overhead matters, but helps less with wide SIMD such as AVX-512 and not when only few conformations are computed, as in the last iterations of relaxing shells. To find the best value on your computer, configure with *-DACT_COUNT_FLOPS=ON*, build with several values of *ACT_PAIR_UNROLL* in the same build directory and compare the GFlop/s of the potentials, such as *Potential COULOMB_POINT*, in the timings at the end of the log file of, for instance, a calculation of the second virial coefficient:

```
cmake -DACT_PAIR_UNROLL=2 . && make -j 8 install
cmake -DACT_PAIR_UNROLL=8 . && make -j 8 install
cmake -DACT_PAIR_UNROLL= . && make -j 8 install
```

The last command returns to the default. The results do not depend on the unroll factor.

2.1.3.1 GPU support with SYCL

GPU support uses SYCL with AdaptiveCpp, which compiles the GPU code at run time for the GPU found, such that the same build runs on NVIDIA and AMD GPUs. AdaptiveCpp is available from conda-forge, in its own environment, for instance for NVIDIA GPUs:

```
conda create -n act-sycl -c conda-forge "adaptivecpp=*=cuda*"
cmake .. -DACT_GPU=SYCL -DAdaptiveCpp_DIR=$HOME/miniforge3/envs/act-sycl/lib/
↪cmake/AdaptiveCpp
```

The variant matters: AdaptiveCpp uses a GPU through a backend, CUDA for NVIDIA GPUs and HIP for AMD GPUs, and the *cpu* variants of conda-forge, which other packages may pull into an environment, lack the CUDA backend. *cmake* reports the backends of the AdaptiveCpp it uses, which the line *GPU support* at the start of the log file repeats, and warns if the CUDA backend is missing. If a program finds no GPU, the log file says why, as far as it can tell: for instance that the node has NVIDIA GPUs but AdaptiveCpp has no CUDA backend, that the queue system gave the job no GPU, or that the node has no GPUs. The command *acpp-info -l* of AdaptiveCpp lists the devices it finds.

Only the GPU code is compiled with AdaptiveCpp's compiler acpp, the rest of ACT with the C++ compiler, which needs to be GCC 13 or newer, since the libraries of AdaptiveCpp need its C++ library. The GPU is used in single precision builds only (*-DACT_DOUBLE=OFF*). The section *Code used* at the start of the log file lists the GPUs found, with an estimate of their peak performance in single precision: the number of compute units times the single precision lanes of a compute unit, which depends on the type of GPU, times two operations per fused multiply-add, times the maximum clock frequency. The processes on a node are spread over its GPUs, by their number on the node, which Slurm or the MPI library sets.

In training with a polarizable force field, by default (*train_ff -gpu_resident*), each process keeps the structures of its molecules on the GPU, and computes there in each evaluation the charges, if they are generated with SQE, the virtual sites, all interactions and the relaxation of the shells. Only the parameters go to the GPU. In the evaluations of training the GPU also computes the deviations of the energies from the reference data, such that only those come back; the energies of the molecules themselves are computed for the output at the end. If the GPU does not support the potentials of the force field, the processor computes everything, as with *-nogpu_resident*. Otherwise the GPU computes the Coulomb interactions of several structures of all molecules at once and relaxes the shells of all molecules at once: the interactions with shells, which may be Coulomb, polarization, Lennard-Jones 12-6, buffered 14-7, Wang-Buckingham and MacDaniel-Schmidt, are computed on the GPU until the shells converged, the other interactions once on the processor. Since starting the GPU takes some tens of microseconds, this is only done for at least 100000 interactions times structures of all molecules, which the environment variable *ACT_GPU_MIN_WORK* changes.

Many processes per GPU keep it busy, since each process waits for the GPU part of the time. When several MPI ranks share a GPU, start the NVIDIA multi-process service in the job script, otherwise the GPU switches between the processes, which makes each operation on it about twenty times slower. The service takes at most 48 or 60 processes per GPU, depending on the version of the NVIDIA driver (48 on an RTX A4500 with driver 550, while NVIDIA documents 60 for current drivers), and the ranks beyond that compute without the GPU. *CUDA_DEVICE_MAX_CONNECTIONS* larger than its default of 2 lowers that number:

```
export CUDA_MPS_PIPE_DIRECTORY=/tmp/mps-$SLURM_JOB_ID CUDA_MPS_LOG_DIRECTORY=/
↪tmp/mpslog-$SLURM_JOB_ID
mkdir -p $CUDA_MPS_PIPE_DIRECTORY $CUDA_MPS_LOG_DIRECTORY
nvidia-cuda-mps-control -d
srun ... alexandria train_ff ...
echo quit | nvidia-cuda-mps-control
```

In *train_ff* with *-nogpu_master* the master process does not use the GPU, which leaves a place among the processes of the GPU and some hundreds of MB of its memory to the other processes. The master then computes the evaluations for the output, at the start and the end, on the processor, which took about

40 s more for the BFdb benchmark on one core.

Each process uses one GPU of the node, chosen by its number on the node, and ACT lets it see only that one: if *CUDA_VISIBLE_DEVICES*, *ROCR_VISIBLE_DEVICES* or *HIP_VISIBLE_DEVICES* lists several GPUs, as Slurm sets them, only that of the process is kept, and without such a list on a node with several NVIDIA GPUs *CUDA_VISIBLE_DEVICES* is set to it. Otherwise the GPU runtime would make a context on each GPU in each process, and with several GPUs per node the multi-process service would run out of processes. The log file tells which GPU each process uses.

With a population larger than the number of processes, each process can take several individuals at once with *train_ff -lockstep*. The GPU then runs the MCMC steps of these individuals together, in the same calculations, such that fewer processes use it better. The result of each individual is the same as when it is computed by itself, except for where the relaxation of the shells starts, see *-shell_keep*. With a population of 128, *-lockstep 8* with 17 processes was the fastest on a node with eight AMD MI250X GCDs and on one with four NVIDIA GH200, 1.5 to 4 times faster than one individual per process with 129 processes; see the chapter on performance for the measurements.

The GPU gives the same results as the processor, within single precision. The option *-nogpu*, which all programs have, turns the GPU off, for instance to compare the run time with and without the GPU. The programs set *ACPP_DEBUG_LEVEL* to 1, unless it is set already, such that AdaptiveCpp does not warn about the GPU backends it cannot load. Set it to 2 to see the messages of AdaptiveCpp.

2.1.4 Troubleshooting Installation

Sometimes, the *cmake* process or building using *make* does not work as described above. For instance, there may be library mismatches like this:

```
undefined reference to `__cxa_call_terminate@CXXABI_1.3.15'
```

which is caused by the fact that conda libraries expect specific versions of system libraries, combined with the make process supplying a wrong version of that library. In that case you can instruct cmake to change the order of libraries by adding the flags to the cmake command line according to Table 2.1.

2.1.5 Testing the ACT

To start testing, you first want to familiarize yourself with the test set. If the ACT is in your home directory, you can:

```
cd ACT/build-Release
```

Then you can build the test set using:

```
make tests
```

and run it using:

```
make test
```

which should give the following output:

```
Running tests...
Test project /Users/spoel/GG/ACT/build_Release_DOUBLE
      Start  1: TestUtilsUnitTests
 1/17 Test  #1: TestUtilsUnitTests ..............   Passed    3.10 sec
```

(continues on next page)

(continued from previous page)

```
      Start  2: WangBuckinghamTests
 2/17 Test  #2: WangBuckinghamTests ..............   Passed    0.33 sec
      Start 16: AlexandriaTests
```

(more tests):

```
16/17 Test #16: AlexandriaTests ..................   Passed    8.80 sec
      Start 17: SobolTests
17/17 Test #17: SobolTests .......................   Passed    0.16 sec

100% tests passed, 0 tests failed out of 17
```

You can also run an individual test, like this bin/sobol-test which should give this output:

```
[==========] Running 2 tests from 1 test case.
[----------] Global test environment set-up.
[----------] 2 tests from SobolTest
[ RUN      ] SobolTest.Test08
[       OK ] SobolTest.Test08 (0 ms)
[ RUN      ] SobolTest.Test09
[       OK ] SobolTest.Test09 (0 ms)
[----------] 2 tests from SobolTest (0 ms total)

[----------] Global test environment tear-down
[==========] 2 tests from 1 test case ran. (0 ms total)
[  PASSED  ] 2 tests.
```

Note that these tests are run every time a change in the ACT source code is uploaded to github, to prevent errors in the code from being introduced.

2.1.5.1 Testing the GPU code

The tests of the GPU code need a GPU, in a single precision build with *-DACT_GPU=SYCL*. Their names end in *Gpu*, and *ctest* runs them separately from the other tests of a test program: *ACTForcesTestsGpu* (the pair potentials on the GPU), *AlexandriaTestsGpu* (the molecules on the GPU, several sets of parameters at once, the deviations on the GPU and *-shell_keep*) and *AlexandriaMpiTestsGpu* (training on the GPU with *-lockstep* and with molecules that are skipped). Without a GPU, or in double precision, *ctest* reports these three as skipped, for instance:

```
20/29 Test #20: ACTForcesTestsGpu ................***Skipped   0.85 sec
```

and lists them at the end under *The following tests did not run*. A test run on a computer without a GPU, such as the head node of a cluster, therefore does not test the GPU code, even when all tests pass. *ACTGpuTests* checks the estimate of the peak performance and the explanation of why no GPU was found, which need no GPU. If a GPU is found but cannot compute the molecules of a test, that test fails.

On a cluster, compile everything on the head node, the tests included, and run the tests in a job on a node with a GPU. The head node needs no GPU to compile, since AdaptiveCpp compiles the GPU code when a program first uses the GPU. Set *ACT_SIMD* to the SIMD instructions of the GPU nodes if their processors differ from that of the head node, see above. If the processors of the GPU nodes are of another type than that of the head node, for instance ARM against x86, compile in a job on a GPU node instead. On csb, for instance, after:

```
cd ACT/build_Release_SINGLE_SYCL
make -j 20 alexandria tests
```

on the head node, a job script like this runs the tests on a GPU node, with the modules that were loaded for compiling:

```
#!/bin/bash -l
#SBATCH -p CLUSTER-AMD
#SBATCH -N 1 -n 8
#SBATCH --gres=gpu:1
#SBATCH -t 00:40:00
ml gcc/14 fftw openmpi/5.0.5
cd $HOME/ACT/build_Release_SINGLE_SYCL
$HOME/miniforge3/envs/ACT/bin/ctest -j 4 -V > ctest-gpu.log 2>&1
grep -E "tests passed|Skipped|\*\*\*" ctest-gpu.log
```

The program *ctest* comes with *cmake*. On csb the *cmake* module does not work on the compute nodes, therefore the script uses the *ctest* of the conda environment of ACT, which has *cmake* as well. The last line should report that all tests passed, and no test should be skipped. Run *make tests* again after each change before submitting the job, since *make* by itself does not rebuild the test programs. To run a single test program on a GPU node interactively, for instance:

```
srun -p CLUSTER-AMD --gres=gpu:1 -n 1 -c 8 bin/act-forces-test --gtest_filter=
↪'*Gpu'
```

The first run on a type of GPU takes longer, since AdaptiveCpp then compiles the GPU code, which it keeps in *~/.acpp* for later runs. If the MPI tests fail in the job because MPI programs are started differently there, run the others with *ctest -LE MpiTest*. Most tests have a time limit of 120 s, since some of them take 40 s on older processors.

2.2 Verifying changes to ACT

The test set described in the previous chapter checks that ACT works as intended. When changing the code, two scripts in the *admin* directory of the source code help to check more thoroughly that nothing broke: *verify.sh* runs the tests in different ways and compares a set of real calculations with a reference, and *coverage.sh* reports which parts of the code the tests do not run. In addition, *make tidy* cleans up the code with *clang-tidy*, see the last section.

2.2.1 Running the test set fully

From the directory with the ACT source code, the command:

```
admin/verify.sh
```

builds ACT in the build directory *build_Release_DOUBLE*, including the test programs, and runs all tests twice: once with the explicit SIMD routines for the energies and forces and once without, by setting *ACT_NO_EXPLICIT_SIMD* (see Table 2.1). Since both ways should give the same results within the tolerances of the tests, this checks the explicit SIMD routines against the others. The options of the script are listed in Table 2.2.

The script does not configure the build directories, except the one without vectorization of *--novec*: it uses the *cmake* and *ctest* programs with which a directory was configured, and the MPI launcher found

there. Configure them once, from the directory with the source code, with the compilers and libraries you normally use, for instance:

```
cmake -S . -B build_Release_DOUBLE -DCMAKE_BUILD_TYPE=Release -DACT_DOUBLE=ON
cmake -S . -B build_Release_SINGLE -DCMAKE_BUILD_TYPE=Release -DACT_DOUBLE=OFF
cmake -S . -B build_ASAN_DOUBLE -DCMAKE_BUILD_TYPE=ASAN -DACT_DOUBLE=ON
```

where the second one is needed for *--single* and the third one for *--asan*. Another directory than *build_Release_DOUBLE* can be given with *-b*; the directories of *--single* and *--asan* always have these names.

Table 2.2: Options of admin/verify.sh.

Option	Description
-b DIR	The build directory, by default build_Release_DOUBLE.
-j N	The number of parallel build and test jobs, default 8.
--novec	Also build in DIR_NOVEC without any vectorization, that is with ACT_SIMD=NONE and with vectorization by the compiler turned off, and run the tests there with and without explicit SIMD.
--asan	Also build and test build_ASAN_DOUBLE, with the address sanitizer, which finds errors in the use of memory.
--single	Also build and test build_Release_SINGLE, in single precision, with and without explicit SIMD.
--gpu	Also build and test the GPU build, on a computer with a GPU, where its tests may not be skipped, and run the regression set there as well, see below.
-g DIR	The GPU build directory, by default build_Release_SINGLE_SYCL.
--save-reference	Run the regression set, see below, and keep the results as the reference.
--regressio	Run the regression set and compare it with the reference.
--tolerance T	The largest relative difference from the reference that is accepted, default 0, that is identical results.
--no-tests	Only build and run the regression set.

The script ends with a list of what failed, if anything, and has a nonzero exit status in that case. The output of the build and of the tests is in the files *verify-build.log* and *verify-tests-simd.log* or *verify-tests-nosimd.log* in each build directory, and that of configuring the directory of *--novec* in its *verify-configure.log*. A test that fails is listed with its output in these files.

Which checks to run depends on the change. Before committing a small change, *admin/verify.sh* is usually enough. Before merging a branch, or after changes to the energy and force routines, also check the other builds:

```
admin/verify.sh --novec --asan --single
```

The build without vectorization shows whether a difference comes from vectorization, the address sanitizer finds reading or writing outside arrays and memory leaks, and the single precision build checks the tolerances of the tests in single precision, which is used with a GPU.

After changes to the GPU code, run the script on a node with a GPU with *--gpu* as well. The GPU build directory has to be configured with *-DACT_GPU=SYCL -DACT_DOUBLE=OFF*, see Table 2.1. The tests of the GPU, whose names end in *Gpu*, run as separate *ctest* entries, for instance *ACTForcesTestsGpu*, which *ctest* reports as skipped without a GPU. With *--gpu*, a skipped GPU entry counts as a failure, since then no GPU was found. With *--save-reference* or *--regression*, the regression set is also run in the GPU build, with the polarizable models trained on the GPU, and compared with a reference made with that build (mode *gpu*).

2.2.2 The regression set

The tests check small parts of the code. The regression set checks whole calculations: training of the hydrogen fluoride dimer, of the halgren-PG force field for alcohols and water, of the polarizable alc-pol force field and of bonded parameters for monomers, as well as the second virial coefficient of the BK3, TIP5P and hydrogen fluoride models. These are small versions of the benchmarks of ACT and take about a minute together. They need the input files of the ACTdata repository, by default in *$HOME/Alexandria/ACTdata*, which can be changed with the environment variable *ACT_VERIFY_DATA*: the force fields, molecules and selections in *Performance/train_ff*, the dimer structures in *Performance/b2*, and the force fields *BK3.xml* and *TIP5P.xml* in *LegacyForceFields*. Each calculation is run with 8 MPI processes, with the MPI launcher found when the build directory was configured, so the build needs MPI (*ACT_MPI*, on by default).

The regression set is useful for changes that should not change the results, such as making the code faster or easier to maintain. Before such a change, keep the results as the reference:

```
admin/verify.sh --save-reference
```

and after the change compare with it:

```
admin/verify.sh --regression
```

Both run the regression set twice, with and without explicit SIMD, after building and testing as above unless *--no-tests* is given. For each calculation, the script prints whether all parameters of the trained force field (in *ff_inter.xml*), or all second virial coefficients (in *b2.xvg*), are identical to those of the reference, or else the largest relative difference, for instance:

```
=== Regression set, simd
    bk3-b2           b2.xvg         identical
    halgren-train    ff_inter.xml   identical
    hf-b2            b2.xvg         identical
    hf-train         ff_inter.xml   identical
    ljpc-intra       ff_inter.xml   identical
    pol-train        ff_inter.xml   identical
    tip5p-b2         b2.xvg         identical
```

and the same for *nosimd*, followed by *=== All passed*. A difference is printed as, for instance, *relative difference 3.2e-07*. A calculation fails if the difference is larger than the tolerance (*--tolerance*, default 0), if its output is missing, or if it has a different number of values than the reference. The results are in *verify/simd* and *verify/nosimd* in the build directory, the reference in *verify-reference*. If a calculation fails while saving the reference, the old reference is kept. The output of each calculation is in *out.txt* in its directory, for instance *verify/simd/hf-train/out.txt*. The training uses a genetic algorithm with random numbers, which amplifies the smallest differences in the energies, so the trained force fields can only be compared within the same build and with explicit SIMD either on or off. A change that only reorders floating point operations, for instance, can change the trained force field considerably, while each energy

changes in the last digits only. For such a change, the tests show whether it is correct, and a tolerance can be given, which applies to the trained force fields and the second virial coefficients alike; in practice it is only useful for the latter.

2.2.3 Test coverage

The command:

```
admin/coverage.sh
```

builds ACT in *build_COVERAGE* with the source-based coverage of the clang compiler, runs all tests and reports which lines and functions of ACT they run, leaving out the GROMACS and external code and the tests themselves. It writes three reports in *build_COVERAGE/coverage*:

- *summary.txt*, the coverage of each directory of *src/act* and the files with the most lines that are not run by any test,
- *report.txt*, the coverage of each file, and
- *html/index.html*, the source code with the number of times each line was run, in which code that is not run by any test stands out in red.

This shows where new tests are most needed. The script needs the clang compiler and its tools *llvm-profdata* and *llvm-cov*, which come with Xcode on macOS.

2.2.4 Cleaning up the code with clang-tidy

The build target *tidy* cleans up the code of *src/act* with the *clang-tidy* program of the LLVM project and changes the source files in place. From a configured build directory, the command:

```
make tidy
```

runs *clang-tidy* on all source files of *src/act* that are compiled in that build directory, collects the fixes it proposes and applies them. The checks are listed in *admin/clang-tidy-fix.yaml*; only checks that come with automatic fixes are used. They

- make local variables and references *const* where they are not changed, but only for variables that are not scalars, such as objects, strings, containers and iterators; variables of the built-in types, enumerations and pointers are left as they are (see *--const-scalars* below), as are the parameters of functions, so that no function signatures change,
- avoid needless copies, for instance by using *const auto &* for a variable that is copied from a constant reference, or *emplace_back* rather than *push_back*,
- use *empty()* rather than comparing *size()* with zero, and remove calls of *c_str()* and initializations of strings that are not needed,
- write = *default* for empty constructors and destructors and *override* for virtual functions that override another,
- simplify boolean expressions, use *nullptr*, and add braces around the statement of *if*, *for*, *while* and *else* when they are missing.

The fixes are applied in the layout of the ACT code, with 4 spaces of indentation: *const* is written in front of the type (*const auto it*), aligned declarations stay aligned, braces that are added get lines of their own, and = *default* follows the parameter list. Only the lines that are fixed change, the rest of each file is left as it is. Fixes that overlap with another fix are left for the next run, and fixes that cannot be written in this

layout, such as braces in a function written on one line, are left out; the output lists how many fixes of each check were applied or left. Run *make tidy* until it reports no more fixes.

The clean-up can be limited to some files or directories, relative to the directory with the source code, and the options of Table 2.3 can be given, with two environment variables:

```
make tidy ACT_TIDY_PATHS=src/act/forces
make tidy ACT_TIDY_PATHS="src/act/topology src/act/qgen" ACT_TIDY_ARGS=--dry-
↪run
```

Header files are changed only when they are in the selected files or directories. The script that does the work, *admin/clang_tidy_fix.py*, can also be run directly, with the build directory given as *--build-dir*.

Table 2.3: Options of make tidy, given with ACT_TIDY_ARGS.

Option	Description
--dry-run	Only report how many fixes of each check would be applied, without changing any file.
--const-scalars	Also make variables of scalar types (built-in types, enumerations and pointers) const.
-j N	The number of clang-tidy processes run at the same time, by default the number of cores.

clang-tidy reads how each file is compiled from *compile_commands.json* in the build directory, which *cmake* writes when it configures the build. It therefore only sees the code that is compiled there: code for another configuration, for instance the SYCL code of a build with GPU support or code for single precision, is only cleaned up from a build directory with that configuration. The target needs *clang-tidy* and Python with the *PyYAML* module. Any recent version of *clang-tidy* will do; if it is not in the path, or has another name, give it when configuring:

```
cmake -DACT_CLANG_TIDY=$HOME/miniforge3/envs/tidy/bin/clang-tidy .
```

where, for instance, the conda environment *tidy* was made with:

```
conda create -n tidy -c conda-forge clang-tools clang
```

The package *clang* is needed for the header files of the compiler that *clang-tidy* uses. As with any change, check the result with *git diff*, then build ACT and run the tests, for instance with *admin/verify.sh*, before committing it.

2.2.5 Code testing on GitHub

When a branch is pushed to github four CI (continuous integration) tests are deployed, so-called *github-workflows*. These all include downloading ACT to a test machine, build it and run the tests, all of which should pass. If a single test does not pass the branch cannot be included into the main branch.

The four different tests are

- Double precision build on Linux
- Double precision build on MacOS
- Single precision build on Linux with explicit SIMD
- Double precision build on Linux without explicit SIMD

If a pull-request is made to the main branch, an additional tests will be done, to verify that the manual will build. This test too, has to pass.

Finally, if a branch is merged into main, all tests are run again, and the both the on-line manual and the pdf manual are updated.

2.3 Using the ACT

To use the ACT, you first need to install the ACT. Once you have finished that, please try the command:

```
alexandria -h
```

and:

```
alexandria help commands
```

the output of which is given in Table 2.4 .

Table 2.4: Commands available in the alexandria program.

Com man	Description
b2	Compute second virial coefficient as a function of temperature.
edit_	Manipulate and compare force field files in various ways and test whether reading and writing works.
edit_	Utility to merge a number of molecular property files and a SQLite database. Can also test reading and writing the molecular property file. It can also check the molecular property file for missing hydrogens and for whether it is possible to generate topologies for all compounds. Finally it can generate charges for all compounds.
gen_f	Generate a force field file from a user specification.
gen-top	Generate a molecular topology and coordinates based on structure files. Only inputs for OpenMM can be generated at this point in time.
ge-om-e-try_fl	Deduce bond/angle/dihedral distributions from a set of structures and add those to a force field file.
help	Print help information.
merg	Utility to merge a number of force field files and write a new file with average parameters. Can also write a LaTeX table.
min_	Generate inputs for an energy scan.
nma	Perform normal mode analysis and compute vibrational frequencies and thermochemistry properties.
sim-u-late	Perform a MD simulation and generate a trajectory.
train_	Train a force field to reproduce reference data.

Note that one can get detailed help for the alexandria modules using the -h flag, e.g.:

```
alexandria train_ff -h.
```

In addition to the *alexandria* program, the ACT contains a number of python scripts and utilities (Table 2.5), some of which are needed for the quantum chemistry calculations as well (see Sections *Generating SAPT data* and *Generating single molecule data*).

Table 2.5: Python utilities available in the ACT.

Command	Description
coords2molpro	Read a bunch of structure files containing molecule dimers A-B and write a molprop file.
dimer_scan	Compute dimer potentials for along a user-defined distance between two molecules.
donchev2mo	Convert dimer interactions from the Donchev paper [28] to a molprop file.
gauss2molpr	Read a Gaussian output file from the Alexandria library [17] to a molprop file.
generate_mp	Read and process many Gaussian output files from the Alexandria library [17] and generate a molprop file.
install_act	Script to install the ACT
molselect	Make a selection file based on compounds from the Alexandria Library.
ncia2molpro	Read an xyz file and write a molprop file.
plot_converg	Read an ACT training log file and plot the converge of the parameters as a function of generations in the evolution of the gene pool.
reshuffle_selection	Read a selection file and randomly assign Train status to entries, then write a new selection file.
view_fitness	Visualizes the fitness per generation of a GA/HYBRID by plotting the maximum, minimum, mean, and median, for an example see Fig. 2.1

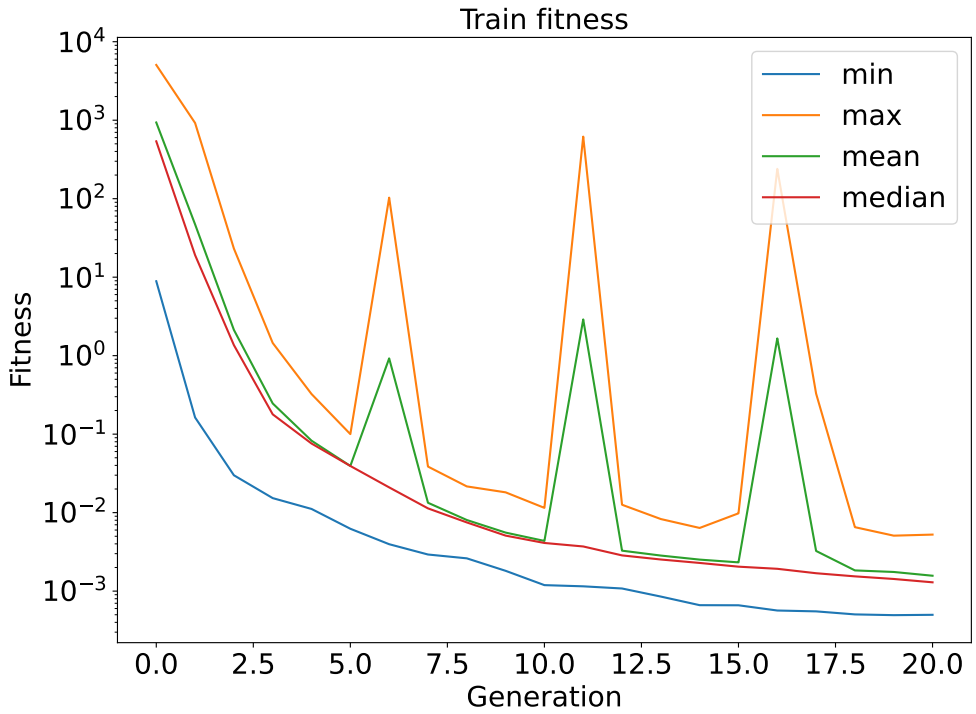


Fig. 2.1: Sample convergence plot from a HYBRID training, generated by the view_fitness script.

2.3.1 ACT File formats

The ACT contains a number of specific file formats, in principle all of them generated by the ACT itself through scripts (Table 2.5) or *alexandria* (Table 2.4). The files are listed below:

- Selection file, with *.dat* extension. List of monomers or dimers and designation on whether they are part of the train or test set. In case of dimers, the two compounds are separated by a hash (#) symbol.:

```
water#ethanol|Test
water#water|Train
ethanol#ethanol|Train
```

- Force field file, with *.xml* extension.
- Molprop file, with *.xml* extension.

2.3.2 Creating a new force field file from scratch

Start by copying the examples directory from the source catalog, and change directory to *examples/NonPol*.

Now, let's see what *alexandria gen_ff* has to offer:

```
alexandria gen_ff -h
```

(to see the output check the help text).

As we see there are many options, but for Alexandria force fields most can remain default. Make a fresh directory and use the one-line script:

./genff.sh

which gives output similar to:

```
There are 62 atom types in the force field with 42 properties.
There are 127 element properties

Thanks for using the Alexandria Chemistry Toolkit.
```

The script prints that it has read and processes a number of files from the ACT installation. These data files can be copied and modified by the user. The files should be relatively simple to understand. Installed files are in the share/act directory. Please note that the force field files use the eXtensible Markup Language that provides a structured way of storing data. Do not edit manually if at all possible.

This force field file gives us something to start with, but it is not complete yet. First, we have to add the possible bond lengths, bond angles etc., and those come from the Alexandria Library (see below). For now, we will use the provided file in XML/alcohol.xml. Thus, we need to run:

```
./geomff.sh
```

which should produce the following output:

```
   Welcome to the Alexandria Chemistry Toolkit

There are 5 molecules in the selection file SELECTIONS/alcohol-monomer.dat.
```

(continues on next page)

(continued from previous page)

```
There are 5 molprops. Will now sort them.
There were 0 double entries, leaving 5 after merging.
There were 5 total molecules before merging, 5 after.
Have generated 34 entries for bond charge correction (SQE algorithm).

Please check output in file geometry_ff.log.
```

This generates a ready-to-optimize force field file, myff2.xml and a log file that is good to inspect. It provides statistics over the geometry of compounds in the input xml file:

```
bond-c3_b~h_b len 108.8 sigma 0.3 (pm) N = 28
bond-c3_b~c3_b len 150.9 sigma 0.6 (pm) N = 7

angle-c3_b~c3_b~c3_b angle 112.1 sigma 0.6 (deg) N = 3
angle-c3_b~c3_b~o3_b angle 108.6 sigma 3.8 (deg) N = 7
```

which informs us, for instance, that there are 28 aliphatic c3-h bonds in the input with an average length of 108.8 $\pm$ 0.3 pm.

2.3.3 Train your first force field

Now you can run the first part of the example training, the intermolecular interactions, using:

```
./run_inter.sh
```

and the adventure has begun! This part will train the van der Waals parameters σ and ϵ (Eqn. (3.18)) as well as the charge distribution width ζ (Eqn. (3.2)) and the electronegativity χ and hardness η. The command will output some text to the terminal:

```
There are 16 threads/processes and 5 parameter types to optimize.
rank: 0/16 nodetype: Master superior: 0
nmiddlemen: 16 nhelper_per_middleman: 0
ordinal: 0 nhelper: 0
middlemen:   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
```

showing the genetic algorithm is using 16 individuals and the code is being run on 16 threads in parallel. Then, at the end you will see something like:

```
There were 39 INTERACTION-Train outliers for Train.
There were 29 INTERACTION-Test outliers for Test.

Please check output in file train_inter.log.
```

encouraging you to inspect the log file. It can be very helpful to check outliers that can help you to find inconsistencies in both the data and the model you are trying to train. After this training instance, a correlation plot *INTERACTION.xvg* of the interaction energies will be produced (Fig. 2.2), as well as one for each energy component, such as *COULOMB.xvg*. With the option *-compound_file* a prefix can be given for the names of these files, for instance *-compound_file inter* gives *interINTERACTION.xvg*. You can use the:

```
./plot-epot.sh
```

script to make the plot yourself as well.

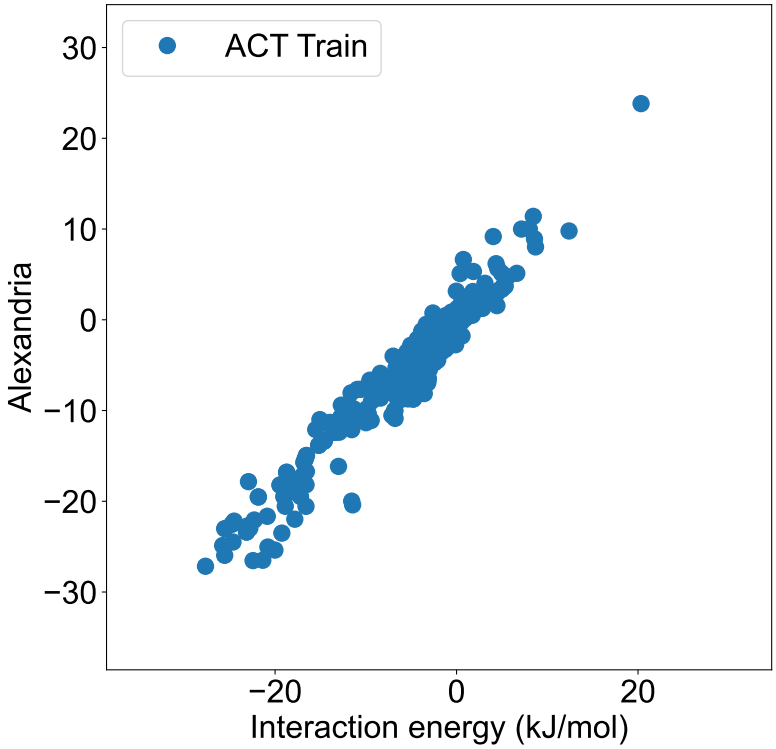


Fig. 2.2: Correlation between interaction energy from SAPT (x-axis) and training (y-axis).

You can inspect the different .xvg output files in the generated subdirectory *inds*. Fig. 2.3 shows how the total deviation from SAPT data, you can recreate the figure using the:

```
./plot-chi2.sh
```

script. Fig. 2.4 shows how well the Gaussian distribution widths converge.

Note that these graphs were made using the plotxvg script that is based on matplotlib.

After training the parameters governing intermolecular interactions, it is time to train the bonded forces. For this, we need to modify the selection of compounds to be monomers, we use the force field file that was trained on SAPT interaction energies (Train-inter.xml) and reference energies come from MP2 calculations.

```
./run_intra.sh
```

will do this training. As targets in this training we use both the intramolecular energies and the forces, however the deviations in the forces are weighted down by a factor of 0.1 because of the magnitude of the numbers. You will get similar output as for the intermolecular training:

```
There were 60 EPOT-Train outliers for Train.

Please check output in file train_intra.log.
```

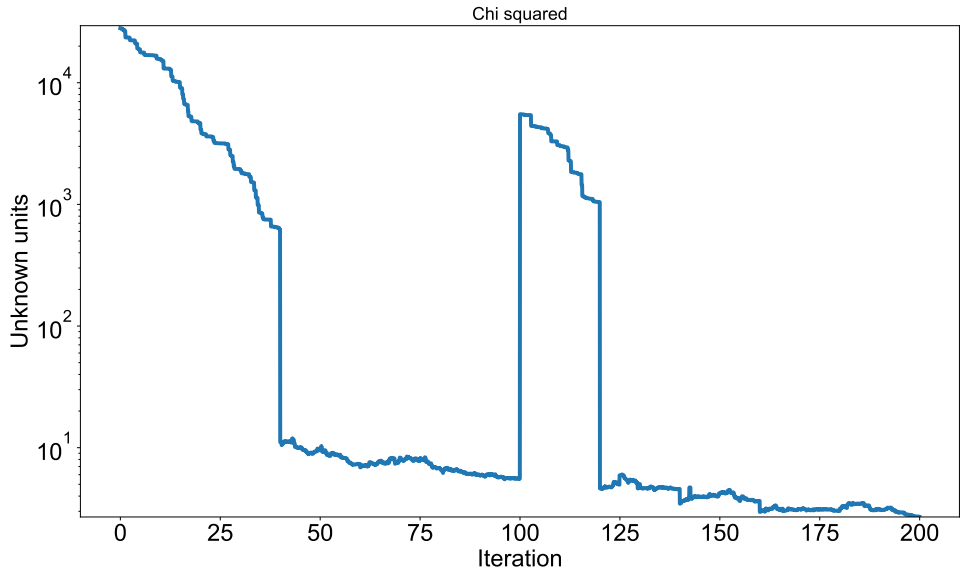


Fig. 2.3: Convergence of the χ^2 fitness value for the first individual in the example training. Note the jump at iteration 100, due to the catastrophe penalizer kicking in.

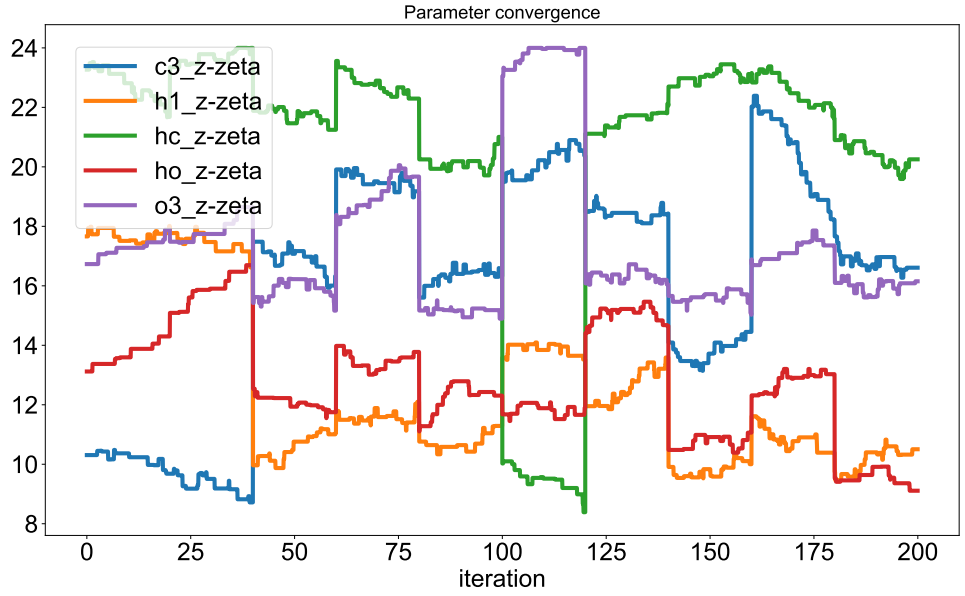


Fig. 2.4: Convergence of the Gaussian distribution widths ζ for five atom types. The jumps in the graphs are due to genetic algorithm performing crossover between individuals.

Again, please inspect the contents of the log file carefully. A quick check of the intramolecular force field can be done by computing frequencies:

```
./run_nma.sh
```

which will perform a normal mode analysis (Sec. *Normal Mode Analysis*) and compute an infrared spectrum (Sec. *Infrared Spectra*) based on your new force field. This should yield a spectrum like the one in Fig. 2.5.

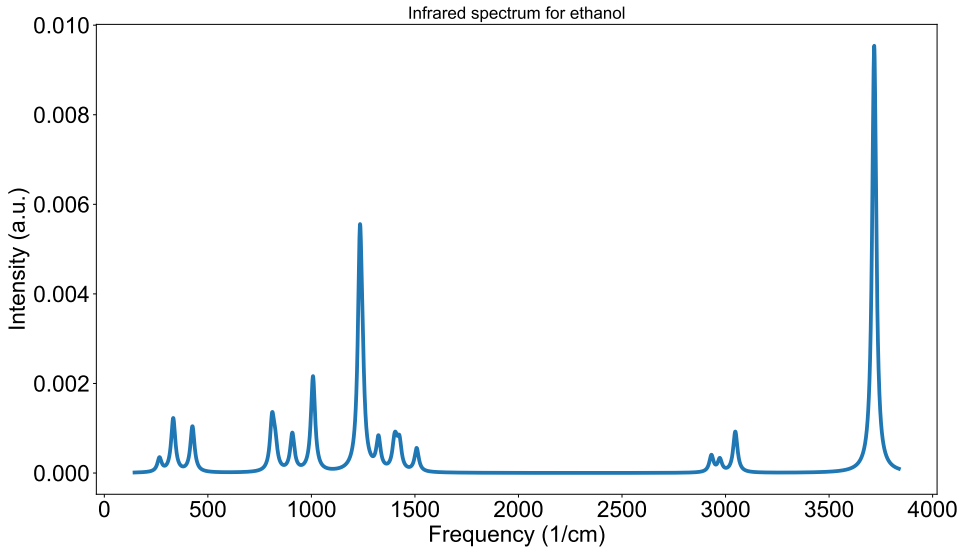


Fig. 2.5: Simulated infrared vibrational spectrum for ethanol based on the force field derived in this tutorial (please compare to Fig. 2.7).

As a final test of your new force field, we can use *alexandria simulate -minimize* to determine the optimized structure of a methanol dimer. For this, run:

```
./minimize.sh
```

and check the output structure, *after_em.pdb*. It should look like Fig. 2.6.

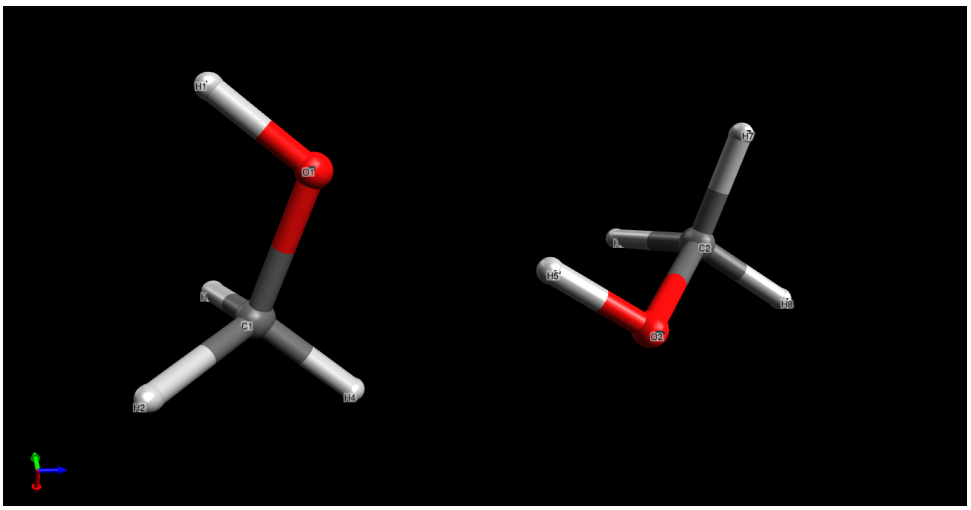


Fig. 2.6: Structure of a methanol dimer after minimization using the force field derived by the ACT in this tutorial. Visualization using Avogadro.

Please make sure to inspect all the scripts provided here to learn about the command line options to ACT modules, but be advised that there are many more options and possibilities.

To design a force field for your own system of choice you will have to provide quantum chemistry data (Sec. *Training Data*), and create selection files (Sec. *ACT File formats*). Most of the steps you have gone through in this tutorial can be adapted to your need. For large trainings it is useful to study the parallellization features of the ACT (Sec. *Running training using parallel processing*).

2.3.4 Is my training hitting a wall?

To make training efficient, it is good to limit the search space. In some cases we can use chemical intuition to estimate the search space. For instance, we know that polarizability should be a positive number and that it is highly element dependent. For hydrogen, a likely range is $0.1 < \alpha < 0.5$ Å^3. Such boundaries are implemented in the force field file like this:

```
<parameterlist identifier="h_s">
  <parameter type="alpha" unit="Angstrom3" value="0.46"
  uncertainty="0.008" minimum="0.1" maximum="0.5"
  ntrain="300" mutability="Bounded" nonnegative="yes"/>
</parameterlist>
```

this example is after an optimization where the optimal value ended up being 0.46, based on a dataset comprising 300 hydrogen atoms. Note the nonnegative=”yes” which will prevent alexandria tools from making this parameter negative ever.

For many cases, we do not have an {em a-priori} intuition of what the values should be and we make a guess. Then, after a series of optimizations, we can check whether the bounds are OK, using an alexandria tool (assuming you have generated and trained a polarizable force field *.xml*):

```
alexandria edit_ff -ff ff.xml -ana EEM
<snip>
POLARIZATION n2_s alpha at maximum 1.5
POLARIZATION n4_s alpha at minimum 0.6
BONDCORRECTIONS c2_~c3_z hardness at minimum -0.4
BONDCORRECTIONS c2_z:n2_z hardness at minimum 0
BONDCORRECTIONS c3_z~c3_z hardness at minimum -0.4
```

Some of the parameters are indeed hitting the wall. To adjust the parameters we use the same tool again:

```
alexandria edit_ff -ff ff.xml -o new.xml -stretch -p alpha
<snip>
Thanks for using the Alexandria Chemistry Toolkit.
```

and then we check the resulting force field file in the same manner as before:

```
alexandria edit_ff -ff new.xml -ana EEM
<snip>
BONDCORRECTIONS c2_z~c3_z hardness at minimum -0.4
BONDCORRECTIONS c2_z:n2_z hardness at minimum 0
BONDCORRECTIONS c3_z~c3_z hardness at minimum -0.4
```

Now the polarizability minimum (for n4_s) and maximum (for n2_s) have been stretched. The same can now be done for hardness.

You can also do the reverse. When you are confident that the optimization is close to the final result, but want to randomize again without starting from scratch, you can set new minimum and maximum values for the parameters using, e.g.:

```
alexandria edit_ff -ff new.xml -o new.xml -p bondlength -limits 0.98
```

please check the on-line help (*alexandria edit_ff -h*) for more information.

When a force field is read, values of trained parameters that are outside their bounds are set to the nearest bound, and of a parameter that is defined more than once the first definition is used. The programs then warn once, in the log file or on the terminal. To see which parameters these are, and to write a force field without these problems, use:

```
alexandria edit_ff -ff ff.xml -validate -o fixed.xml
```

This also lists problems that have to be corrected by hand: values outside the bounds of parameters that may not be changed, a minimum larger than the maximum, and negative values of parameters that are marked as non-negative (attribute *nonnegative* in the file). If the attribute is wrong, for instance for bond energies, which are negative, it can be cleared with:

```
alexandria edit_ff -ff fixed.xml -p bondenergy -nonneg no -o fixed2.xml
```

where *-a* limits the change to some identifiers. Parameters of particle types other than the charge are always non-negative.

2.3.5 Running training using parallel processing

The training process is heavy in computer time. Therefore, it has been parallellized using the MPI library. Since this is a prerequisite for compiling the ACT, you likely have it installed if you got this far. In fact this is used in the example scripts, where N is the number of cores you have available. Please consult with your cluster manager if in doubt. Clusters using the Slurm queueing system may need to use *srun* instead of *mpirun*. As far as we have tested [1], the code is quite efficient down to 4-5 molecules per core. In the above example there are 10 compounds in the training set, such that it is not worthwhile to use more than 2-3 cores, but do experiment with the number of cores.

2.4 Getting the best performance from ACT

Training a force field takes most of the computer time spent with ACT, so this chapter is mainly about *alexandria train_ff*. The run times quoted are examples from our own tests. The best settings depend on the computer and on the training set, so the most useful advice is to do a short test run with a few settings before a long one, see *Measuring and comparing the run time*.

2.4.1 Building ACT for speed

- Build in single precision, with *-DACT_DOUBLE=OFF*. The explicit SIMD routines then compute twice as many interactions per instruction, and a GPU can only be used in single precision builds. Energies are still summed in double precision.
- On a cluster, set *ACT_SIMD* to the SIMD instructions of the compute nodes if you compile on a head node with another processor, and give the compiler the instructions of the compute nodes in *CMAKE_CXX_FLAGS*, for instance *-march=native* when compiling on a compute node.
- For a GPU, build with *-DACT_GPU=SYCL*, see the installation chapter.

The section *Code used* at the start of each log file tells which SIMD instructions, which precision and which GPU the program uses. Check it once on each new computer.

2.4.2 How train_ff uses the processes

One process, the master, runs the genetic algorithm. How the others work depends on the size of the population (*-pop_size*):

- **Population no larger than the number of processes.** The processes are divided into one group per individual, a middleman and its helpers, and the molecules of the training set are divided over the processes of a group. Each individual needs the same number of processes, so use a multiple of the population size; otherwise the processes left over are idle, which *train_ff* warns about. Splitting the molecules over helpers stays efficient down to 4-5 molecules per process [1].
- **Population larger than the number of processes.** This is possible with the GA and HYBRID optimizers. Each process except the master is then a middleman that computes one individual at a time, and gets the next one from the master when it is done. The master itself computes no individuals.

In the second case, a generation ends when the last individual is done. If the number of middlemen does not divide the population, the last round of individuals keeps only some of the middlemen busy, and the others wait. Choose the number of processes such that every middleman computes the same number of individuals: one more than a divisor of the population. For example, a training with a population of 256 on a node with 288 cores (arrhenius, GH200) took:

Table 2.6: Run time of a training with a population of 256, by the number of processes.

Processes	Middlemen	Individuals per middleman	Run time (s)
64	63	4 or 5	89.1
65	64	4	82.0
129	128	2	40.9

With 64 processes, 4 middlemen compute a fifth individual in each generation while the 59 others wait. All three runs gave the same result, which does not depend on the number of processes. In general, use as many processes as there are physical cores, one more than a divisor of the population. At the start, *train_ff* notes on the terminal and in the log file when the middlemen would share the individuals unevenly, how long they would wait on average, and which numbers of processes avoid it.

2.4.3 Processor or GPU?

The GPU computes the training only for **polarizable** force fields, in single precision builds, with *train_ff -gpu_resident* (the default). Each process then keeps its molecules on the GPU, which computes the interactions, the virtual sites, the relaxation of the shells and the deviations from the reference data, and the charges if they are generated with SQE. The relaxation of the shells repeats the force calculation many times in each evaluation, which is expensive on a processor, and this is where the GPU gains most.

For force fields **without shells**, the processor computes everything, also in a build with GPU support. Such a training takes the same time with and without *-nogpu*. Without shells, ACT only recomputes the molecules that a changed parameter affects, and of a dimer only the interactions between its two molecules, so one evaluation is short: about a hundred microseconds on a core for a training set of small dimers in our tests. That is shorter than the time needed just to start the work on a GPU and wait for the result, so for these force fields, use all cores of the node rather than the GPUs.

2.4.4 Training on GPUs

- **Several processes per GPU.** Each process waits for the GPU part of the time, so several processes per GPU keep it busy. On NVIDIA GPUs, start the multi-process service in the job script when several processes share a GPU, see the installation chapter. The service takes at most 48 or 60 processes per GPU, depending on the NVIDIA driver; the ranks beyond that compute without the GPU.
- **-lockstep.** With a population larger than the number of processes, *-lockstep k* gives each middleman *k* individuals at once. The GPU runs the MCMC steps of these individuals together, in the same calculations: each step takes as many operations on the GPU, and as much waiting for it, for *k* individuals as for one. Fewer processes are needed, *pop_size/k* + 1, which also shortens the start-up of the GPU in each process. With a population of 128 on a node with eight AMD MI250X GCDs (dardel) and on a node with four NVIDIA GH200 (arrhenius) we measured:

  Table 2.7: Wall time of the master in s of short trainings with a population of 128, by *-lockstep*, with *-shell_keep*.

Computer	Training	*-lockstep 1* 129 processes	*4* 33	*8* 17	*16* 9
dardel, 8 MI250X GCDs	HHpol	63.3	32.4	**21.0**	28.5
	BFdb	243.1	167.6	**156.4**	172.8
arrhenius, 4 GH200	HHpol	101.0	34.8	23.5	**22.0**
	BFdb	156.5	84.6	76.2	**75.5**

  HHpol is a small training set (10 generations of 10 MCMC iterations), and most of what it gains from *-lockstep 8* is start-up time. BFdb is a large one (5 generations of 5 iterations); its GPUs are busy already with *-lockstep 4*, and computing more individuals at once does not make them faster. With 9 processes on 8 GCDs the GPUs have too little to do. *-lockstep 8* is a good choice on both computers. The result does not depend on *-lockstep*, except for where the relaxation of the shells starts, see *-shell_keep*, and through which middleman computes which individual: with the same seed, the BFdb runs above gave the same force field for all values, the HHpol runs did not.

  Without the GPU, or for a force field without shells, *train_ff* notes that *-lockstep* has no effect and hands out one individual at a time, which shares them more evenly over the middlemen: in a training with 256 individuals and 64 processes on dardel this took 1062 s, against 1601 s with groups of 4 individuals, since then one middleman computed a second group in each generation while the others waited. With *-lockstep*, the number of middlemen should divide the number of groups, see the note at the start. Each individual that a process computes at once also needs a copy of the molecules on the GPU, about 43 MB for BFdb; if the GPU runs out of memory, *train_ff* stops with a message that says so.

- **-shell_keep.** By default the relaxation of the shells starts from where the shells were the previous time, which needs fewer iterations when the parameters changed little. On an RTX A4500 this took a training from 179 s to 146 s. The relaxed shells differ within the tolerance on the forces, so the results differ slightly from those of *-noshell_keep*. In the trainings of Table 2.7, which used *-shell_keep*, the shells needed about 25% fewer iterations than with *-noshell_keep*; BFdb took 7% less time on dardel (156 s against 167 s with *-lockstep 8*), and on arrhenius, where the relaxation is a smaller part of the time, the difference was 1-2%. Since where the relaxation starts depends on which individuals a process computed before, the result of a training with *-shell_keep* depends on the number of processes and on the computer. With *-noshell_keep*, the HHpol trainings reached the same χ^2 on dardel and on arrhenius for each value of *-lockstep*, and with *-lockstep 4* the same force field; with *-shell_keep* they did not. Both are correct within the tolerance on the forces; use *-noshell_keep* when runs must be reproduced exactly.

- Molecules that a changed parameter does not affect are not computed again; this happens automatically, also on the GPU.

The best value of *-lockstep* depends on the GPU, on the number of GPUs per node and on the training set. Start with *-lockstep 8* and *pop_size/8* + 1 processes, and compare with 4 and 16 in a short run.

2.4.5 Measuring and comparing the run time

At the end of the log file, *train_ff* prints how much time each part of the program took, summed over all processes and as an average per process for the master and for the middlemen, preceded by the wall time of the master. Useful lines are:

- *MPI waiting in Middleman loop*: middlemen waiting for work, for instance at the end of each generation. A large share means that the processes are not used evenly, see Table 2.6.
- *MPI waiting in Genetic algorithm*: the master waiting for the middlemen, which is normal.
- *GPU initialization* and *Startup*: one-time costs. In short test runs they can be a large part of the time; leave them out when comparing settings.
- *GPU resident relaxation*, *GPU resident evaluation* and *Charge generation*: the computation itself.

When comparing settings, keep in mind that:

- The run time of the same training on a GPU can vary by about 10% between runs. Compare settings in one job, run them alternately, and repeat them.
- The training may stop before *-max_generations* when the fitness of the test set no longer improves (*-max_test_generations*). Runs that differ in the last digits, for instance on different computers, may then run a different number of generations. Compare the time per generation, from the lines *Generation n/N* in the log file.
- *Wall time of the master* starts after MPI has started. On some clusters starting MPI takes long, for instance about 0.7 s per process on the GH200 nodes of arrhenius, so the job takes longer than this time.

2.5 MD Simulations with Alexandria Force Fields

2.5.1 Using ACT simulate

The command *alexandria simulate* will perform a simulation in the gas phase, i.e. without periodic boundary conditions. This obviously limits the usefulness but the OpenMM software can be used for condensed-phase simulations using ACT force fields [1, 25]. The *alexandria simulate* utility is aimed for validation of the potentials, for instance by evaluating whether the total energy is conserved, it can be verified that the force is the derivative of the potential. The integration algorithm and the relaxation of shells are described in *Shell Relaxation and Integration*.

The energy of a molecule or small cluster can be minimized using *alexandria minimize*:

```
alexandria minimize -ff actff.xml -f coords.pdb
-charges mp2.xml -c afterem.pdb -g minimize.log
```

where *actff.xml* is the force field file, *coords.pdb* is a structure file (*xyz* and *sdf* are supported as well). The *-charges mp2.xml* option indicates a molprop file (database) with monomeric (optimized) structures used for generating charges. The files *afterem.pdb* and *minimize.log* are output files. The minimization algorithm is selected with *-minalg*, atoms can be frozen using an index file with *-freeze* and bond lengths or angles can be constrained with *-constrain*, see *Constraints*. The same minimization can be done

using *alexandria simulate -minimize*, which can in addition run a simulation starting from the minimized structure.

2.5.2 The ACT-OpenMM interface

The OpenMM [29] software in its native form is controlled from Python scripts. This allows users great control over the simulations. OpenMM allows to specify user-defined energy functions, which we use to implement both Gaussian-distributed charges [19, 30] and many Van der Waals potentials [25] (see Section *Energy Function*). To facilitate this, a special python code was implemented that makes it relatively easy for the user to run simulations and minimization. The first step is to convert an ACT force field file (*actff.xml}) to one compatible with OpenMM (*openmmff.xml*):

```
alexandria gentop -ff actff.xml -openmm openmmff.xml
-charges mp2.xml -db "water methanol"
```

with two additional arguments. First, flag *-charges mp2.xml* indicates a molprop file (database) with monomeric (optimized) structures used for generating charges and, second, the flag *-db "water methanol"* instruct the code to produce an OpenMM topology for those compounds. If one or more of the compounds is not present in the database, a warning will be issued.

Each compound becomes a residue in the OpenMM force field file. With the flag *-c conf.pdb*, gentop writes a PDB file with the same residue and atom names, such as C1, H2 and so on. The residue names can be given with *-resnames "water=SOH methanol=MOH"*, or *-resnames "SOH MOH"* for the compounds in the order of the input. A name has at most four letters or digits, and cannot be a residue that OpenMM knows itself, such as HOH or SOL. Compounds without a name get one made from their IUPAC name. A simulation box made from these PDB files, for instance with packmol, should keep the residue and atom names. Then the ACT python interface tells OpenMM the bonds of the residues from the force field file, and the PDB file does not need CONECT records.

2.5.2.1 MD simulations using OpenMM

It is easy to perform simulations based on an Alexandria force field. For this, we rely on the OpenMM software that you need to install separately. Then you can use the ACT python interface to OpenMM, as in this simple script, that we will call run.py:

```
#!/usr/bin/env python3

from act_openmm import ActOpenMMSim

sim = ActOpenMMSim(pdbfile="file.pdb",
                   datfile="dimer.dat",
                   xmlfile="ff.xml",
                   txtfile="output2.txt",
                   verbose=True)
sim.run()
sim.log_to_xvg("energy.xvg", [ "Potential Energy (kJ/mole)"])
sim.log_to_xvg("temperature.xvg", [ "Temperature (K)" ])
sim.log_to_xvg("density.xvg", [ "Density (g/mL)" ])
```

Here, we first instantiate an ActOpenMMSim object, and pass it a structure file *file.pdb*, a simulation parameter file *dimer.dat* and a force field file *ff.xml*. We also instruct the code that output should be written to *output2.txt* and that we want a lot of information in that file. Then we can run it, that's all! The last three lines are for convenience of plotting the results. We can run this script using:

```
python ./run.py
```

or submitted to a cluster, preferably with GPUs available.

2.5.2.2 Energy Minimization using OpenMM

An energy minimization of the input structure can be done using this script:

```
#!/usr/bin/env python3

from act_openmm import ActOpenMMSim

sim = ActOpenMMSim(pdbfile="file.pdb",
                   datfile="dimer.dat",
                   xmlfile="ff.xml",
                   txtfile="output2.txt",
                   verbose=True)
sim.setup()
sim.minimize()
sim.write_coordinates("final.pdb")
```

which is run here using the same flags as the simulation. Note that details about simulations and minimization can be specified in the *dimer.dat* file.

2.6 Predicting Molecular Properties

The ACT can be used to perform MD simulations of clusters in the gas-phase using the *alexandria simulate* command. This module includes the possibility to perform energy minimizations with the flag *-minimize*. For simulations employing periodic boundaries the OpenMM package [29] should be used instead. For more details about MD simulations, see Section *MD Simulations with Alexandria Force Fields*.

Below we describe some of the properties that can be computed using the ACT.

2.6.1 Electrostatic Potential

The charge distribution ρ of a molecule is determined by the nuclear position of the N atoms at positions $\mathbf{x}$ and the electron density $n(\mathbf{r})$. The molecular electrostatic potential (MEP) at a point in space r' is thus given by

$$\Phi(\mathbf{r}') = \frac{1}{4\pi\varepsilon_0}\left[\sum_{i=1}^{N}\frac{z_i}{\|\mathbf{x}_i - \mathbf{r}'\|} - \int\frac{n(\mathbf{r})}{\|\mathbf{r} - \mathbf{r}'\|}\mathrm{d}r\right] \tag{2.1}$$

where N is the number of atoms, z_i are the nuclear charges, ε_0 is the permittivity of vacuum and integration is over the entire space. The minus sign before the integral is due to the negative charge of electrons.

Accurate knowledge of the MEP contributes to, for example, the understanding of interactions and function of biological macromolecules in solution [31]. For a molecule in the gas phase, Eqn. (2.1) can be evaluated using density functional theory and wave function quantum chemistry, albeit at a significant computational cost. Databases of such calculations for small molecules are available to facilitate reuse [8]. For large condensed-phase systems, however, it is common to apply classical force fields, where electrons are not taken into account explicitly. Instead, effective partial charges on atoms are used. The

electronic degrees of freedom, charge polarization, is sometimes taken into account through induced point dipoles and higher electrostatic moments, or by using a core-shell model [32, 33, 34]. For additional background we refer to some excellent reviews [35, 36, 37].

The MEP can be used as a target in model development in the ACT, however we recommend against that for both fundamental and practical reasons [38].

2.6.2 Electrostatic Moments

If point $\mathbf{r}'$ in Eqn. (2.1) is outside the distribution of electron density and $\mathbf{r}' >> \mathbf{r}$, the electrostatic potential can be evaluated through the Taylor expansion of $|\mathbf{r} - \mathbf{r}'|^{-1}$ [39]:

$$\frac{1}{|\mathbf{r}' - \mathbf{r}|} \approx \frac{1}{r} + (\hat{\mathbf{r}} \cdot \mathbf{r}) \frac{1}{r^2} + \frac{1}{2} \left[3 (\hat{\mathbf{r}} \cdot \mathbf{r})^2 - r^2 I \right] \frac{1}{r^3} + \cdots \tag{2.2}$$

where $\hat{\mathbf{r}} = \mathbf{r}'/r$ and I is the identity matrix. By inserting Eqn. (2.2) into Eqn. (2.1), we get

$$\Phi(\mathbf{r}') \approx \frac{1}{4\pi\epsilon_0} \frac{Q}{r} + \frac{\boldsymbol{\mu_0}}{r^2} + \frac{\boldsymbol{\Theta_0}}{r^3} + \cdots$$

Q is the monopole moment, sometimes called the zeroth moment of the molecular electron density. In principle this is the total charge of the molecule. In what follows we combine both the atomic cores and the electron density into $n(r)$. Then, Q is given by

$$Q = \int n(\mathbf{r}) d\mathbf{r}$$

$\boldsymbol{\mu_0}$ is the vector of the permanent dipole moment, which measures the polarity of the molecular electron density. $\boldsymbol{\mu_0}$ is given by

$$\boldsymbol{\mu_0} = \int n(\mathbf{r}) (\hat{\mathbf{r}} \cdot \mathbf{r}) \, d\mathbf{r}$$

$\boldsymbol{\Theta_0}$ is the tensor of the permanent quadrupole moment, which exhibits the deviation of the distribution of the molecular electron density from spherical symmetry. $\boldsymbol{\Theta_0}$ is given by

$$\boldsymbol{\Theta_0} = \frac{1}{2} \int n(\mathbf{r}) \left[3 (\hat{\mathbf{r}} \cdot \mathbf{r})^2 - r^2 I \right] d\mathbf{r}$$

The quadrupole tensor can be written as a traceless 3×3 matrix in the Cartesian coordinate if one writes $(\hat{\mathbf{r}} \cdot \mathbf{r})^2$ as:

$$(\hat{\mathbf{r}} \cdot \mathbf{r})^2 = \hat{\mathbf{r}} \cdot (\mathbf{rr}) \cdot \hat{\mathbf{r}}$$

where $\mathbf{rr}$ is the outer product of vector $\mathbf{r}$ with itself. This results in a matrix that can be written in terms of the Cartesian components of the vector $\mathbf{r}$ as follows:

$$\mathbf{rr} = \begin{bmatrix} x^2 & xy & zx \\ yx & y^2 & yz \\ zx & zy & z^2 \end{bmatrix}$$

Finally, we get

$$\frac{1}{2} \left[3 (\hat{\mathbf{r}} \cdot \mathbf{r})^2 - r^2 I \right] = \hat{\mathbf{r}} \cdot \frac{1}{2} \begin{bmatrix} 3x^2 - r^2 & 3xy & 3zx \\ 3yx & 3y^2 - r^2 & 3yz \\ 3zx & 3zy & 3z^2 - r^2 \end{bmatrix} \cdot \hat{\mathbf{r}} \tag{2.3}$$

2.6.3 Polarizability

The shape of the molecular electron density changes when it interacts with an external electric field; hence, the total energy of the molecule changes. The static response of a molecule to a homogeneous external electric field, ($\mathbf{F}$), can be studied by expanding its energy in a Taylor series [40, 41]:

$$E\left(\mathbf{F}\right) = E\left(\mathbf{0}\right) + \frac{\partial E}{\partial \mathbf{F}}\bigg|_{\mathbf{F}=0} \mathbf{F} + \frac{1}{2}\frac{\partial^2 E}{\partial \mathbf{F}^2}\bigg|_{\mathbf{F}=0} \mathbf{F}^2 + \frac{1}{6}\frac{\partial^3 E}{\partial \mathbf{F}^3}\bigg|_{\mathbf{F}=0} \mathbf{F}^3 + \cdots$$

where

$$-\frac{\partial E}{\partial \mathbf{F}}\bigg|_{\mathbf{F}=0} = \mu_{\mathbf{0}}$$

$$-\frac{\partial^2 E}{\partial \mathbf{F}^2}\bigg|_{\mathbf{F}=0} = \alpha$$

$$-\frac{\partial^3 E}{\partial \mathbf{F}^3}\bigg|_{\mathbf{F}=0} = \beta$$

where $\mu_{\mathbf{0}}$ is the vector of permanent dipole moment, α is the tensor of polarizability, which is the linear part of the response of the molecular electron density with respect to the external electric field, and $\boldsymbol{\beta}$ is the first hyperpolarizability [40].

Instead of expanding the energy, we can expand the dipole moment of a molecule in an external electric field [41], written as

$$\mu = \mu_0 + \alpha\mathbf{F} + \frac{1}{2}\beta\mathbf{F}^2 + \cdots$$

where $\alpha\mathbf{F}$ gives the vector of induced dipole moment, $\mu_{\mathbf{1}}$ [40]:

$$\mu_{\mathbf{1}} = \alpha\mathbf{F}$$

that can be written in matrix form as

$$\begin{bmatrix} \mu_x \\ \mu_y \\ \mu_z \end{bmatrix} = \begin{bmatrix} \alpha_{xx} & \alpha_{xy} & \alpha_{xz} \\ \alpha_{yx} & \alpha_{yy} & \alpha_{yz} \\ \alpha_{zx} & \alpha_{zy} & \alpha_{zz} \end{bmatrix} \begin{bmatrix} F_x \\ F_y \\ F_z \end{bmatrix} \tag{2.4}$$

From the polarizability tensor the polarizability isotropy [41, 42],

$$\bar{\alpha} = \frac{(\alpha_{xx} + \alpha_{yy} + \alpha_{zz})}{3}$$

and the polarizability anisotropy [42],

$$\Delta\alpha = \sqrt{[(\alpha_{xx} - \alpha_{yy})^2 + (\alpha_{xx} - \alpha_{zz})^2 + (\alpha_{yy} - \alpha_{zz})^2 + 6(\alpha_{xy}^2 + \alpha_{xz}^2 + \alpha_{yz}^2)]/2}$$

can be calculated.

2.6.4 Normal Mode Analysis

Please note that the text below is largely taken (with permission) from a paper by Henschel *et al.* [43].

The ACT contains the *alexandria nma* tool that performs a normal mode analysis to determine the vibrational frequencies of a compound. Vibrational frequencies are required to compute the IR spectra and

thermochemistry of molecules. The normal modes of molecular vibrations can be obtained by eigenvalue decomposition of the Hessian matrix, whose elements are the second derivatives of the energy with respect to the atomic coordinates q.

$$H_{ij} = \frac{\partial^2 E}{\partial q_i \partial q_j}$$

where i and j run from 0 to $N-1$, where N is the number of atoms in the molecule. If virtual sites v are used, for example, to model the σ-hole for halogen atoms, the energy, E, depends on the positions of both atoms and virtual sites; that is, $E = E(q_0, ..., q_{N-1}, v_0, ..., v_{M-1})$, where the positions of the M virtual sites, v, in the compound are a function of the atomic coordinates q.

The Hessian is computed numerically—the N atoms are moved independently in all three spatial dimensions, and the forces are computed. From these forces, the second derivative of the energy is then evaluated numerically. Note that the positions of the virtual sites are updated before each force calculation, which means that their influence on the Hessian is taken into account explicitly when computing H.

2.6.4.1 Infrared Spectra

For the calculation of a full IR spectrum, in addition to the vibrational frequencies, the intensities and the line shapes are required.

In case of the quantum chemical calculations both the eigenfrequencies and the corresponding IR intensities are produced by default when a frequency calculation is requested in, for instance, the Gaussian software [44]. The frequencies for about 5000 compounds are available from the Alexandria library [17] at the B3LYP/aug-cc-pvtz level of theory [45, 46, 47, 48].Details of the quantum chemical calculations from which the frequencies were obtained have been presented previously [16, 17].

For the force field calculations, the intensities I_n were derived from the transition dipole derivatives:

$$I_n = \sum_{k=1}^{3} \left(\frac{\partial p_k}{\partial Q_n} \right)^2 \tag{2.5}$$

where k iterates over cartesian dimensions, p is the dipole moment of the molecule, and Q_n the normal coordinate n. In order to take into account virtual sites v we note that

$$p_k = p_k(q_0, ..., q_{N-1}, v_0, ..., v_{M-1})$$

and rewrite equation (2.5) as:

$$I_n = \sum_{k=1}^{3} \left(\frac{\partial p_k}{\partial q_s} \frac{\partial q_s}{\partial Q_n} \right)^2$$

where s iterates over the N atomic coordinates. To make the calculation of intensities practical, the numerical derivative of the dipole moment with respect to the atomic coordinates $\frac{\partial p_k}{\partial q_s}$ is stored in a text file during the normal mode analysis and finally we note that the term $\frac{\partial q_s}{\partial Q_n}$ corresponds to one over component s of eigenvector n.

As an example, an infrared spectrum can be generated using the OPLS2020 force field [49] and a molprop file using:

```
alexandria nma -ff OPLS2020 -charges OPLS2020-charges -db ethanol -ir ir-
↪ethanol
```

yielding the spectrum in Fig. 2.7.

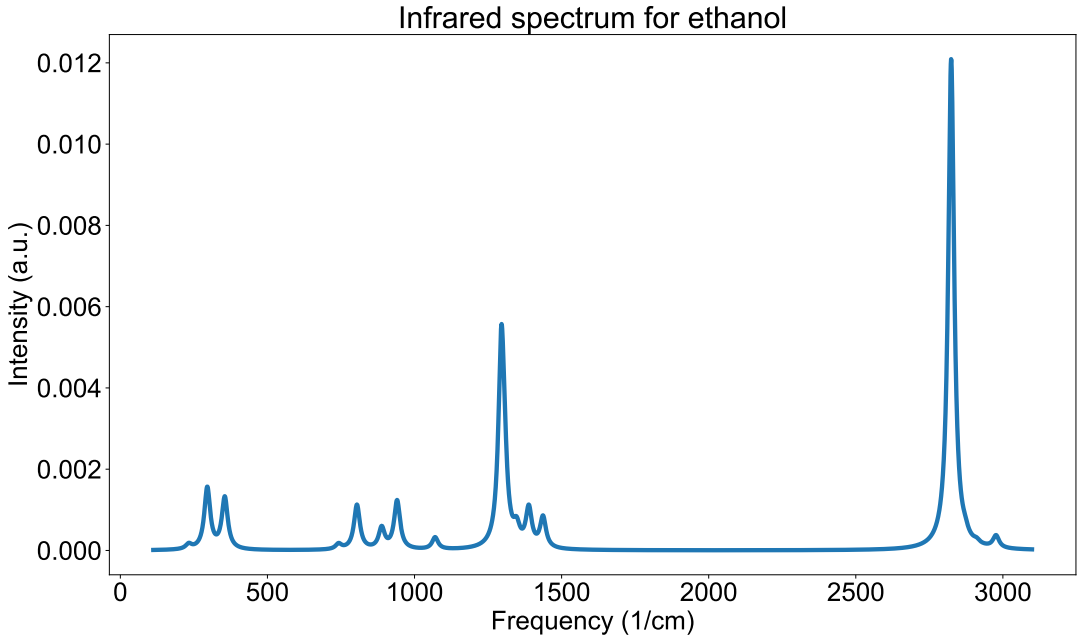


Fig. 2.7: Simulated infrared vibrational spectrum for ethanol, based on the OPLS2020 force field.

2.6.4.2 Thermochemistry

The Canonical ensemble $Q(N,V,T)$ can be used to compute the molecular Internal energy E ((2.6)), the standard Entropy S^o ((2.7)), and the Heat-Capacity C_v ((2.8)) at constant volume, using

$$E = RT^2 \left(\frac{\partial lnQ}{\partial T}\right)_{N,V} \tag{2.6}$$

$$S^o = RlnQ + RT \left(\frac{\partial lnQ}{\partial T}\right)_{N,V} \tag{2.7}$$

$$C_v = 2RT \left(\frac{\partial lnQ}{\partial T}\right)_{N,V} + RT^2 \left(\frac{\partial^2 lnQ}{\partial T^2}\right)_{N,V} \tag{2.8}$$

where R is the ideal gas constant and T the absolute temperature. For an ideal gas, $Q(N,V,T)$ can be decomposed into Partition-Function of different degrees of freedom: electronic (el), translational (tr), rotational (rot) and vibrational (vib) motions. Therefore, for a molecular ideal gas, $Q(N,V,T)$ can be expressed as:

$$Q(N,V,T) = \frac{(q_{el}q_{tr}q_{rot}q_{vib})^N}{N!} \tag{2.9}$$

The Rigid-Rotor and the Quantum-Harmonic-Oscillator can be used to approximate the contribution of the rotational and vibrational motions. The partition function of a rigid rotator is defined as

$$q_{\mathrm{rot}} = \frac{T}{\sigma\Theta_{\mathrm{rot}}}$$

where σ is the symmetry number, Θ_{rot} the rotational temperature defined as

$$\Theta_{\mathrm{rot}} = \frac{h^2}{8\pi^2 \mathrm{I} k_\beta}$$

where I is the moment of inertia, k_β the Boltzmann constant, and h the Planck constant. The partition function of a quantum harmonic oscillator is defined as

$$q_{\mathrm{vib}} = \frac{e^{\frac{-\beta h\nu}{2}}}{1 - e^{-\beta h\nu}}$$

where ν is the vibrational frequency of the oscillator. If we define the vibrational temperature as $\Theta_{\mathrm{vib}} = \frac{h\nu}{k_\beta}$, then we will have

$$q_{\mathrm{vib}} = \frac{e^{\frac{\Theta_{vib}}{2T}}}{1 - e^{\frac{\Theta_{vib}}{T}}}$$

Applying Eqn. (2.9) to Eqn. (2.6) followed by the multiplication rule in logarithm yields:

$$E = E_{tr} + E_{rot} + E_{vib}$$

and similarly,

$$C_v = C_{tr} + C_{rot} + C_{vib}$$

$$S^o = S_{tr} + S_{rot} + S_{vib}$$

Thermochemical properties are computed automatically by the *alexandria nma* command. For more details, please see Van der Spoel *et al.* [13].

The *alexandria nma* used above to generate the infrared spectrum in Fig. 2.7 computes the thermochemical variables as well (Table 2.8). Information on calculation of the enthalpy of formation will be pubslished in the near future.

Table 2.8: Thermochemical values for ethanol at 298.15 K based on the OPLS2020 force field.

Property	Experiment	OPLS2020
S^o (J/mol K)	281 [50]	273.0
C_v (J/mol K)	57 [16]	62.2

2.6.5 Second Virial Coefficient

The second Virial-Coefficient is the second term in the Virial-Expansion that describes the deviation from the ideal gas law for real gases:

$$\frac{P}{RT\rho} = A + B_2(T)\rho + C_3(T)\rho^2 + ...$$

with P the pressure, R the gas constant, T the temperature and ρ the density.

$B_2(T)$ is a useful property gauging interactions in the gas phase because experimental values are available for close to 2000 compounds as a function of temperature. It is computed from an integral weighting the

interaction between two molecules over three-dimensional space. The theory behind this is described in textbooks [51, 52] but here we follow the notation due to Schenter *et al.* [53]

$$B_2^{cl}(T) = -\frac{1}{2}\int_0^\infty \left\langle e^{-\beta u_{12}(\mathbf{r})} - 1 \right\rangle d\mathbf{r}$$

where $u_{12}(\mathbf{r})$ is the potential energy of two compounds, Eqn. (3.48), $\beta = 1/k_B T$ and the integral is over all space and relative orientations of the compounds. If we sample these adequately (including at close, repulsive, distance) we can simplify the integral to a one-dimensional one:

$$B_2^{cl}(T) = -2\pi\int_0^\infty r^2 \left\langle e^{-\beta u_{12}(r)} - 1 \right\rangle dr$$

The above equation is entirely classical and quantum corrections have to be added according to:

$$B_2^F(T) = \frac{\hbar^2}{24(k_B T)^3}\sum_{j=1}^{2}\left[\frac{\left\langle \mathbf{F}_j^2 \right\rangle}{m_j}\right]$$

for the force on the compounds and

$$B_2^\tau(T) = \frac{\hbar^2}{24(k_B T)^3}\sum_{j=1}^{2}\left[\sum_{\alpha=x,y,z}\frac{\left\langle \tau_{j,\alpha}^2 \right\rangle}{I_{j,\alpha}}\right]$$

for the torque on the compounds, where m_j is the mass of the molecules j and $\mathbf{F}^2$ is the averaged square force on one molecule given by

$$\left\langle \mathbf{F}^2 \right\rangle = k_B T\int_0^\infty \left\langle e^{-\beta u_{12}(\mathbf{r})}\left[\nabla u_{12}(\mathbf{r})\right]^2 \right\rangle d\mathbf{r}$$

and where I is the moment of inertia of the molecule and τ^2 is the average square torque on one molecule defined by

$$\left\langle \tau_{j,\alpha}^2 \right\rangle = k_B T\int_0^\infty \left\langle e^{-\beta u_{12}(\mathbf{r})}\left[\nabla_\omega u_{12}(\mathbf{r})\right]_{j,\alpha}^2 \right\rangle d\mathbf{r}.$$

The change in $B_2(T)$ as a function of temperature can be used to scrutinize the repulsive and attractive components of the potential energy. $B_2(T)$ is negative at low temperatures due to attraction forces, while it becomes positive at higher temperatures as repulsion forces start to dominate, and passes through a maximum and eventually decreases at very high temperatures where repulsion force are fully dominant [54].

Code to compute the second virial coefficient is available in the *alexandria b2* command. Since the calculation is relatively expensive it has been implemented to use parallel processing using the message passing library. You can run it on a 16-core machine like:

```
mpirun -n 16 alexandria b2 -v 3 -g TIP4P -b2 TIP4P -ff TIP4P
-f water#water.pdb -T1 373.15 -T2 673.15 -dT 25.0
-maxdimer 32768
```

where TIP4P corresponds to the well-known water model [55], the T_1 and T_2 are the temperature limits (note gas-phase for water), dT is the temperature interval and maxdimer determines how many relative orientations will be evaluated. For each orientation the energies are computed at *-ndist* equidistant distances between the centers of mass from *-mindist* to *-maxdist*, and the integrals are computed over this range. Below *-mindist* the molecules are assumed to overlap, that is the Mayer function $e^{-\beta u_{12}} - 1$ is -1, which adds $\frac{2\pi}{3} r_{min}^3$ to $B_2^{cl}(T)$. Therefore *-mindist* should be well inside the repulsive wall of the potential.

Several algorithms can be used to sample orientations (*-randalg*).

- If the **Sobol** algorithm for Quasi-Random numbers is used [56], the number of dimers should be a power of two.
- If, on the other hand, **Pseudo-random** numbers are used, there is no such requirement.
- The **Gauss** algorithm uses a grid of Euler angles instead, as in the program Orient [57]. The cosine of β of each molecule is integrated with *-nbeta* Gauss-Legendre points, and α of the second molecule and γ of each molecule with *-nalpha* and *-ngamma* equally spaced points. Since rotating both molecules around the line between their centers of mass does not change the dimer, α of the first molecule is zero. This gives *-nalpha* $\times$ *-nbeta*^{2} $\times$ *-ngamma*^{2} orientations, *-maxdimer* is not used, and the orientations are computed in a single round. For linear molecules *-ngamma 1* is sufficient. For water, a grid of 9 points for each angle gives the classical second virial and the quantum corrections within a few tenths of a percent, at a fraction of the cost of Sobol sampling with a similar accuracy.

The result is plotted in Fig. 2.8.

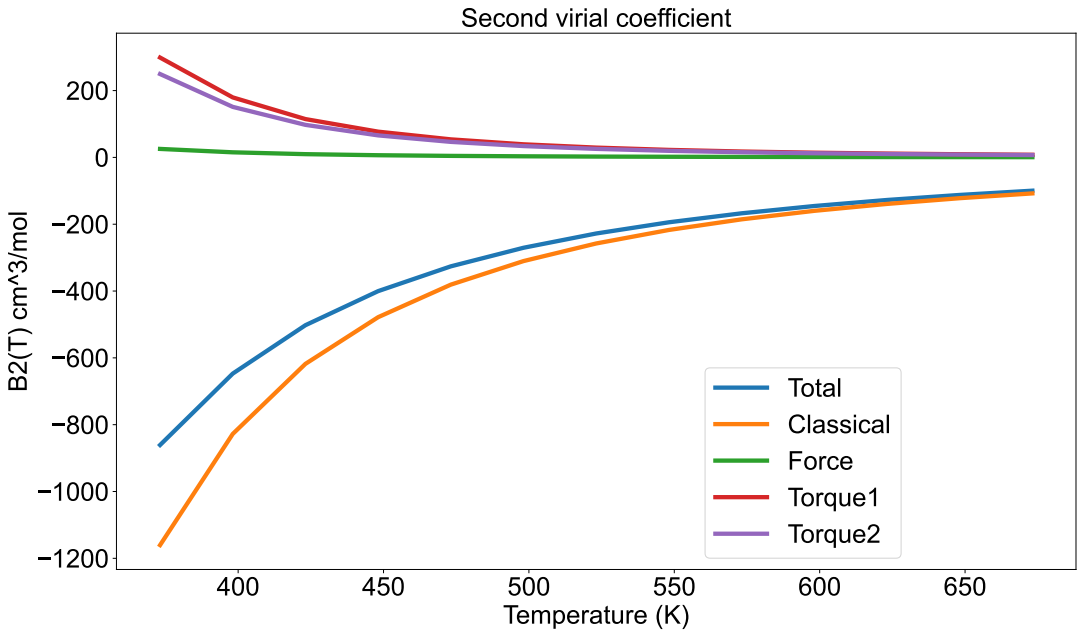


Fig. 2.8: Sample second virial coefficient and components for water using the TIP4P model.

CHAPTER

THREE

REFERENCE MANUAL

3.1 Energy Function

The ACT supports a range of different potentials for classical force field simulations. These potentials are described briefly in what follows.

3.1.1 Non-bonded interactions

3.1.1.1 Coulomb Interaction using Point charges

The Coulomb interaction can be computed with point charges:

$$V_{coul}(r_{ij}) \;=\; \frac{q_i q_j}{4\pi\varepsilon_0\varepsilon_r r_{ij}}. \tag{3.1}$$

3.1.1.2 Coulomb Interaction using Gaussian charges

Alternatively, the Coulomb interaction can be described using Gaussian shielded charges

$$V_{coul}(r_{ij}) \;=\; \frac{q_i q_j \mathrm{erf}\left(\zeta_{ij} r_{ij}\right)}{4\pi\varepsilon_0\varepsilon_r r_{ij}}, \quad \zeta_{ij} = \frac{\zeta_i \zeta_j}{\sqrt{\zeta_i^2 + \zeta_j^2}}, \tag{3.2}$$

where ε_0 is the permittivity of vacuum, $q_{i,j}$ are the charges and $\zeta_{i,j}$ are the charge distribution widths (screening factors). Note that Eqn. (3.2) includes a relative dielectric constant ε_r that can be used to parameterize a non-polarizable force field using charge-scaling [58].

3.1.1.3 Coulomb Interaction using Slater charges

In addition, Slater distributed charges can be used as described in ref. [30], from which the text below is an adapted version.

The spherical Slater orbital wavefunction is given by

$$\psi_n(\mathbf{r}) \;=\; \sqrt{\frac{(2\zeta)^{2n+1}}{4\pi(2n)!}}\mathbf{r}^{n-1}\mathrm{e}^{-\zeta\mathbf{r}} \tag{3.3}$$

where n is the highest quantum number of the element and ζ is the exponent giving the width of the charge density. The distribution of charge can be described by the charge density, which is the square of the wave function ((3.3)). The Coulomb integral can then be written as [59, 60]:

$$J_{ij}(\mathbf{r}) \sim \int\int |\psi_n(\mathbf{r_i})|^2 \frac{q_i q_j}{|\mathbf{r_i} - \mathbf{r_j}|} |\psi_m(\mathbf{r_j})|^2 d\mathbf{r_i} d\mathbf{r_j} \tag{3.4}$$

where ψ_n and ψ_m are the Slater wave functions of atoms i and j with quantum numbers n and m and with partial charges q_i and q_j, respectively. Both (3.2) and (3.4) have a finite limit as $r \rightarrow 0$. As a result, these functions are well behaved at small r. A number of methods [61, 62] have been proposed to evaluate (3.4). Hentschke gives an analytical solution [60]:

$$\begin{aligned} J_{ij}(\mathbf{r}) = & \frac{1}{4\pi\epsilon_0}\frac{q_i q_j}{\|\mathbf{r_i}-\mathbf{r_j}\|}\frac{4\zeta_i^{2n+1}\zeta_j^{2m+1}}{(2n)!(2m)!}\frac{\partial^{2n-2}\partial^{2m-2}}{\partial\zeta_i^{2n-2}\partial\zeta_j^{2m-2}}\left(\frac{1}{\zeta_i^3\zeta_j^3}\right) \\ & \times\left[1-\frac{(3\zeta_i^2-\zeta_j^2)\zeta_j^4}{(\zeta_i-\zeta_j)^3(\zeta_i+\zeta_j)^3}e^{-2\zeta_i r_{ij}}-\frac{(\zeta_i^2-3\zeta_j^2)\zeta_i^4}{(\zeta_i-\zeta_j)^3(\zeta_i+\zeta_j)^3}e^{-2\zeta_j r_{ij}}\right. \\ & \left.-\frac{\zeta_i\zeta_j^4}{(\zeta_i-\zeta_j)^2(\zeta_i+\zeta_j)^2}r_{ij}e^{-2\zeta_i r_{ij}}-\frac{\zeta_i^4\zeta_j}{(\zeta_i-\zeta_j)^2(\zeta_i+\zeta_j)^2}r_{ij}e^{-2\zeta_j r_{ij}}\right]. \end{aligned} \tag{3.5}$$

Eqn. (3.5) was implemented in a Mathematica ™ program from which C++ code was generated for the analytical computation of J_{ij} and its analytical derivatives with respect to r, which are necessary for computing forces. Due to the nature of Eqn. (3.5), there are many terms with large powers, particularly for $n > 3$. Thus, the equations have to be implemented using the arbitrary precision arithmetic library Class Library for Numbers (CLN). to avoid numerical instabilities. It should be noted that the arbitrary precision library significantly increases the computational cost to analytically solve Eqn. (3.5)

3.1.1.4 Long range Coulomb interactions

It is good practice [63] to use the particle-mesh Ewald [64, 65] method to treat long-range electrostatics interactions in the condensed phase. In short, the method splits the Coulomb potential, either Eqn. (3.1), Eqn. (3.2) or something similar into a short and a long-range part as follows

$$V_{coul}(r_{ij}) \;=\; V_{coul}\,\mathrm{erfc}(\alpha r_{ij}) + V_{coul}\,\mathrm{erf}(\alpha r_{ij}) \tag{3.6}$$

using the fact that the complementary error function {rm erfc} equal 1 - {rm erf}. The first term here is the short-range part, that is computed in real space, and the second term is the long-range part that is computed in Fourier (reciprocal) space by moving charges on a regular grid [65]. The constant α determines how quickly the short-range potential decays to zero, and it can be computed from a user-specified relative error tolerance related to moving charges on a grid. Given the cut-off distance rc and an error tolerance ϵ, typically 10^{-4}, we have

$$\alpha \;=\; \frac{\sqrt{-\ln 2\epsilon}}{rc} \tag{3.7}$$

where ln is the natural logarithm. This shows that α has the dimension of 1 over distance in the units of rc. Fig. 3.1 shows how the division over short and long-range interactions works in practice, and how the long range contribution should be incorporated into simulations using a modified Coulomb function.

3.1.1.5 Polarization

Polarization can be treated explicitly as well in the ACT using the core-shell model [32, 33]

$$V_{pol}(r_{cs}) \;=\; \frac{1}{4\pi\varepsilon_0}\frac{q_s^2}{2\alpha_c}r_{cs}^2, \tag{3.8}$$

where q_s is the shell charge, α_c is the polarizability and r_{cs} the distance between core and shell particles.

3.1.1.6 Induction correction

A term to add extra attraction between atoms was proposed in ref. [66]

$$V_{ic}(r_{ij}) \;=\; -A_{ij}\mathrm{e}^{-b_{ij}r_{ij}} \tag{3.9}$$

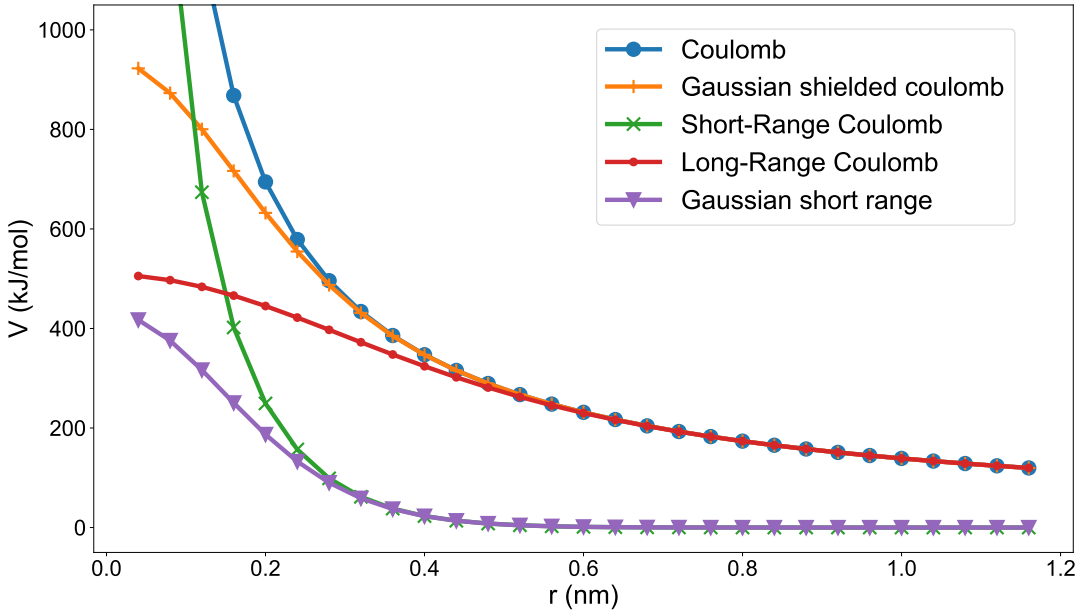


Fig. 3.1: Total Coulomb energy V as a function of distance r for two like unit charges, Gaussian-shielded Coulomb and short and long-range parts of the energy function. Ewald-tolerance ϵ 0.0001, rc 0.9 nm, α 3.24269/nm and Gaussian screening ζ 5/nm.

with A_{ij} and b_{ij} constants to be trained. The geometric combination rule is used for A_{ij} and the arithmetic combination rule for b_{ij}. It is not certain whether the exponential functional form is optimal to describe this interaction and there is no theory behind this.

3.1.1.7 Pauli Repulsion and London Dispersion

The repulsion and dispersion interactions acting between atoms i and j can be described by a number of potentials. In a recent study on noble gases [25] we found that the 14-7 potential due to Halgren [67] was one of the most accurate ones:

$$V_{14-7}(r_{ij}) = \epsilon_{ij}\left(\frac{1+\delta_{ij}}{\frac{r_{ij}}{\sigma_{ij}}+\delta_{ij}}\right)^7\left(\frac{1+\gamma_{ij}}{(\frac{r_{ij}}{\sigma_{ij}})^7+\gamma_{ij}}-2\right) \tag{3.10}$$

where ϵ_{ij} is the well depth at minimum, γ_{ij} and δ_{ij} are dimensionless numbers that were originally shared for all elements. However, in our previous work [25], we treated γ and δ as free atom-specific parameters subject to optimization and combination rules.

Furthermore, the generalized 4-parameter Buckingham potential with an adjustable long-range attraction is implemented as [68]

$$V_{GBH}(r_{ij}) = \epsilon_{ij}\frac{\delta_{ij}+2\gamma_{ij}+6}{2\gamma_{ij}}\frac{1}{1+(\frac{r_{ij}}{r_{min,ij}})^6}\left[\frac{6+\delta_{ij}}{\delta_{ij}+2\gamma_{ij}+6}e^{\gamma_{ij}(1-\frac{r_{ij}}{r_{min,ij}})}-1\right]-\frac{\epsilon_{ij}}{1+(\frac{r_{ij}}{r_{min,ij}})^\delta} \tag{3.11}$$

where γ and δ again are dimensionless constants and r_{min} is the position of the minimum in the potential. δ should be larger than zero for this potential to converge to 0 at long distance, and with δ = 6, the Wang-Buckingham potential ((3.12)) is recovered.

Another potential that is supported is the buffered (Wang) Buckingham potential [69]:

$$V_{WBH}(r_{ij}) = \left(\frac{2\epsilon_{ij}}{1-\frac{3}{\gamma_{ij}+3}}\right)\left(\frac{\sigma_{ij}^6}{\sigma_{ij}^6+r_{ij}^6}\right)\left[\frac{3}{\gamma_{ij}+3}e^{\gamma_{ij}\left(1-\frac{r_{ij}}{\sigma_{ij}}\right)}-1\right] \tag{3.12}$$

where ϵ_{ij} is the well depth at minimum, σ_{ij} is the Van der Waals radius and γ_{ij} determines the steepness of the potential and r_{ij} is the distance between particles i and j. The buffered Buckingham potential mentioned above have been used to develop an accurate phase-transferable model for alkali-halides [19, 20, 21, 22, 23] and the original Buckingham potential [70] is implemented as

$$V_{BH}(r_{ij}) \;=\; A_{ij}\mathrm{e}^{-b_{ij}r_{ij}} - \frac{C_{ij}}{r_{ij}^6} \tag{3.13}$$

where A_{ij}, b_{ij} and C_{ij} are parameters to be trained. An alternative way of formulating the Buckingham potential [71] is

$$E_{MBH} = \frac{\epsilon}{1-\frac{6}{\gamma}}\left[\frac{6}{\gamma}e^{\gamma(1-\frac{r}{\sigma})}-\left(\frac{\sigma}{r}\right)^6\right] \tag{3.14}$$

where ϵ is the well-depth σ reflects the position of the minimum and γ is again a dimensionless constant. Although mathematically identical to Eqn. (3.13) we have shown that the potentials behave differently when combination rules are applied [25].

The well-known Tang-Toennies potential [72, 73] containing five parameters is implemented according to:

$$V_{TT}(x) = Ae^{-bx} - \sum_{n=3}^{5}\left[1-e^{-bx}\sum_{k=0}^{2n}\frac{(bx)^k}{k!}\right]\frac{C_{2n}}{x^{2n}} \tag{3.15}$$

where repulsion and dispersion terms shared the parameter b. Here, all five parameters can be trained by the ACT. This potential was used in our recent work on noble gases published where we investigated combination rules [25].

In other works, the Tang-Toennies potential is extended with an additional parameter [74] giving the exponential functions a different decay length b:

$$V_{TT2}(x) = Ae^{-b_{exp}x} - \sum_{n=3}^{5}\left[1-e^{-b_{disp}x}\sum_{k=0}^{2n}\frac{(b_{disp}x)^k}{k!}\right]\frac{C_{2n}}{x^{2n}} \tag{3.16}$$

and the ACT supports both these functions. Van Vleet *et al.* described a more accurate formula for the Pauli repulsion [75] that was applied to derive a force field later [76]. In this model the exchange repulsion is given by the Slater-ISA formula

$$V_{exch} \;=\; A\left(\frac{1}{3}(br)^2+br+1\right)\exp^{-br} \tag{3.17}$$

where it can be noted, that no additional parameters are needed for equation~:eq:*slater_isa* compared to the repulsion part in the Tang-Toennies potential (Eqn. (3.15)). In the ACT the Slater-ISA exchange can be combined with the damped dispersion part of the latter potential.

The Lennard-Jones 12-6 potential [77] is available as well:

$$V_{LJ}(r_{ij}) \;=\; 4\varepsilon_{ij}\left(\frac{\sigma_{ij}^{12}}{r_{ij}^{12}}-\frac{\sigma_{ij}^6}{r_{ij}^6}\right) \tag{3.18}$$

and, in addition, the 12-6-4 potential

$$V_{LJ}(r_{ij}) \; = \; 4\varepsilon_{ij}\left(\frac{\sigma_{ij}^{12}}{r_{ij}^{12}} - \frac{\sigma_{ij}^{6}}{r_{ij}^{6}}\right) - \frac{\gamma}{r^4} \tag{3.19}$$

which is meant to model ion-dipole interactions is supported too.

It is worth noting that in all these potentials (except Buckingham, Eqn. (3.13)) parameters describe both the exchange and dispersion interactions at the same time, which necessitates simultaneous training of these interaction. To account for anisotropy in exchange, a correction can be implemented using a virtual site particle on the atom that has a σ-hole interacting with other particles [26]:

$$V_{EXCH,Corr}(r_{ij}) = -A_{ij}\mathrm{e}^{-b_{ij}r_{ij}} \tag{3.20}$$

where A_{ij} and b_{ij} are parameters to be optimized. This correction term (Eqn. (3.20)) does not need to be applied to all possible atom pairs, therefore a new combination rule, dubbed "Kronecker" is introduced

$$A_{ij} \; = \; (1-\delta_{ij})\frac{A_i + A_j}{2}$$

where δ_{ij} is the Kronecker delta operating on particle types i and j. This means that only interactions between σ-holes (represented by a virtual site) and atoms are non-zero. To avoid parameter explosion, the atomic A_i are set to zero and only the virtual site A_j is non-zero. This is reasonable since the virtual site represents a property of the σ-hole of the atom it is connected to. The arithmetic combination rule is used for b_{ij}.

Multiple different combination rules can be used for parameters involved in van der Waals interactions, both within ACT and through the interface to OpenMM. For details, see the paper by Kříž *et al.* [25].

3.1.1.8 Treatment of long range dispersion

The different Van der Waals potentials described above can be used with Lennard-Jones PME [64, 78] if treated carefully. LJ-PME assumes a simple dispersion interaction given by

$$V_{LJPME}(r_{ij}) \; = \; -\frac{C_{ij}}{r_{ij}^6} \tag{3.21}$$

for atoms i and j, which differs from the potentials mentioned above. For LJ-PME, it is beneficial to use the geometric combination rule, hence

$$V_{LJPME}(r_{ij}) \; = \; -\frac{C_i C_j}{r_{ij}^6}$$

where C_i and C_j, in contrast to Eqn. (3.21), have units of distance to the third power.

To minimize the difference between the potential used and the LJ dispersion over the whole volume outside the cut-off, and to specify correct inputs to OpenMM, starting from e.g. Eqn. (3.12) we have to solve the following equation:

$$0 \; = \; \int_{rc}^{\infty} 4\pi r_{ij}^2 \left[\left(-\frac{2\epsilon_{ij}}{1-\frac{3}{\gamma_{ij}+3}}\right)\left(\frac{\sigma_{ij}^6}{\sigma_{ij}^6 + r_{ij}^6}\right) + \frac{C_i C_j}{r_{ij}^6}\right] \mathrm{d}r_{ij} \tag{3.22}$$

where we integrate from the cut-off rc to infinity. In this notation, we have to insert the combination rules for ϵ, σ and γ. For the special case that i = j, the constant C_{ii} can be derived using Mathematica ™ as

$$C_{ii} \; = \; \epsilon_i \frac{3+\gamma_i}{\gamma_i} rc^3 \sigma_i^3 \left(\pi - 2\arctan\left[\left(\frac{rc}{\sigma_i}\right)^3\right]\right)$$

which we can then convert back to a σ' compatible with standard Lennard-Jones by

$$\sigma'_i = \left(\frac{C_{ii}}{4\epsilon_i}\right)^{1/6}.$$

Note however, that we need to get effective σ' for all atoms, which means the problem turns into a minimization problem where

$$\varepsilon^2 = \sum_{i,j}\left[\int_{rc}^{\infty} 4\pi r_{ij}^2 \left[\left(-\frac{2\epsilon_{ij}}{1-\frac{3}{\gamma_{ij}+3}}\right)\left(\frac{\sigma_{ij}^6}{\sigma_{ij}^6+r_{ij}^6}\right)+\frac{C_iC_j}{r_{ij}^6}\right]\mathrm{d}r_{ij}\right]^2 \tag{3.23}$$

has to be minimized with respect to C_i independently for each set of combination rules. In the Alexandria alkali-halide model we used the Wang-Buckingham potential (Eqn. (3.12)) in conjunction with the combination rules due to Kong [79] in which σ_{ij} depends on all the σ, ϵ and γ. This makes it cumbersome to analytically derive integrals like Eqn. (3.23) for many combinations of potentials and combination rules.

Another problem is, that the parameters C_i that we need to compute the long-range dispersion interaction now depend on the cut-off, which means they should preferably not be computed beforehand. In summary, the C_i should be derived numerically given the potential and combination rules at the start of a simulation.

3.1.1.9 Combination Rules for Van der Waals Potentials

In a recent paper [25] we studied the effect of combination rules on potentials for noble gases and introduced two new combination rules (Eqn. (3.34) and Eqn. (3.35)). Here is a brief summary, copied (with permission) from that paper.

Combination rules reduce the number of parameters required for the pair-wise potentials introduced above because the parameters describing the interaction between dissimilar X–Y atoms are reconstructed from parameters of homodimers X–X and Y–Y. In this way, it is necessary only to fit atomic parameters on data for homodimers. Different sets of mathematical expression were considered, as detailed below.

The two simplest expressions that have historically been used as combination rules are the geometric [80] and arithmetic [81] averages, respectively:

$$X_{12} = \sqrt{x_1 x_2} \tag{3.24}$$

%was used for all three parameters ϵ, γ, σ in "geometric" rules. It is also used for ϵ in "arithmetic" rules and for σ in "Kong-Mason" rules [79].

$$X_{12} = \frac{x_1 + x_2}{2} \tag{3.25}$$

where x_1 and x_2 are the atomic parameters. Both these rules can in principle be applied to all parameters in the Van der Waals potentials described above and the rules are well-behaved mathematically.

Hogervorst introduced a set of combination rules for 12-6 Lennard-Jones and exp-6, modified Buckingham, potential (Eqn. (3.14)) [82]. He proposed using Eqn. (3.26) for ϵ for both potential forms. For the σ of 12-6 potential (Eqn. (3.18)), the arithmetic mean (Eqn. (3.25)) was used. In the case of the modified Buckingham potential (eqn. (3.14)), expression~:eq:*cr_sigma5*, which depends on the combined parameters $\epsilon_{1,2}$ and $\gamma_{1,2}$ was advocated for the σ, along with arithmetic mean for γ.

$$X_{12} = \frac{2x_1x_2}{x_1+x_2} \tag{3.26}$$

$$\sigma_{12}^6 = \sqrt{\frac{\epsilon_1\gamma_1\sigma_1^6}{\gamma_1-6}\frac{\epsilon_2\gamma_2\sigma_2^6}{\gamma_2-6}}\frac{(\gamma_{1,2}-6)}{\gamma_{1,2}\epsilon_{1,2}} \tag{3.27}$$

Where the $\gamma^{1,2}$ used Eqn. (3.25) and $\epsilon^{1,2}$ Eqn. (3.26). It should be noted that equation (3.26) is ill-behaved if both x_1 and x_2 are zero, while Eqn. (3.27) is ill-behaved if either $\gamma_{1,2}$ or $\epsilon_{1,2}$ is zero. Yang *et al.* [83] introduced an expression for the Morse potential [84], using Eqn. (3.26) for ϵ, while eqn. (3.28) is used for σ and γ.

$$X_{12} = \frac{x_1 x_2 (x_1 + x_2)}{x_1^2 + x_2^2} \tag{3.28}$$

The expression for γ proposed by Mason [85] for the exp-6 potential has the form %red{while used other rels for epsilon, sigma in alexandria}

$$\gamma_{12} = \sqrt{\sigma_1 \sigma_2} \left(\frac{\gamma_1}{2\sigma_1} + \frac{\gamma_2}{2\sigma_2} \right) \tag{3.29}$$

Waldman and Hagler [86] introduced expressions (3.30) for ϵ and (3.31) for σ
to reproduce experimental well-depths and interaction distances.

$$\epsilon_{12} = \sqrt{\epsilon_1 \epsilon_2} \frac{2\sigma_1^3 \sigma_2^3}{\sigma_1^6 + \sigma_2^6} \tag{3.30}$$

$$X_{12} = \left(\frac{X_1^6 + X_2^6}{2} \right)^{1/6} \tag{3.31}$$

Although Eqn. (3.31) was devised for σ we have evaluated it for other parameters here as well, hence the notation with X. Qi and coworkers advocated the use of buffered 14-7 Lennard-Jones potential (Eqn. (3.10)) due to Halgren [67], alongside combination expressions (3.30) for ϵ and (3.32) for σ: [87]

$$X_{12} = \frac{x_1^3 + x_2^3}{x_1^2 + x_2^2} \tag{3.32}$$

A further relation, the harmonic mean rule, was proposed by Halgren [67]:

$$X_{12} = \frac{4 x_1 x_2}{(x_1^{1/2} + x_2^{1/2})^2} \tag{3.33}$$

Finally, we introduce two new combination rules that we have not seen published previously. Since the σ in most potentials can be interpreted as a Van der Waals radius, we introduced a relation averaging third powers, corresponding to an atomic volume [25]:

$$\sigma_{12} = \left(\frac{\sigma_1^3 + \sigma_2^3}{2} \right)^{1/3}. \tag{3.34}$$

In addition, we applied the following rule for in particular ϵ since it yield an X_{12} that is smaller than the geometric one (Eqn. (3.24)):

$$X_{12} = \left(\frac{2}{x_1^{-2} + x_2^{-2}} \right)^{1/2} \tag{3.35}$$

The combination relations described above were permuted with each other into new combination rules. In this way, relations that depend on only one parameter type were used for any parameter. Relations depending on multiple parameters were used only for the specific parameter type combination they depend on (e.g. Eqn. (3.30) was only used for ϵ, using homodimer ϵ and σ). In our previous work on alkali halides [19] we used combination rules according to eqn. (3.27) for σ, eqn. (3.26) for ϵ and eqn. (3.25) for γ with the Wang-Buckingham potential (Eqn. (3.12)).

A number of these combination rules can be written using the generalized mean equation [88]

$$M_p(x_1, x_2, ..., x_N) = \left(\frac{1}{N} \sum_{i=1}^{N} x_i^p \right)^{1/p} \tag{3.36}$$

for $p \neq 0$. For $p = 0$, Eqn. (3.36) turns into the geometric rule. Table 3.1 lists other well-known combination rules and their respective exponent p. Hohm also describes combinations of the generalized mean and other similar expressions, but the most important observation he made was that the exponent p can be varied at will [88]. In the ACT it is possible to train this parameter along with the Van der Waals parameters.

Table 3.1: Correspondence between well-known combination rules and the generalized mean equation (3.36).

p	Name	Equation
-2	Inverse Square	(3.35)
-1	Hogervorst ϵ	(3.26)
-1/2	Harmonic Mean	(3.33)
0	Geometric	(3.24)
1	Arithmetic	(3.25)
3	Volumetric	(3.34)
6	Waldman-Hagler σ	(3.31)

3.1.2 Bonded interactions

3.1.2.1 Harmonic potential

Bond vibrations can be described using a harmonic term based on the bond length r_{ij}

$$V_b(r_{ij}) \;=\; \frac{k^b_{ij}}{2} \left(r_{ij} - r^0_{ij}\right)^2 , \tag{3.37}$$

where k^b_{ij} is the force constant, and r^0_{ij} is the equilibrium bond length.

3.1.2.2 Morse potential

A Morse potential [84] can be used, with one addition:

$$V_M(r_{ij}) \;=\; D^e_{ij} \left[\mathrm{e}^{-2\beta_{ij}(r_{ij} - r^0_{ij})} - 2\mathrm{e}^{-\beta_{ij}(r_{ij} - r^0_{ij})} \right] + D^0_{ij} \tag{3.38}$$

The term D^e_{ij} roughly corresponds to a dissociation energy, however since there are Coulomb and/or Buckingham interactions between the atoms as well, the total "bond" potential is given by the sum of three terms and a correction term D^0_{ij} is needed to get the correct energy minimum.

3.1.2.3 Wei-Hua potential

In a very recent study we found the potential due to Hua [89, 90] to be the best compromise between accuracy of the vibrational frequencies and computational cost [27]. It is given by:

$$U(r) \;=\; D_e \left(\left[\frac{1 - e^{-b(r - r_e)}}{1 - ce^{-b(r - r_e)}} \right]^2 - 1 \right) \tag{3.39}$$

where D_e is the well-depth, r_e the equilibrium bond length, and b and c are constants with $\|c\| < 1$.

3.1.2.4 Angle potential

Angle vibrations are described using a harmonic term based on the angle θ_{ijk}

$$V_a(\theta_{ijk}) \;=\; \frac{k^{\theta}_{ijk}}{2}\left(\theta_{ijk} - \theta^{0}_{ijk}\right)^2, \tag{3.40}$$

where k^{θ}_{ijk} is the force constant, and θ^{0}_{ijk} is the equilibrium angle.

3.1.2.5 Angle potential for linear compounds

The reference position, corresponding to a minimum energy structure, $\mathbf{x}^0_j$ for a central atom j in a linear triplet of atoms i, j, k is given by

$$\mathbf{x}^0_j \;=\; \mathrm{a}\,\mathbf{x}_i + (1-\mathrm{a})\,\mathbf{x}_k \tag{3.41}$$

where a is a constant defined by the bond-lengths $i-j$ and $j-k$. In a group with bonds $i-j$ and $j-k$ with lengths b_{ij} and b_{jk} respectively, the constant is

$$a \;=\; \frac{b_{jk}}{b_{ij}+b_{jk}}. \tag{3.42}$$

If the order of atoms is flipped a will change to $1-a$. The potential V_{lin} is then given by

$$V_{lin} \;=\; \frac{\mathrm{k}_{lin}}{2}\left(\mathbf{x}_j - \mathbf{x}^0_j\right)^2 \tag{3.43}$$

with k $_{\mathrm{lin}}$ the force constant [24].

3.1.2.6 Out-of-plane vibrations

Finally, out-of-plane vibrations are treated by another harmonic potential

$$V_i(\phi_{ijkl}) \;=\; \frac{k^{\phi}_{ijkl}}{2}\phi^2_{ijkl}, \tag{3.44}$$

where k^{ϕ}_{ijkl} is the force constant and ϕ_{ijkl} is defined by the angle between the two planes i, j, k and j, k, l. This potential was historically termed *improper dihedral*.

3.1.2.7 Torsion potential

A torsion potential is implemented using a Fourier series:

$$V_d(\phi_{ijkl}) \;=\; \sum_{n=0}^{5} c_n \cos^n(\pi + \phi_{ijkl}) \tag{3.45}$$

where c_n are constants and the torsion angle is defined as above. The constant π is added to be compatible with the Ryckaert-Bellemans potential [91] that is implemented in simulation codes like GROMACS [92] and OpenMM [93].

3.1.2.8 Proper dihedral

A simpler torsion potential is implemented for backward compatibility as:

$$V_d(\phi_{ijkl}) \;=\; \cos(n\phi_{ijkl} + \phi_0) \tag{3.46}$$

where n is the multiplicity (number of minima in 360 degrees) ϕ_0 is an offset angle.

3.1.3 Special potentials

The ACT includes a flat-bottom position restraint potential according to

$$V_{fbpr}(r) = \begin{cases} 0 & r \leq r_0 \\ \frac{k}{2}(r - r_0)^2 & r > r_0 \end{cases} \tag{3.47}$$

where k is the force constant and r_0 the radius of the sphere (centered at the origin) in which the potential is zero. The flat-bottom potential is activated by flags to the *alexandria simulate* command. It is useful mainly to keep molecules close to the origin and prevent them from flying into outer space.

3.1.4 Virtual Sites

A virtual site is an extra point, located at a defined position in a molecule. A variety of virtual site options is currently implemented within the ACT framework:

- a virtual site on top of an atom (VSITE1) [94]
- a virtual sites along the bond (VSITE2) for the description of anisotropic charge distribution and exchange [26] such as encountered in σ holes
- a virtual site on the bisector of a angle, like in the TIP4P water model [55] (VSITE3S) or in alcohol (VSITE3)
- off-plane virtual sites for modeling lone-pairs in sp^3 hybridized compounds, such as water (VSITE3SOUT, symmetric) or asymmetric for compounds like alcohols [26, 95].
- Four-particle virtual sites (VSITE4) to model a lone-pair on an amine group.

3.1.5 Total energy

The total energy E of a compound then follows from

$$E = V_{vdw}(r_{ij}) + V_{coul}(r_{ij}) + V_{pol}(r_{cs}) + V_b(r_{ij}) + V_a(\theta_{ijk}) + V_i(\phi_{ijkl}) + V_d(\phi_{ijkl}). \tag{3.48}$$

Finally, it should be noted that the number of excluded neighbors is user-configurable. That means that atoms that are covalently bonded can interact both through the Buckingham (or Lennard-Jones) and Coulomb potentials, and through the bonded potentials. The main reason for this is that the short-range Coulomb interactions yield polarization anisotropy that is difficult to reproduce by a non-interacting model. To make sure that the forces on the atoms in a molecule are zero in the reference minimum-energy structure from quantum chemistry, both bond lengths r^0_{ij} and angles θ^0_{ijk} can be treated as free parameters, that may differ substantially from the reference geometry. The number of exclusions can be selected separately for Coulomb and Van der Waals forces.

In total there are up to seven "atom" parameter types (ϵ, σ, γ, δ, ζ, q_s, and α) and 7-14 "bond" parameter types (k^b, D^e, r^0, r^{max}, k^θ, θ^0, k^{lin} and c_n) where n is the dihedral term index running from 0 to 6 to determine. All the atomic parameters are taken to be hybridization state dependent (corresponding to, for instance, sp ^{1}, sp ^{2} and sp ^{3} carbon atoms). %It is straightforward to add support for other potentials, such as proper dihedrals.

3.2 Shell Relaxation and Integration

In polarizable force fields the positions of the shells (or Drude particles) are not determined by the input structure, they have to be computed for each structure. This is done by minimizing the energy with respect to the shell positions, keeping the atoms fixed, as described in this section. The second part of the section describes the integration of the equations of motion in *alexandria simulate*.

3.2.1 Relaxation of shells

The shells have no mass. In the adiabatic approximation they follow the atoms instantaneously, that is, for each configuration of the atoms they are at the minimum of the potential energy. The shells are moved to this minimum iteratively, using the forces on them, as follows.

1. The shells are put on top of their core particle and the energy and forces are computed.
2. The mean square force on the N_s shells is computed

$$F_{ms} \;=\; \frac{1}{N_s}\sum_{s=1}^{N_s} |\mathbf{F}_s|^2 \,. \tag{3.49}$$

   If F_{ms} is at most the tolerance, the shells are converged. The tolerance is given in (kJ/mol nm)^2, see below.
3. Otherwise each shell is displaced along the force on it,

$$\Delta\mathbf{r}_s \;=\; \lambda\,\frac{\mathbf{F}_s}{k_s}, \qquad k_s \;=\; \frac{1}{4\pi\varepsilon_0}\frac{q_s^2}{\alpha_c}, \tag{3.50}$$

   where k_s is the force constant of the core-shell spring, see Eqn. (3.8). With $\lambda = 1$ this step would bring an isolated shell to its minimum, since the force of the spring then cancels the force $\mathbf{F}_s$ from the other particles. With hyperpolarization, the potential of the core-shell pair has an additional term $k_h\,(r_{cs} - r_h)^4$ for distances r_{cs} larger than r_h. The stiffness of the spring then depends on the direction. For a shell away from its core, the component of $\mathbf{F}_s$ perpendicular to the core-shell vector is divided by the ratio of the core-shell force and the distance

$$k_s + 4k_h\,\frac{(r_{cs} - r_h)^3}{r_{cs}} \qquad \text{if} \quad r_{cs} > r_h, \tag{3.51}$$

and the component along the core-shell vector by the second derivative of the potential

$$k_s + 12k_h\,(r_{cs} - r_h)^2 \qquad \text{if} \quad r_{cs} > r_h. \tag{3.52}$$

This is a Newton step for an isolated shell. For a shell on top of its core, the quartic term only contributes if $r_h = 0$, and then the displacement Δr is the solution of $|\mathbf{F}_s| = k_s\,\Delta r + 4k_h\,\Delta r^3$. The force constants are computed from the current polarizabilities and shell charges, such that they are correct during training as well. #. Since the shells interact with each other, a full step ($\lambda = 1$) can overshoot. If F_{ms} increases, the step is reduced to $\lambda = 1/2$ for the rest of the relaxation. Smaller steps are not used, since they can freeze the shells far from their minimum. #. A shell with one core is not allowed to move further away from it than a maximum distance d_{max}. If the step brings a shell further away, it is moved back to the sphere with radius d_{max} around its core, along the core-shell vector. This prevents the polarization catastrophe, where shells of neighbouring molecules attract each other without limit. With hyperpolarization the quartic term usually prevents it as well, then d_{max} can be switched off. #. The forces are computed again and the procedure is repeated from step 2, until the shells are converged or the maximum number of iterations is reached.

A shell held at d_{max} by a force pointing away from its core can never reach a zero force. For such a shell only the component of the force perpendicular to the core-shell vector $\mathbf{r}_{cs}$ is used in Eqn. (3.49)

$$\mathbf{F}_{s,\perp} \;=\; \mathbf{F}_s - \frac{\mathbf{F}_s\cdot\mathbf{r}_{cs}}{r_{cs}^2}\,\mathbf{r}_{cs} \qquad \text{if} \quad r_{cs} = d_{max} \quad \text{and} \quad \mathbf{F}_s\cdot\mathbf{r}_{cs} > 0, \tag{3.53}$$

which is zero at the minimum of the energy on the sphere. When a strong field holds the shell there, which is common with the poor parameters early in training, $\mathbf{F}_{s,\perp}$ is the difference of two large force components. It can then stay well above the tolerance on F_{ms}, in a cycle or unchanged, even though

the shell is practically at the minimum. Moving the shell over a small angle θ on the sphere changes its energy by about

$$\Delta E \approx \frac{1}{2} |\mathbf{F}_s| \, d_{max} \, \theta^2, \tag{3.54}$$

since the force away from the core acts as a force constant $|\mathbf{F}_s| / d_{max}$ across the core-shell vector. Therefore, the force of such a shell counts as zero in Eqn. (3.49) if $|\mathbf{F}_{s,\perp}| \leq 10^{-4} |\mathbf{F}_s|$. For $|\mathbf{F}_s| = 10^4$ kJ/(mol nm) and d_{max} = 0.02 nm the energy can then change by about 10^{-6} kJ/mol. The perpendicular force is computed as $|\mathbf{F}_s \times \mathbf{r}_{cs}| / r_{cs}$, since the difference in Eqn. (3.53) loses its precision when the perpendicular force is that small compared to the force, in particular with forces in single precision. Without this, a few conformations with shells held at d_{max} keep the relaxation of all the others going until the maximum number of iterations.

The precision of the forces and of the coordinates also limits how small F_{ms} can get. The forces are sums of terms that can be much larger than the force itself. When F_{ms} no longer decreases, the relaxation therefore stops if F_{ms} is within $(32 \, \varepsilon \, F_{max})^2$, where ε is the machine precision and F_{max} the largest force component of any particle. Terms that cancel can be larger than F_{max}, however. In the test force field *PG-pol-ions.xml*, for instance, chloride has a shell charge of -9 e on a core of about +8 e, which are practically on top of each other since its polarizability is at the lower bound of 10^{-6} Å^3. A water shell at 0.3 nm then feels forces of about $6 \cdot 10^4$ kJ/(mol nm) from both, which nearly cancel. In single precision their rounding errors add up to a few hundredths of a kJ/(mol nm), and in a dimer of water and chloride F_{ms} went down to 10^{-5} (kJ/mol nm)^2 and then cycled between $3 \cdot 10^{-4}$ and 10^{-3}. A tolerance of 10^{-8} was then not reached, while in double precision it was. The energies are as precise as the forces allow, but the conformation counts as not converged, takes the maximum number of iterations, and its shells are not kept with *-shell_keep*. With the default tolerance of 10^{-3}, and with 10^{-6}, all conformations converged. Therefore, in single precision, and hence on a GPU, do not use a tolerance far below the default.

The coordinates are integers in units of 2^{-56} nm, about $1.4 \cdot 10^{-17}$ nm. A shell with a very stiff spring, such as that of the chloride above with $k_s \approx 10^{13}$ kJ/(mol nm^2), does not move when its displacement in Eqn. (3.50) is smaller than that unit, which happens at forces of about 10^{-4} kJ/(mol nm). A conformation of which no shell moved in a step counts as converged, since the next steps would be the same.

The maximum distance d_{max} is part of the force field, since its parameters are trained with it. It is the option *shellmaxdistance* (in nm) of the polarization in the force field file,

```
<interaction type="POLARIZATION" function="POLARIZATION" canswap="true">
  <option key="shellmaxdistance" value="0.02"/>
```

where 0 means that there is no maximum. It can be set with *alexandria gen_ff -shellmaxdistance*, or by editing the file, but not when training or using the force field. Without the option it is 0.02 nm. The other options that control the relaxation are listed in Table 3.2.

Table 3.2: Options for the shell relaxation.

Quantity	train_ff	simulate	Default
Tolerance on F_{ms}	-shellToleran	-shelltole	0.001 (train_ff), 1e-6 (simulate), at most a tenth of -toler in simulate
Maximum number of iterations	-shell_max_it	-maxiter	25 (train_ff), 100 (simulate)
Start from the previous shells	-shell_keep		yes (train_ff), from the cores in simulate, see *Shell relaxation in training*

3.2.1.1 Shell relaxation in training

During training the energies of all conformations (structures) of a molecule are computed together, see *Force Field Training Algorithms*. The shells of all conformations are relaxed at the same time, and the conformations for which the shells converged are not computed again in the next iterations. For a dimer, the shells of each monomer are relaxed first in the isolated monomer, then all shells in the dimer. The induction energy is the change of the electrostatic energy upon relaxation of the shells. If the induction correction is trained separately (see *Induction correction*), the second order induction is computed by relaxing the shells of one monomer in the field of the other one, of which the shells stay at their positions in the isolated monomer, and the other way around. The rest of the induction is the induction correction.

A training step changes the parameters only a little, so the relaxed shells of the previous evaluation are much closer to the new minimum than their cores are. Therefore, by default (*train_ff -shell_keep*), the relaxation of each conformation starts from the shells of the previous evaluation by the same process, in the monomers and in the dimer. The shells are still put on their cores first to compute the energies before relaxation, so the induction energy is defined as before. The shells of conformations that did not converge are not kept, they start from their cores in the next evaluation. For the PC+GS force field of the electrostatics benchmark, starting from trained parameters, the number of iterations per conformation decreased from 2.6 to 0.4, which made training about 40% faster on the processor and about 17% faster on a GPU, where the energy calculations take a larger part of the time.

The relaxed positions depend on where the relaxation started, within the tolerance on F_{ms}, which has no noticeable effect on χ^2 with the default tolerance of 0.001 (kJ/mol nm)^2. A smaller tolerance is not needed, and in single precision, for instance on a GPU, a tolerance of 10^{-4} cannot always be reached, such that more conformations fail to converge. With poor parameters, such as those of *-random_init* at the start of training, the shells can have more than one minimum, however. Shells of neighbouring molecules may then attract each other until one of them is held at d_{max}, the polarization catastrophe, and which shell ends up at d_{max} depends on where the relaxation started. A shell held at d_{max} in a strong field can also have minima on different sides of its core. In both cases the energy of the conformation, and hence χ^2, depends on the previous evaluation. In a test with random parameters, 0.08% of the conformations ended in a different minimum than when starting from the cores, while with trained parameters none did. As a consequence, a training started from random parameters depends slightly on the order in which the processes evaluate the parameter sets, such that two runs with the same random number seed need not be identical, and the χ^2 of the best parameters, which is recomputed at the end of the training, is checked to agree to within 10^{-3} relative to its value only, instead of 10^{-3} absolute. With *-noshell_keep* the shells start from their cores in every evaluation, which gives reproducible results.

Without polarization the interaction energy of a dimer is computed from the intermolecular interactions only, unless forces are needed, since the intramolecular energies of the dimer and the monomers cancel.

3.2.2 Integration in simulate

The command *alexandria simulate* integrates the equations of motion in the gas phase (see *MD Simulations with Alexandria Force Fields*) using the velocity Verlet algorithm [96]. For each atom i with mass m_i, coordinates $\mathbf{r}_i$ and velocity $\mathbf{v}_i$, a step of length Δt is

$$\mathbf{r}_i(t+\Delta t) \;=\; \mathbf{r}_i(t) + \Delta t\,\mathbf{v}_i(t) + \frac{\Delta t^2}{2m_i}\mathbf{F}_i(t), \tag{3.55}$$

after which the forces $\mathbf{F}_i(t+\Delta t)$ are computed for the new coordinates and the velocities are updated

$$\mathbf{v}_i(t+\Delta t) \;=\; \mathbf{v}_i(t) + \frac{\Delta t}{2m_i}\left[\mathbf{F}_i(t) + \mathbf{F}_i(t+\Delta t)\right]. \tag{3.56}$$

Only particles with a mass are integrated. Virtual sites are constructed from the atoms before the forces are computed, and the forces on them are distributed over the atoms they are constructed from. The shells

are relaxed for each new configuration of the atoms as described in *Relaxation of shells*, starting from their cores. Bond lengths and angles can be constrained, as described in *Constraints*.

With *-temp* larger than zero, the initial velocities are generated from a Maxwell distribution at that temperature, using the random number seed *-seed*. The motion of the center of mass is removed (see below), the velocity components along the constraints are removed, and finally the velocities are scaled such that the temperature, Eqn. (3.57), is exactly the requested one. There is no thermostat, the simulation conserves the total energy, which can be used to validate the forces. The energy file contains the potential energy terms, the kinetic energy $E_{kin} = \frac{1}{2}\sum_i m_i \mathbf{v}_i^2$, the total energy and the temperature

$$T \;=\; \frac{2E_{kin}}{N_{df}\,k_B}, \tag{3.57}$$

which are written every *-nstener* steps before the step is taken. The number of degrees of freedom is

$$N_{df} \;=\; 3N - N_{com} - N_c, \tag{3.58}$$

for N atoms with a mass, N_{com} degrees of freedom of the motion of the center of mass that is removed and N_c constraints, see *Constraints*. Shells and virtual sites do not contribute, since they have no mass.

The option *-comm* sets which motion of the center of mass is removed, from the initial velocities and every *-nstcomm* steps (default 100) after the velocities are updated, or only from the initial velocities if *-nstcomm* is 0. The choices are:

- *angular*: the translation of and the rotation around the center of mass (the default). This is needed for stable simulations of clusters in vacuum, since the rotation of the cluster otherwise takes up part of the kinetic energy. The velocity $\mathbf{v}_{cm}$ of the center of mass and the rigid rotation $\boldsymbol{\omega} \times (\mathbf{r}_i - \mathbf{r}_{cm})$ are subtracted from the velocities, where the angular velocity $\boldsymbol{\omega} = \mathbf{I}^{-1}\mathbf{L}$ follows from the inertia tensor $\mathbf{I}$ and the angular momentum $\mathbf{L}$ around the center of mass $\mathbf{r}_{cm}$. For a molecule that is linear in the starting coordinates, the rotation around the axis, with the smallest moment of inertia, is not removed. This removes $N_{com} = 6$ degrees of freedom, or 5 for a linear molecule.
- *linear*: only the translation of the center of mass, $N_{com} = 3$.
- *none*: nothing, $N_{com} = 0$.

Since this is a rigid body motion, the constraints are still satisfied afterwards. Without external forces, the linear and angular momentum are conserved, apart from numerical errors, such as those of the shell relaxation and the constraints, which are removed every *-nstcomm* steps. With the flat-bottom position restraints of *-kfbpr*, which act around the origin, the center of mass does move. Since the potential energy takes up part of the initial kinetic energy, the temperature during the simulation is lower than the initial one, about half of it if the starting structure is close to the minimum of the energy. The time step is set with *-deltat* (ps) and the number of steps with *-nsteps*.

3.2.2.1 Constraints

With the option *-constrain* the fastest motions can be removed from the simulation, using the RATTLE algorithm [97, 98]. Each constraint keeps the distance d_c between two atoms constant. The option has three choices:

- *no*: there are no constraints (the default).
- *bonds*: the length of each bond is kept at the reference bond length b_0 of the force field.
- *angles*: in addition to the bonds, for each angle $i - j - k$ the distance between the outer atoms i and k is kept constant. Together with the constrained bonds $i - j$ and $j - k$ this keeps the angle at its reference value θ_0, since

$$d_{ik} \;=\; \sqrt{b_{ij}^2 + b_{jk}^2 - 2\,b_{ij}\,b_{jk}\cos\theta_0}. \tag{3.59}$$

Not all angles can be constrained independently. The positions of n atoms bonded to the same atom, relative to each other, are determined by $2n - 3$ angles, for instance five of the six angles around a tetrahedral carbon. Around an atom with three neighbours of which the reference angles add up to more than 355°, only two angles are constrained, since the third one follows from the other two in a planar group. Angles larger than 175° are not constrained, since the distance d_{ik} does not determine a (nearly) linear angle. Finally, an angle constraint that depends linearly on the other constraints in the starting coordinates is left out as well. This is the case in rings, where the angles also have to close the ring. The log file lists the number of bond and angle constraints and the number of angles that were left out. The angles that are not constrained still contribute to the potential energy. Note that when the reference values of the angles in a ring are not consistent with a planar ring, the ring has to pucker to satisfy the constraints, which may not be possible from a planar starting structure. In that case the simulation stops with an error, and *-constrain bonds* can be used instead.

In the first part of RATTLE, the coordinates $\mathbf{r}'_i$ from Eqn. (3.55) are corrected by constraint forces along the distance vectors $\mathbf{r}_c(t)$ of the constrained pairs at time t

$$\mathbf{r}_i(t + \Delta t) \;=\; \mathbf{r}'_i + \frac{1}{m_i} \sum_c g_c \, \sigma_{ic} \, \mathbf{r}_c(t), \tag{3.60}$$

where σ_{ic} is 1 if atom i is the first atom of constraint c, -1 if it is the second and 0 otherwise. The multipliers g_c follow from the conditions $|\mathbf{r}_c(t + \Delta t)|^2 \;=\; d_c^2$. Since angle constraints are strongly coupled to the bond constraints, these non-linear equations are solved for all constraints together by Newton iterations, as in the matrix SHAKE algorithm [99], rather than one constraint at a time as in the original SHAKE algorithm, which converges very slowly for coupled constraints. The step is halved when it does not reduce the deviations from the constraints. The displacements due to the constraints, divided by Δt, are added to the velocities. In the second part of RATTLE, after Eqn. (3.56), the velocity components along the constraints are removed by solving the linear equations

$$\mathbf{r}_c \cdot (\mathbf{v}_{c_1} - \mathbf{v}_{c_2}) \;=\; 0 \tag{3.61}$$

for all constraints together, where c_1 and c_2 are the atoms of constraint c. The starting coordinates are constrained as well. The relative tolerance on the constrained distances and the maximum number of iterations are set with *-constr_toler* and *-constr_maxiter*. Each constraint removes one degree of freedom, such that N_c in Eqn. (3.58) is the number of constraints. For acetone, for instance, there are 9 bond constraints and 12 angle constraints, leaving 6 degrees of freedom: the rotation of the molecule, the rotation of the two methyl groups and the out of plane motion of the carbonyl carbon.

The constraints are applied in energy minimization (*alexandria minimize* or *alexandria simulate -minimize*) as well, with all three algorithms that can be selected with *-minalg*. The starting coordinates are constrained first. After each computation of the forces, the components of the forces along the constraints are removed in the same way as the velocity components in RATTLE, Eqn. (3.61), such that the forces vanish at a minimum of the energy under the constraints. These are the forces used by the minimization algorithms and for the convergence criterion *-toler*. After each step the coordinates are constrained with SHAKE, using the distance vectors before the step. With the Newton algorithm the step follows from the equations

$$\mathbf{H}\,\Delta\mathbf{x} + \mathbf{J}^T\boldsymbol{\mu} \;=\; \mathbf{F}, \qquad \mathbf{J}\,\Delta\mathbf{x} \;=\; 0, \tag{3.62}$$

where $\mathbf{H}$ is the second derivative matrix of the energy, $\mathbf{F}$ the forces, the rows of $\mathbf{J}$ are the derivatives of the constraints with respect to the coordinates and $\boldsymbol{\mu}$ are Lagrange multipliers. Since the steps close to the minimum are small, the constraints are satisfied to a relative tolerance of 10^3 times the machine precision in minimization, rather than *-constr_toler*. Frozen atoms (option *-freeze*) are not moved by the constraints. A planar ring of which the angle constraints are not consistent with a planar ring, as discussed above, has to pucker during the minimization, which the L-BFGS and Newton algorithms may not achieve from an exactly planar starting structure. The steepest descent algorithm, or *-constrain bonds*, can be used instead.

3.3 Force Field Training Algorithms

This chapter was written by Juli{'a}n Marrades and is based in part on his M.Sc. thesis :cite:p:`Marrades2022a`.

Within the *alexandria train_ff* module of the Alexandria Chemistry Toolkit you can choose among three algorithms to optimize force field parameters:

- Markov Chain Monte Carlo (flag *-optimizer MCMC*)
- Genetic Algorithm (flag *-optimizer GA*)
- Hybrid GA/MCMC (flag *-optimizer HYBRID*)

Let us see what they do behind the scenes and how to control them.

3.3.1 MCMC

Assume we want to set the values for five parameters (nParam = 5) by performing ten MCMC iterations (flag *-maxiter 10*). Then, the code "reads"

```
for (int i = 0; i < 10; i++)
  for (int j = 0; j < 5; j++)
    stepMCMC();
```

That is, we do 10 X 5 = 50 MCMC steps. What is a MCMC step though? In essence,

- *prevDev*: previous deviation from data
- Choose a parameter and alter it
- *newDev*: new deviation from data
- If *newDev* < *prevDev*, we accept the parameter change and continue into the next step. Else, apply Metropolis Criterion. That is, with some probability we accept the change. Otherwise, we restore the parameter to its previous value and proceed into the next step.

Now, let us add some more detail to the algorithm.

1. we already have the deviation from data of the previous step in *prevDev*.
2. we arbitrarily choose a parameter out of the *param* vector, say *param[i]*, which is bounded by its maximum *pmax[i]* and its minimum *pmin[i]*, yielding the range *prange[i] = pmax[i] - pmin[i]*. Here is where the *-step [0, 1]* flag comes into play by specifying a fraction.\
3. we draw a value of *delta*, which is uniformly distributed in *[- step * prange[i], + step * prange[i]]* and add it to the parameter value *param[i] += delta*. If the parameter has gone beyond its maximum, we set its value to the maximum. Same goes for the minimum.
4. we compute the deviation *newDev* from data of the modified parameter vector. If *newDev* < *prevDev*, we accept the change and finish the MCMC step. Otherwise, we apply the Metropolis Criterion and finish the step. Then go back to 1.

3.3.1.1 Metropolis Criterion

If Mathematics is your thing and you *really* want to know the nitty-gritty stuff, you have thorough explanations of this method on href{https://en.wikipedia.org/wiki/Metropolis%E2%80%93Hastings_algorithm}{Wikipedia} and Shuyi Qin's Master thesis [100]. Here, we shall say that the Metropolis

Criterion allows us to take steps that do not get us closer to a minimum, giving the opportunity to explore the parameter space and avoid local minima. How exactly?

The probability of accepting a "bad" parameter change is controlled by the flag *-temp T* flag and follows the equation

$$prob = exp(-deltaDev/T)$$

where deltaDev = newDev - prevDev. Note that the unit of temperature is the same as that of the deviation.

Given deltaDev, a higher temperature gives a higher probability of acceptance, as *-deltaDev/T* tends to *0* and *exp(0) = 1*. On the other hand, a lower T gives less chances of acceptance, as *-deltaDev/T* tends to $-\infty$ and $exp(-\infty) = 0$.

ACT gives us the possibility to lower the temperature (simulated annealing) during the MCMC optimization with the flag *-anneal_begin [0, 1] -anneal_end [0,1]* flag. If we use flag *-anneal_begin 0.5*, the temperature will be flag *-temp T* until 50% of the iterations have been completed. Then, it will be linearly decreased until it reaches *0* on the last iteration.

There are two remarks to be made here:

- The temperature is kept constant during the *nParam* MCMC steps that take place for a given iteration.
- Since division by *0* is not defined, we set *T = 1e-6* on the last iteration.

Fig. 3.2 shows a schematic example of the temperature over time with and without global annealing.

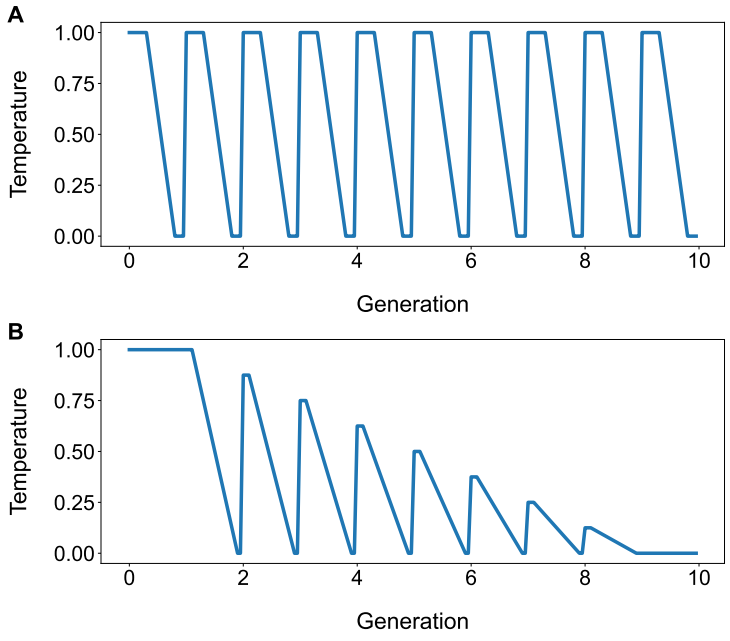


Fig. 3.2: A: Annealing during a HYBRID run with global annealing turned off, i.e. using *-maxiter 10 -temp 5 -anneal_begin 0.3 -anneal_end 0.8*. B: With global annealing turned on, i.e. using *-maxiter 10 -temp 5 -anneal_begin 0.1 -anneal_end 0.9 -anneal_global_begin 0.1 -anneal_global_end 0.9*.

3.3.1.2 Multiple MCMC runs

We can optimize several candidate solutions in parallel using the *-pop_size* flag. If the *-random_init* flag is set (default), each candidate solution will be initialized arbitrarily and respecting parameter ranges. If *-norandom_init* is employed, the candidate solution(s) will be initialized as specified by the force field file(s) provided by the user.

3.3.2 Genetic Algorithm

Quoting WikipediaGA

> In computer science and operations research, a genetic algorithm is a metaheuristic inspired by the process of natural selection that belongs to the larger class of evolutionary algorithms. Genetic algorithms are commonly used to generate high-quality solutions to optimization and search problems by relying on biologically inspired operators such as mutation, crossover and selection.

In this section, we describe our implementation of a genetic algorithm for force field parameterization. If this is the first time you hear about genetic algorithms and want to acquaint yourself, we recommend you to read Chapter 2 of Steven F. van Dijk's PhD thesis [101] and/or head over to Youtube.

Our implementation is based upon a class definition and an external function for computing random numbers (Listing *Class definition.*).

Listing 3.1: Class definition.

```
class Genome
{
    double params[];
    double deviation;
    double probability;
};

// Returns a sample of a random variable that is
// uniformly distributed in [0, 1]
double random();
```

then the genome evolution boils down to the code in Listing *Schematic of the evolution algorithm*

Listing 3.2: Schematic of the evolution algorithm

```
void evolve(int popSize, int nElites, int prCross, int prMut)
{
    Genome oldPop[popSize] = initialize(popSize); // Initialize
    Genome newPop[];
    computeDeviation(oldPop); // Deviation from data
    int generation = 0;
    do
    {
        sort(oldPop); // Sorting
        if (penalize(oldPop, generation)) // Penalty?
        {
            // Recompute deviation and sort again
            computeDeviation(oldPop);
            sort(oldPop);
        }
        generation++;
        computeProbabilities(oldPop); // Selection probabilities
        // Elitism
        for (int i = 0; i < nElites; i++)
```

(continues on next page)

(continued from previous page)

```
            newPop.add(oldPop[i]);
        // Rest of population
        for (int i = nElites; i < popSize; i += 2)
        {
            Genome parent1, parent2 = select(oldPop); // Selection
            Genome child1, child2;
            if (random() <= prCross) // Crossover?
                child1, child2 = crossover(parent1, parent2);
            else
                child1, child2 = parent1, parent2;
            // Mutation
            for (Genome child : {child1, child2})
            {
                mutate(child, prMut);
                newPop.add(child); // Add to new population
            }
        }
        computeDeviation(newPop); // Deviation from data
        oldPop = newPop; // Swap populations
        newPop.clear();  // Erase all genomes in the population
    }
    // Termination
    while (!terminate(oldPop, generation));
}
```

flag *popSize* and flag *nElites* are assumed to be even. Let us explore the different stages of the process.

3.3.2.1 Initialization

This stage fills the *params* field in the *Genome* class, generating popSize (flag *-pop_size*) genomes.

If we are using flag *-norandom_init*, the genomes will be initialized as specified by the force field file(s) provided. If we are employing flag *-random_init*, each genome will be initialized by setting a random value for each parameter, uniformly distributed over the allowed range.

The population does not have to match the number of processes. With more genomes than processes, as can be needed for problems with many parameters, each process is a worker that gets the next genome as soon as it is done with the previous one, during initialization and for each generation. The workers therefore balance the load among themselves: those that get genomes that are quicker to evaluate simply compute more of them. Each genome brings its own random numbers, so the results do not depend on the number of processes. This works with the GA and HYBRID optimizers. With a GPU, a few processes per GPU keep it busy, for instance about a dozen on an NVIDIA RTX A4500 with MPS, and the population can be much larger than that.

3.3.2.2 Deviation from data

Here we fill the *deviation* field in for each *Genome* in the population by computing the deviation from the dataset.

3.3.2.3 Sorting

Sorting is not a mandatory step but may be required depending on the GA components selected by the user. We sort the population in ascending order of *deviation*. Whether we sort or not is controlled by the flag *-sort* flag.

3.3.2.4 Penalties

At this stage, we may alter the population if certain conditions are met, with the main goal of preventing premature convergence and enforcing solution diversity.

> Covering such a small portion of the space you are. Broaden your search, you should. - Yoda

To that end, we have a function *penalize()* which returns *true* if the population was penalized and *false* otherwise.

For now, there are two components in this function:

- **Catastrophe.** Each flag *-cp_gen_interval* generations, a fraction flag *-cp_pop_frac [0, 1]* of the genomes in the population will be randomly reinitialized. Genomes to reinitialize are arbitrarily chosen.
- **Volume.** This option enables flag *-sort*.

If there are N parameters p_i, the population volume is computed as

$$V_{pop} = \prod_{i=1}^{N} [MAX(p_i) - MIN(p_i)]$$

where *MAX* and *MIN* retrieve the maximum and minimum values for each parameter in the population. If flag *-log_volume* is used, the volume will be computed in logarithmic scale

$$logV_{pop} = \sum_{i=1}^{N} log[MAX(p_i) - MIN(p_i)]$$

The total volume V_{tot} is computed in a similar manner, based on the bounds in the force field. Then, if the volume spanned by the population divided by the total volume of the parameter space, or

$$\left[\frac{V_{pop}}{V_{tot}}\right]^{1/N} < x$$

or, in log notation, if

$$logV_{pop} < Nlogx + logV_{tot}$$

where x is given by flag *-vfp_vol_frac_limit (0, 1]*, the *worst* fraction flag *-vfp_pop_frac [0, 1)* of genomes in the population will be randomly reinitialized. This is done repeatedly, but at most 100 times, until V_{pop} increases. To summarize it...

> Death comes equally to us all, and makes us all equal when it comes. - John Donne

3.3.2.5 Selection probabilities

We provide three options for computing selection probabilities:

1. Rank (flag *-prob_computer RANK*)

2. Fitness (flag *-prob_computer FITNESS*)
3. Boltzmann (flag *-prob_computer BOLTZMANN*)

The sum of the selection probabilities of the genomes in the population, is 1.

3.3.2.6 Rank

This option enables flag *-sort*. The selection probability depends exclusively on the index (rank) of genome in the population (Listing *Calculation of the probability from the order of probabilities.*).

Listing 3.3: Calculation of the probability from the order of probabilities.

```
for (int i = 0; i < popSize; i++)
{
  oldPop[i].probability = (popSize - i) / (popSize * (popSize + 1) / 2);
}
```

That is, the lower the index, the higher the probability of being selected. The independence of the *deviation* avoids the possible phenomena of a genome with a very high selection probability dominating the population.

3.3.2.7 Fitness

The selection probability is inversely proportional to the *deviation* (Listing *Calculation of the probability from the deviations from data.*).

Listing 3.4: Calculation of the probability from the deviations from data.

```
double total = 0;
double inverses[popSize];
double epsilon = 1e-4;
for (int i = 0; i < popSize; i++)
    inverses[i] = 1 / (epsilon + oldPop[i].deviation);
    total += inverses[i];
for (int i = 0; i < popSize; i++)
    oldPop[i].probability = inverses[i] / total;
```

3.3.2.8 Boltzmann

The *temperature* parameter is specified by the flag *-boltz_temp* flag and controls the smoothing of the selection probabilities (Listing *Use of Boltzmann-weighting when calculating the probability*). A higher value will avoid polarization in the probability values and vice versa. The flag *-boltz_anneal* flag allows us to decrease the temperature over time and has the same logic as flag *-anneal*, except that it targets the Boltzmann selection temperature and operates based on the maximum amount of generations.

Listing 3.5: Use of Boltzmann-weighting when calculating the probability

```
double total = 0;
double exponentials[popSize];
double epsilon = 1e-4;
```

(continues on next page)

(continued from previous page)

```
for (int i = 0; i < popSize; i++)
    exponentials[i] = exp(1 / (epsilon + oldPop[i].deviation) / temperature);
    total += exponentials[i];
for (int i = 0; i < popSize; i++)
    oldPop[i].probability = exponentials[i] / total;
```

3.3.2.9 Elitism

In order to avoid losing the best candidate solutions found so far, the GA will move the top nElites flag *-n_elites* genomes, *unchanged*, into the new population. That means, the genome will not undergo crossover nor mutation.

When flag *-n_elites* > *0*, flag *-sort* will be enabled.

3.3.2.10 Selection

Once the selection probabilities are computed, the population becomes a href{https://en.wikipedia.org/wiki/Probability_density_function}{probability density function} from which we can sample genomes based on their *probability*.

As of now, only a vanilla selector is available. It only looks at the probability and can select the same genome to be *parent1* and *parent2*.

3.3.2.11 Crossover

With certain probability *prCross*, controlled by the flag *-pr_cross* flag, two parents will combine their parameters to form two children. Right now, only an N-point crossover is available, where N is defined by the flag *-n_crossovers* flag.

If $N = 1$, we arbitrarily select on crossover point (v) and:

```
   v                       v
|X|X|X|X|X|X|X|X|  --> |X|X|Y|Y|Y|Y|Y|Y|
|Y|Y|Y|Y|Y|Y|Y|Y|  --> |Y|Y|X|X|X|X|X|X|
```

If $N = 2$, we arbitrarily select two crossover points (v) and:

```
  v     v                   v     v
|X|X|X|X|X|X|X|X|  --> |X|Y|Y|Y|X|X|X|X|
|Y|Y|Y|Y|Y|Y|Y|Y|  --> |Y|X|X|X|Y|Y|Y|Y|
```

If $N = 3$, we arbitrarily select three crossover points (v) and:

```
      v     v     v           v     v     v
|X|X|X|X|X|X|X|X|  --> |X|X|Y|Y|Y|X|X|X|
|Y|Y|Y|Y|Y|Y|Y|Y|  --> |Y|Y|X|X|X|Y|Y|Y|
```

If $N = 4 \ldots$ you get the idea (hopefully).

3.3.2.12 Mutation

Given the mutation probability *prMut*, controlled by flag *-pr_mut*, we iterate through the parameters. If the probability is met, we alter the parameter, otherwise, we leave it unchanged.

```
for (int i = 0; i < nParams; i++)
{
  if (random() <= prMut)
  {
    changeParam(params, i);
  }
}
```

The parameter is changed in the same way as in MCMC, by a fraction of its allowed range and not allowing values outside of it. The fraction in this case is controlled by the *-percentage* flag, which has the same meaning as *-step*, but applies to GA instead of MCMC.

3.3.2.13 Termination

This stage decides whether the GA evolution should continue or halt. We allow the user to tweak the termination criteria with several flags:

1. flag *-max_generations*: evolution will halt after so many generations.
2. flag *-max_test_generations* (disabled by default): evolution will halt if in the last flag *-max_test_generations* the best *deviation* of the test set found so far has not improved.

3.3.2.14 Fitness files

The *deviation* of each genome in the population is written to the fitness files, one for each data set, given by flag *-fitness*, at the start and then every flag *-nstfitness* fitness evaluations per genome (default 1). The last population is always written. A generation of GA has one fitness evaluation per genome, from the mutation, while a generation of HYBRID has flag *-maxiter* times the number of parameters, the number of MCMC steps. Since GA therefore needs many more generations than HYBRID, the two can be compared by using the same value of flag *-nstfitness* for both, for instance the number of MCMC steps in a HYBRID generation. With MCMC and HYBRID, the flag also sets the number of steps between writing to the parameter and *deviation* convergence files of each individual, which are written with flag *-v 4* or higher.

3.3.3 HYBRID

Even though this optimizer has a very fancy name, it is just a GA with MCMC as its mutator engine. When MCMC acts as a mutator, it will always alter the genomes independently of flag *-pr_mut*. Also, it is important to note that the simulated annealing by default is applied independently in each MCMC run. For instance, in case we would use flag *-max_generations 2 -maxiter 10 -temp 5 -anneal 0.5*, Fig. 3.2 A shows the temperature during the MCMC part.

However, when using the flag flag *-anneal_global_begin [0,1]*, the starting temperature of the annealing will be decreased in steps at the beginning of each generation (Fig. 3.2 B).

3.4 Fitness function

In genetic algorithm terminology, the idea is to maximize the fitness of the population. However, in practice the ACT minimizes a function $F(\Theta)$ where Θ is the force field genome, containing all the parameters. Zero indicates perfect correspondence to reference data and algorithmic bounds. The function consists of multiple components corresponding to observables that can be combined at will by the user:

$$F(\Theta) = \sum_{i=1}^{N_{obs}} \Omega_i \sum_{j=1}^{N_i} \mathcal{L}\left(x_{i,j}^{calc}(\Theta) - x_{i,j}^{ref}\right) + \Lambda(\Theta)$$

where N_{obs} is the number of observables, the Ω_i are the corresponding user-supplied weights for each of these and N_i is the number of individual data points x for observable i and $\mathcal{L}$ is the loss function selected. Finally, Λ adds a penalty to keep (some of the) parameters within bounds.

The supported loss functions can be described by first defining the deviation (for property i and data point j) where it should be noted that E does not have to indicate an energy

$$\delta E = x_{i,j}^{calc}(\Theta) - x_{i,j}^{ref}.$$

Loss functions that are implemented are the mean squared error (MSE):

$$\mathcal{L}_{MSE} = \delta E^2$$

the mean absolute error (MAE):

$$\mathcal{L}_{MAE} = |\delta E|$$

the Huber function [102]:

$$\mathcal{L}_{Huber} = a^2\left(\sqrt{(\delta E/a)^2 + 1} - 1\right)$$

where a is a scaling factor in the same units as δE.

and the Asinh function [103] which, following the authors, is defined using

$$\delta U = \delta E/a$$

and

$$s(x) = \sqrt{1 + x^2}$$

as:

$$\mathcal{L}_{Asinh} = a^2\left(1 - s(\delta U) + \delta U \ln\left[\delta U + s(\delta U)\right]\right).$$

3.4.1 Supported training targets

Table *Overview of command-line flags and corresponding observables that can be used in training a force field in the ACT.* lists the supported observables and the corresponding command line flags.

Table 3.3: Overview of command-line flags and corresponding observables that can be used in training a force field in the ACT.

Flag	Observable
-fc_epot	Potential energy of a compound.
-fc_force	Force components on each atom in a compound.
-fc_freq	Vibrational frequencies from a normal mode analysis (NMA).
-fc_inten	Intensities corresponding to the frequencies from NMA.
-fc_mu	Dipole components of a compound.
-fc_quad	Quadrupole components of a compound.
-fc_oct	Octopule components.
-fc_hexa	Hexadecapole components.
-fc_esp	Electrostatic potential (ESP) on all of the grid points. This can be modulated by the -watoms flag which introduces a weight for the ESP on the atoms.
-fc_pola	Six independent components of the molecular polarizability tensor.
-fc_inten	The total interaction energy from SAPT calculations.
-fc_elec	Electrostatic component of the interaction energy.
-fc_indu	Induction component of the interaction energy.
-fc_allel	Sum of the electrostatic and induction components.
-fc_disp	Dispersion component of the interaction energy.
-fc_exch	Exchange component of the interaction energy.
-fc_exch	Sum of exchange and induction components.
-fc_delta	The Δ HF part of the induction can be trained independently of the induction component. If this is used, this part should be subtracted from the induction to prevent double counting.
-fc_boun	Penalty for going outside bounds (see below).

3.4.2 Parameter mutability

A force field consists many parameters and typically only a subset is trained at any one time. Therefore, parameters are qualified by a mutability value as indicated in Table *Parameter mutabilities and their interpretation in ACT.*.

Table 3.4: Parameter mutabilities and their interpretation in ACT.

Mutability	Meaning
Fixed	This parameter will not be modified in training.
Bound	Parameter values will be strictly between the minimum and maximum value specified in the force field. If the initial value is outside the bounds, it will be reset to the nearest bound (either minimum or maximum). If the minimum is equal to the maximum the mutability will be set to Fixed.
Dependent	Parameter value is not trained directly, but may change due to changes in other parameters. An example would be a pair parameter such as σ_{ij} that is derived from σ_j and σ_j using combination rules.
ACM	Alexandria Charge Model is used for charges that are generated using the EEM or SQE algorithms (section *Charge Methods*). If different minimum and maximum values are given in the force field and the *-fc_bound* flag is used, a bounds penalty will be applied.

3.4.3 Bounds Penalty

A bounds penalty can be applied to charges when the mutability is ACM (Table *Parameter mutabilities and their interpretation in ACT.*). In that case the penalty V(q) is

$$V(q) = \begin{cases} (q - q_{min})^2 \text{ if } q < q_{min} \\ (q - q_{max})^2 \text{ if } q > q_{max} \\ 0 \text{ else} \end{cases}$$

Importantly, if a point+distributed model is used [94], then the sum of these charges is used as the target q. So if your force field specifies:

```
<particletype identifier="hw" type="Atom" description="hydrogen">
  <parameter type="charge" unit="e" value="0.5" uncertainty="0" minimum="0.3"␣
↪maximum="0.7" ntrain="0" mutability="ACM" nonnegative="no"/>
</particletype>
```

and the hw atom type is equiped with a distributed charge v1hw at the same position, it is the sum of these charges that will be penalized if it is outside the interval 0.3 - 0.7.

3.5 Sensitivity Analysis

The sensitivity analysis is a post-optimization tool that evaluates how sensitive the total chi-squared (χ^2) deviation is to small changes in each individual force field parameter and, optionally, in each pair of parameters. It helps identify which parameters have the most influence on the fitting quality, which parameters are correlated, and whether the optimization has converged to a well-defined minimum.

The sensitivity analysis runs automatically after parameter optimization. The *-sensitivity* option selects it: *1D* (the default) changes each parameter on its own, *2D* also changes every pair of parameters, and *No* disables the analysis. The analysis can also be run on an existing force field without re-training by combining it with the *-nooptimize* and *-norandom_init* flags.

Note

The sensitivity analysis is only available with the MCMC and HYBRID optimizers. It is not supported with the Genetic Algorithm (*-optimizer GA*).

3.5.1 Running Sensitivity Analysis

The sensitivity analysis runs automatically after training with MCMC or HYBRID. To disable it, pass *-sensitivity No*:

```
alexandria train_ff -sensitivity No -optimizer MCMC [other flags...]
```

To analyze an existing force field without re-training, use:

```
alexandria train_ff -nooptimize -norandom_init \
    -fc_inter -fit 'param1 param2 param3' [other flags...]
```

Here, *-fc_inter* selects intermolecular energy as the fitting target (alternatively *-fc_epot* for intramolecular energy), and *-fit* specifies the space-separated names of the parameters to analyze. Add *-sensitivity 2D* for the two-dimensional analysis, and *-sensout* to choose the name of the output file.

The analysis is done for the parameters as they are. Parameters that are zero in a force field that is not trained yet, like the Lennard-Jones parameters of a template force field, may not change χ^2 at all; they are reported as *flat*, see below.

3.5.2 Algorithm

For each trainable force field parameter p_i, the algorithm evaluates χ^2 at three nearby points, one of which is the parameter itself:

- $p_i - \delta p,\ p_i,\ p_i + \delta p$ if these are within the bounds
- $p_i,\ p_i + \delta p,\ p_i + 2\delta p$ if $p_i - \delta p$ is below the lower bound
- $p_i - 2\delta p,\ p_i - \delta p,\ p_i$ if $p_i + \delta p$ is above the upper bound

where $\delta p = (p_{\mathrm{upper}} - p_{\mathrm{lower}})/200$ (0.5% of the allowed range) and p_{lower}, p_{upper} are the allowed bounds for the parameter. The χ^2 of the parameter itself is that of the current parameter set, so two evaluations are needed per parameter.

The three $(p,\ \chi^2)$ pairs are then fit to a parabola

$$\chi^2(p) = a \cdot p^2 + b \cdot p + c$$

using least-squares regression. The fit is done in coordinates centred on p_i and scaled by δp, such that it stays accurate for parameters with narrow bounds, and converted to a, b, c afterwards. A curvature smaller than the rounding errors of χ^2 is set to zero. If χ^2 is the same at all three points, the parameter is *flat*: it does not change χ^2 here, and has neither curvature nor slope. The minimum of this parabola is located at

$$p_{\mathrm{opt}} = -\frac{b}{2a}, \qquad \chi^2_{\mathrm{min}} = a\,p_{\mathrm{opt}}^2 + b\,p_{\mathrm{opt}} + c.$$

All other parameters are kept at their trained values while each parameter is perturbed in turn, so the analysis captures the *local* sensitivity around the training solution. The analysis is performed on the training set only.

A parameter is *converged* if

1. the parabola curves upwards, $a > 0$, such that it has a minimum, and
2. the parameter is within 5% of its allowed range of that minimum, $|p_i - p_{\mathrm{opt}}| \leq 0.05\,(p_{\mathrm{upper}} - p_{\mathrm{lower}})$.

Here p_{opt} is limited to the bounds, such that a parameter at a bound is converged if the minimum of the parabola lies beyond that bound. The parameter set is in a minimum if all parameters are converged.

3.5.3 Two-dimensional analysis

Each parameter on its own can be at the minimum of its parabola while the parameter set is not in a minimum: two strongly correlated parameters form a valley in which each one is optimal given the other, and a saddle point looks like a minimum along each axis. With *-sensitivity 2D* every pair of parameters (p_i, p_j) is analysed as well, on the 3×3 grid of the three values of each parameter from the one-dimensional analysis. Five points of the grid are known from that analysis, the four corners, where both parameters differ from their own value, are computed. The surface is described as the sum of the two parabolas plus a coupling term

$$\chi^2(x, y) \approx P_i(x) + P_j(y) - \chi^2_0 + a_{ij}\,(x - p_i)(y - p_j)$$

where P_i and P_j are the parabolas of the parameters and χ^2_0 is χ^2 of the parameter set. The coupling is fitted to what the parabolas do not describe at the corners,

$$a_{ij} = \frac{\sum_k r_k\,\Delta x_k \Delta y_k}{\sum_k (\Delta x_k \Delta y_k)^2}, \qquad r_k = \chi^2_k - \left[P_i(x_k) + P_j(y_k) - \chi^2_0\right]$$

with $\Delta x_k = x_k - p_i$ and $\Delta y_k = y_k - p_j$. On a symmetric grid this is the mixed central difference, and it is exact for a quadratic surface, also at bounds. The root mean square of the corners that the surface does not describe is reported as *corner_rms*; a large value means that χ^2 is not quadratic on the scale of δp. The Hessian and the gradient of the pair at the parameter set are

$$H = \begin{pmatrix} 2a_i & a_{ij} \\ a_{ij} & 2a_j \end{pmatrix}, \qquad g = \begin{pmatrix} 2a_i p_i + b_i \\ 2a_j p_j + b_j \end{pmatrix}, \qquad \rho_{ij} = \frac{a_{ij}}{2\sqrt{a_i a_j}}$$

where the dimensionless coupling ρ_{ij} is defined if both curvatures are positive, and is between -1 and 1 for a minimum. The surface of a pair is

- a *minimum* if both curvatures and the determinant of H are positive, that is $|\rho_{ij}| < 1$,
- a *saddle* if the determinant is negative,
- a *maximum* if both curvatures are negative and the determinant is positive,
- *degenerate* if a curvature is zero, or the determinant is zero compared to its terms, for instance for a flat parameter or a valley without a minimum.

A pair is *converged* if its surface is a minimum, and both parameters are within 5% of their range of the minimum of the surface within the bounds. If the stationary point $p - H^{-1}g$ lies outside the bounds, that minimum is on an edge of the bounds, where the surface is a parabola in the other parameter. In 2D the parameter set is in a minimum if all parameters and all pairs are converged. The analysis takes $2N$ evaluations of χ^2 in 1D and $2N^2$ in 2D for N parameters:

Table 3.5: Number of evaluations of χ^2 of the training set

Parameters *N*	1D	2D	Pairs
10	20	200	45
42	84	3,528	861
100	200	20,000	4,950

They are divided over the master and the middlemen. The Hessian of all parameters follows from the pairs as well. It is scaled by the ranges of the parameters, $H_{ij}(p_i^{\mathrm{upper}} - p_i^{\mathrm{lower}})(p_j^{\mathrm{upper}} - p_j^{\mathrm{lower}})$, such that its eigenvalues do not depend on the units of the parameters, and the analysis reports whether it is positive definite, its extreme eigenvalues and the largest component of the Newton step $-H^{-1}g$ relative to the range of the parameter. All pairs can be minima while the Hessian of all parameters is not positive definite, so this is a stricter test; it is reported but not used for the convergence.

3.5.4 Convergence during training

During training with the HYBRID optimizer, the *-max_local_minimum_generations* option stops the training when the best parameter set has been in a minimum for that number of generations. The test for a minimum is two-dimensional with *-sensitivity 2D*, and one-dimensional otherwise, also with *-sensitivity No*. It is done in each generation, so the two-dimensional test costs $2N^2$ evaluations of χ^2 per generation. With *-v 3* the log file reports for each generation how many parameters, and in 2D pairs, are converged. Adaptive step sizes (*-adaptive_steps*) use the force constants of the one-dimensional analysis.

3.5.5 Output Format

The results are written to a JSON file, set with *-sensout* (default *sensitivity.json*). The log file only gets a summary:

```
Sensitivity analysis (1D) of 42 parameters on the Train set, 84 chi2␣
↪evaluations, chi2_0 = 0.482
Converged: 41/42 parameters, not in a minimum.
Details of the sensitivity analysis are in sensitivity.json
```

or, in 2D:

```
Sensitivity analysis (2D) of 42 parameters and 861 pairs on the Train set,␣
↪3528 chi2 evaluations, chi2_0 = 0.482
Converged: 41/42 parameters, 850/861 pairs (858 minimum, 3 saddle, 0 maximum,␣
↪0 degenerate), not in a minimum.
The Hessian of all parameters is positive definite.
Details of the sensitivity analysis are in sensitivity.json
```

If there are flat parameters, a line says how many.

and the json output of *train_ff* (*-json*) gets the same summary under the key *sensitivity*, with the name of the file. The file has the following structure, shown here for one parameter:

```
{
  "sensitivity": {
    "mode": "1D",
    "dataset": "Train",
    "chi2_0": 0.482,
    "n_parameters": 42,
    "n_evaluations": 84,
    "tolerance_fraction": 0.05,
    "minimum": false,
    "n_parameters_converged": 41,
    "n_parameters_flat": 0,
    "parameters": [
```

(continues on next page)

(continued from previous page)


```
      {
        "index": 0,
        "name": "OW epsilon",
        "particle": "OW",
        "parameter": "epsilon",
        "lower": 0.0,
        "upper": 20.0,
        "p_orig": 5.0,
        "chi2_orig": 0.482,
        "points": [ { "p": 4.9, "chi2": 0.496 },
                    { "p": 5.0, "chi2": 0.482 },
                    { "p": 5.1, "chi2": 0.495 } ],
        "fit_ok": true,
        "flat": false,
        "a": 1.35, "b": -13.6, "c": 34.4,
        "p_min": 5.002,
        "chi2_min": 0.482,
        "p_min_bounded": 5.002,
        "converged": true,
        "delta": 0.1,
        "force_constant": 0.054
      }
    ]
  }
}
```


The fields are:

- **mode** - the analysis that was done, *1D* or *2D*.
- **chi2_0** - the χ^2 of the current (trained) parameter set before any perturbation.
- **n_evaluations** - the number of χ^2 computations, not counting *chi2_0*.
- **minimum** - whether all parameters are converged, see above.
- **name**, **particle**, **parameter** - the force field parameter.
- **lower**, **upper**, **delta** - its bounds and the step δp.
- **points** - the parameter values at which χ^2 was computed, in increasing order, with their χ^2.
- **a, b, c** - coefficients of the fitted parabola $\chi^2(p) = ap^2 + bp + c$. The fit is done in coordinates centred on the parameter and scaled by the step, such that it stays accurate for parameters with narrow bounds. **fit_ok** is false if the parabola could not be fitted.
- **p_min**, **chi2_min** - the extremum of the parabola, *null* if the parabola is a straight line. **p_min_bounded** is *p_min* limited to the bounds.
- **flat** - whether χ^2 is the same at all points, and **n_parameters_flat** the number of flat parameters.
- **converged** - whether the parameter is converged, see above.
- **force_constant** - the dimensionless force constant a/p^2, used for adaptive step sizes.

In 2D the file has these as well:

```
{
  "sensitivity": {
    "n_pairs": 861,
    "n_pairs_converged": 850,
    "pair_shapes": { "minimum": 858, "saddle": 3, "maximum": 0, "degenerate":␣
↪0 },
    "pairs": [
      {
        "i": 0, "j": 1,
        "names": [ "OW epsilon", "OW sigma" ],
        "grid": { "p_i": [ 4.9, 5.0, 5.1 ],
                  "p_j": [ 0.312, 0.315, 0.318 ],
                  "chi2": [ [ 0.509, 0.496, 0.490 ],
                            [ 0.488, 0.482, 0.484 ],
                            [ 0.488, 0.495, 0.509 ] ] },
        "a_ij": 66.1, "rho": 0.91, "corner_rms": 2.1e-6,
        "hessian": [ [ 2.7, 66.1 ], [ 66.1, 1960.0 ] ],
        "gradient": [ -0.05, 0.42 ],
        "shape": "minimum",
        "p_min": [ 5.03, 0.3138 ], "chi2_min": 0.4818,
        "p_min_bounded": [ 5.03, 0.3138 ],
        "converged": true
      }
    ],
    "hessian_diagnostic": {
      "positive_definite": true,
      "eigenvalue_min": 0.21,
      "eigenvalue_max": 5.4e3,
      "max_newton_step_fraction": 0.012
    }
  }
}
```


- **grid** - the values of both parameters and χ^2 on the grid, *chi2[k][l]* at *p_i[k]* and *p_j[l]*.
- **a_ij**, **rho**, **corner_rms**, **hessian**, **gradient**, **shape** - the coupling, the dimensionless coupling, the deviation of the corners, the Hessian, the gradient and the shape of the pair, see above.
- **p_min**, **chi2_min** - the stationary point of a minimum and its χ^2, **p_min_bounded** the minimum within the bounds, all *null* if the pair is not a minimum.
- **hessian_diagnostic** - the Hessian of all parameters, see above.

Numbers are written with full precision, flags as *true* or *false* and undefined values as *null*.

3.5.6 Interpretation

The coefficient a of the fitted parabola quantifies how sharply χ^2 responds to changes in the parameter:

- **Large** $|a|$ - the χ^2 surface is steeply curved near the current value; the parameter is *sensitive* and must be determined accurately. A large a also indicates a well-defined minimum.
- **Small** $|a| \approx 0$ - the χ^2 surface is nearly flat; the parameter has little influence on the training targets in the explored range. Such parameters may be poorly constrained by the data and are candidates

for fixing at a reasonable value or removing from the fit.

The estimated minimum (**pmin**, **chi2min**) gives a first-order estimate of the optimal parameter value and the achievable χ^2 from the local curvature. If *pmin* is far from the trained value, the optimization may not have fully converged for that parameter, or the training data does not strongly constrain it. When a is very small (insensitive parameter), the estimated *pmin* extrapolates far outside the sampled range and should be treated with caution.

If the fitted parabola has *negative* curvature ($a < 0$), the current parameter value sits at a local *maximum* of the chi-squared surface, which suggests an ill-conditioned or degenerate optimization landscape for that parameter.

A *flat* parameter does not influence the training targets at all at its current value. In a trained force field it is not constrained by the data; in a force field that is not trained yet it is often zero together with a parameter it multiplies, like σ with $\epsilon = 0$, or a virtual site without charge. All pairs with a flat parameter are degenerate.

In 2D, ρ_{ij} close to 1 or -1 means that two parameters are strongly correlated: the data constrain a combination of them better than each one, and the trained values of each may not be meaningful on their own. A *saddle* means that the parameter set is not at a minimum even if both parameters are converged on their own; the training can still lower χ^2 by changing both.

The sensitivity analysis is particularly useful for:

- Identifying insensitive (poorly constrained) parameters that can be fixed or removed from the fitting.
- Identifying correlated parameters, with the two-dimensional analysis.
- Checking whether the optimization has converged near a well-defined minimum.
- Guiding subsequent optimization runs by revealing parameters that may benefit from a broader or finer search range.

3.6 Force field design

3.6.1 Physical Background

The Alexandria force-fields have a functional form consisting of van der Waals (*vdw*), electrostatics (*coul*), polarization (*pol*) and bonded terms including a radial (*b*), angular (*a*), out-of-plane dihedral (*i*) and torsion (*d*) terms.

$$E = V_{vdw}(r_{ij}) + V_{coul}(r_{ij}) + V_{pol}(r_{cs}) + V_b(r_{ij}) + V_a(\theta_{ijk}) + V_i(\phi_{ijkl}) + V_d(\phi_{ijkl}) \tag{3.63}$$

where r_{ij} refers to the distance between atoms i and j, r_{cs} to the distance between a core and a shell (Drude) particle, θ_{ijk} to the angle given by three atoms i, j and k, and ϕ_{ijkl} to a torsional angle between the planes given by atoms i, j, k and atoms j, k, l. For each of the terms, multiple functional forms are available, so that within the Alexandria framework, different force-fields, including previously published ones, can be reconstructed and compared to one another in a systematic manner [6]. For details on these potentials we refer to Chapter *Energy Function*.

In total there are seven *atom* parameter types and 7-14 *bond* parameter types. A variety of virtual sites can be used, like those used in water models [34, 104] or to model anisotropy due to σ-holes on halogen atoms or water [26].

3.6.2 Determining atom types

When importing structure files (SDF, PDB, or XYZ), ACT uses RDKit for the initial chemical perception, including sanitization and aromaticity assignment [105].

Attention

If bond information is present in the PDB or SDF file, it will be used by ACT. That means the user is responsible for providing correct information, unless the compound is a special case, see below. If there is no bond information in the PDB (or a XYZ) file, it will be generated using RDKit. There is no guarantee that the output from RDKit is correct in all cases, however. Please check your output.

Atom typing in ACT is based on SMARTS patterns stored in *share/atom_bond.xml* [106]. The distributed *atom_bond.xml* file assigns atom types for carbon, nitrogen, oxygen, phosphorus, and sulfur according to their hybridization state, adding 1,2,3 behind the element symbol. For all other elements, atom types are assigned based on formal charge. For neutral atoms, the default atom type is the lowercase element symbol.

The file also contains special cases, such as resonance structures or situations where RDKit assigns bond orders or hybridizations that differ from those required by the ACT force fields. For example, sulfate (SO_4^{2-}) is assigned with one sulfur as s3 and four oxygens with mixed types (o- and o2), which does not reflect the resonance equivalence of the oxygens. These special rules are typically placed at the top of the file.

You may provide your own *atom_bond.xml*. ACT always checks the current working directory first (the filename must be exactly *atom_bond.xml*), and if no file is found there, it falls back to the shared directory. The file is ordered from most specific to most general rules (see Table 3.6 and Table 3.7).

ACT provides two Jupyter notebooks to assist with inspecting and developing SMARTS patterns:

- *atom_bond_demo.ipynb*: Provide a specific molecule to obtain the assigned bonds and atom types, together with a labeled 3D visualization.
- *test_smarts.ipynb*: Provide a molecule and a SMARTS pattern and the notebook returns the matching atoms and a 2D depiction with highlighted hits.

Users are strongly encouraged to verify that the chemical interpretation is appropriate for their specific use case. If you need assistance creating new SMARTS patterns, please open an issue on GitHub.

The code to determine atomtypes is implemented in both the C++ code and in a Python file *get_mol_dict.py* that is part of the ACT distribution. The latter can be used in conjunction with the generating quantion chemistry data in the SaptACT repository.

Table 3.6: Special-case SMARTS patterns used by ACT

Molecule/Rule	SMARTS / recursive SMARTS	atom_type(s)	q	bo
sulfate	[#16](=[#8])(=[#8])(-[#8-])-[#8-]	s3, o2, o2, o2, o2	-2	1.5
phosphate2	[#8-]-[#15](-[#8-])(-[#8-])=[#8]	o3, p3, o3, o3, o3	-3	1.5
phosphate	[#15D4]([#8D1])([#8D1])([#8-,#8D1])([#8-,#8D1])	p3, o3, o3, o3, o3	-3	1.5
phosphate2 (variant 2)	[#8-]-[#15](-[#8-])(-[#8-])	o3, p3, o3, o3	-2	1.5
carboxylate	[#6X3](-[#8-])=[#8]	c2, o2, o2	-1	1.5
phosphate3	[#8-]-[#15](-[#8-])	o3, p3, o3	-1	1
nitro1	[#7+](-[#8-])=[#8]	n2, o2, o2	0	1.5
nitro2	[#7+](-[#8])=[#8]	n2, o2, o2	0	1.5
water	[#8](-[#1])-[#1]	ow, hw, hw	0	1

Table 3.7: Atom type SMARTS patterns used by ACT

Atom type rule	SMARTS / recursive SMARTS	atom_typ
gaff h5 ring 2EWG	[#1X1;$(*-[c](~[#7,#8,#16,#17,#35,#53])~[#7,#8,#16,#17,#35,#53])]	h5
gaff h4 ring 1EWG	[#1X1;$(*-[c]~[#7,#8,#16,#17,#35,#53])]	h4
gaff h3 chain 3EWG	[#1X1;$(*-[C]([#7,#8,#16,#17,#35,#53,F,Cl, | h3 | 0	
gaff hc tertcarbon like	[#1X1;$(*-[#6X4]([#6])([#6])[#1])]	hc
gaff h2 C eq 2EWG	[#1X1;$(*-[C](=[#7,#8,#9,#16])([#7,#8,#9,#16]))]	h2
gaff h2 chain 2EWG	[#1X1;$(*-[C]([#7,#8,#9,#16,#17,#35,#53])[#7,#8,#9,#16,#17,#35,#53])]	h2
gaff h1 chain 1EWG	[#1X1;$(*-[C]-[#7,#8,#9,#16,#17,#35,#53])]	h1
gaff h1 chain 1EWG double	[#1X1;$(*-[C]=[#7,#8,#9,#16])]	h1
gaff ha conjugated CX3 eq C	[#1X1;$(*-[#6X3]=[#6])]	ha
gaff ha triplebond	[#1X1;$(*-[#6X2]#[#6])]	ha
gaff hc aliphatic C	[#1X1;$(*-[#6X4])]	hc
hcl	[#1X1;$(*-Cl)]	hcl
hbr	[#1X1;$(*-Br)]	hbr
gaff ho oxygen	[#1X1;$(*-O)]	ho
gaff hn nitrogen	[#1X1;$(*-N)]	hn
gaff hn aromatic n	[#1X1;$(*-n)]	hn
gaff hs sulfur	[#1X1;$(*-S)]	hs
gaff hp phosphorus	[#1X1;$(*-P)]	hp
hf	[#1X1;$(*-F)]	hf
hi	[#1X1;$(*-I)]	hi
gaff ha generic	[#1X1]	ha
gaff hc any	[#1]	hc
p sp	[#15;^1]	p1
p sp2	[#15;^2]	p2
p sp3	[#15;^3]	p3
p sp3 sp3d	[#15;^4]	p3
p sp3 sp3d2	[#15;^5]	p3
s sp	[#16;^1]	s1

Table 3.7 – continued from previous page

Atom type rule	SMARTS / recursive SMARTS	atom_typ
s sp2	[#16;^2]	s2
s sp3	[#16;^3]	s3
s sp3 sp3d	[#16;^4]	s3
s sp3 sp3d2	[#16;^5]	s3
c sp	[#6;^1]	c1
c sp2	[#6;^2]	c2
c sp3	[#6;^3]	c3
c sp3 sp3d	[#6;^4]	c3
c sp3 sp3d2	[#6;^5]	c3
n sp3 +1	[#7+;^3]	n4
n sp	[#7;^1]	n1
n sp2	[#7;^2]	n2
n sp3	[#7;^3]	n3
n sp3 sp3d	[#7;^4]	n3
n sp3 sp3d2	[#7;^5]	n3
o sp	[#8;^1]	o1
o sp2	[#8;^2]	o2
o sp3	[#8;^3]	o3
o sp3 sp3d	[#8;^4]	o3
o sp3 sp3d2	[#8;^5]	o3
li +1	[Li+]	Li+
li 0	[Li]	li
na +1	[Na+]	Na+
na 0	[Na]	na
k +1	[K+]	K+
k 0	[K]	k
rb +1	[Rb+]	Rb+
rb 0	[Rb]	rb
cs +1	[Cs+]	Cs+
cs 0	[Cs]	cs
be +2	[Be+2]	Be2+
be 0	[Be]	be
mg +2	[Mg+2]	Mg2+
mg 0	[Mg]	mg
ca +2	[Ca+2]	Ca2+
ca 0	[Mg]	mg
al +3	[Al+3]	Al3+
al 0	[Al]	al
si -4	[Si-4]	Si4-
si +4	[Si+4]	Si4+
si 0	[Si]	si
p -3	[P-3]	P3-
p +3	[P+3]	P3+
p +5	[P+5]	P5+
f -1	[F-]	F-
cl -1	[Cl-]	Cl-
cl +1	[Cl+]	Cl+
cl +3	[Cl+3]	Cl3+

Table 3.7 – continued from previous page

Atom type rule	SMARTS / recursive SMARTS	atom_typ
cl +5	[Cl+5]	Cl5+
cl +7	[Cl+7]	Cl7+
cl 0	[Cl]	cl
br -1	[Br-]	Br-
br +1	[Br+]	Br+
br +3	[Br+3]	Br3+
br +5	[Br+5]	Br5+
br +7	[Br+7]	Br7+
i -1	[I-]	I-
i +1	[I+]	I+
i +3	[I+3]	I3+
i +5	[I+5]	I5+
i +7	[I+7]	I7+
br 0	[Br]	br
he 0	[He]	he
ne 0	[Ne]	ne
ar 0	[Ar]	ar
kr +2	[Kr+2]	Kr2+
kr 0	[Kr]	kr
xe 0	[Xe]	xe
xe +2	[Xe+2]	Xe2+
xe +4	[Xe+4]	Xe4+
xe +6	[Xe+6]	Xe6+
rn 0	[Rn]	rn
rn +2	[Rn+2]	Rn2+

3.6.3 Determining partial charges

The electrostatic potential (ESP, Section *Electrostatic Potential*) has historically been used to determine partial charges [107] and the ACT supports training models to reproduce the ESP. However, in a very recent paper we have shown that fitting charges to reproduce the ESP in a limited volume around a compound is fundamentally flawed due to lack of information [38]. If the purpose is to build models that reproduce electrostatic interactions, this can be done directly by training models to reproduce SAPT energy components. Here, the split-charge equilibration (SQE) algorithm [108] is used to generate the effective partial charge on each atom in a molecule. SQE, in turn, is based on the electronegativity equalization method (EEM), as developed by Rapp{'e} and Goddard [109]. In brief, EEM uses the atomic hardness η and electronegativity χ to determine the atomic charges in a molecule from a Taylor expansion of the molecular energy in terms of charges. The SQE algorithm introduces a correction to the atomic electronegativities for bonded atoms $\Delta\chi$ as well as a bond hardness $\Delta\eta$. With this addition, charge can *flow* through bonds only, which overcomes issues with over-polarization in the EEM [110].

The ACT code implements the possibility to generate charges for compounds in dimers or clusters where charge transfer between compounds is disallowed which is a reasonable approximation since charge transfer has been shown to have limited impact on the binding energy of non-covalent complexes [111]. For the SQE algorithm two atomic parameters (χ and η) as well as two bond parameter types ($\Delta\chi$ and $\Delta\eta$) need to be determined and the ACT can train SQE parameters to reproduce electrostatic and induction energies [38]. For background information we refer the reader to an excellent review by Jensen [112], but below follows a break-down of using SQE with shells or virtual sites.

3.6.4 Charge Equilibration with Shells or Virtual Sites

Among the approaches to modeling the charge-dependent component of a force field, those rooted in the chemical potential equalization principle are especially notable, as the principle stems directly from density functional theory [113]. The first computational implementation of the chemical potential equalization principle was the electronegativity equalization method (EEM) [109, 114]. However, due to limitations of this model, Chelli *et al.* proposed the atom-atom charge transfer (AACT) model [115]. Later, Nistor and co-workers combined the EEM and AACT approaches into a single framework, the split-charge equilibration (SQE) model, which fulfills the essential criteria for a successful charge-transfer potential [108, 110]. The ACT implements both the EEM and the SQE as algorithms for determining partial charges.

In brief, EEM minimizes an empirical model of the intramolecular electrostatic energy (computed from the atomic electronegativity χ_i and atomic hardness η_i) with respect to the atomic partial charges q_i, where i are the atoms. This method comprises a second order expansion of the molecular energy E_{EEM} in terms of the partial charges q_i:

$$E_{\mathrm{EEM}}(q_1, q_2, \ldots, q_N) = \sum_{i=1}^{N} \left[\chi_i q_i + \frac{1}{2}\eta_i q_i^2 + \frac{1}{2} \sum_{\substack{l=1 \\ l \neq i}}^{N} q_i q_l J_{il} \right], \tag{3.64}$$

where N is the number of atoms, χ_i are the atomic electronegativities, η_i the atomic hardness, and J_{il} the Coulomb interaction between atoms. The factor $\frac{1}{2}$ before the Coulomb matrix is to avoid double counting.

In this work, the SQE method is used, which addresses a shortcoming of the EEM, namely that molecules tend to get over-polarized [110]. For more background, we refer to the recent review on charge flow models by Jensen [112].

Verstraelen and co-workers proposed the following variant of the molecular energy:

$$E_{\mathrm{SQE}} = E_{\mathrm{EEM}} + \sum_{i,j}^{M} \left(\frac{1}{2}\Delta\eta_{ij} p_{ij}^2 + \Delta\chi_{ij}(q_i - q_j) \right) \tag{3.65}$$

where p_{ij} corresponds to the (intramolecular) charge transfer over bonds, $\Delta\eta_{ij}$ is the bond hardness and $\Delta\chi_{ij}$ is the bond electronegativity correction. Therefore, the charge variables q_i are replaced by charge-transfer variables p_{ij} which are related by

$$q_i = \frac{q_{\mathrm{tot}}}{N} + \sum_{\substack{i,j \\ \text{bonds}}} p_{ij}, \tag{3.66}$$

where q_{tot} is the net charge on the compound and $p_{ij} = -p_{ji}$. Although it is trivial to determine the partial charges q_i from the charge transfer p_{ij}, the reverse is not necessarily true. As outlined by Chen *et al.* [116], the problem can be solved by expressing the energy in terms of the charge transfer variables. By substituting $J_{ii} = \eta_i$ in Eq. (3.64), inserting Eq. (3.64) into Eq. (3.65) and introducing M_x as the number of bonds for species x, we obtain:

$$\begin{aligned} E_{\mathrm{SQE}} = &\sum_{n=1}^{N} \left[\left(\frac{q_{\mathrm{tot}}}{N} + \sum_{m=1}^{M_n} p_{nm} \right) \left(\chi_n + \frac{1}{2}\eta_n \left(\frac{q_{\mathrm{tot}}}{N} + \sum_{m=1}^{M_n} p_{nm} \right) \right. \right. \\ &\left. \left. + \frac{1}{2} \sum_{\substack{l=1 \\ l \neq n}}^{N} \left(\frac{q_{\mathrm{tot}}}{N} + \sum_{m=1}^{M_l} p_{lm} \right) J_{nl} \right) \right] \\ &+ \sum_{i,j}^{M} \left[\frac{1}{2}\zeta_{ij} p_{ij}^2 + \Delta\chi_{ij} \left(\left(\frac{q_{\mathrm{tot}}}{N} + \sum_{m=1}^{M_i} p_{im} \right) - \left(\frac{q_{\mathrm{tot}}}{N} + \sum_{m=1}^{M_j} p_{jm} \right) \right) \right]. \end{aligned}$$

The next step is to determine the p_{ij} that minimize E_{SQE}. Since all summations run over atoms i, j, k, l, we take the derivative with respect to p_{ij} and equate it to zero:

$$
\begin{aligned}
0 &= \frac{\partial E_{\mathrm{SQE}}}{\partial p_{ij}} \\
&= \left(\chi_i - \chi_j + \frac{q_{\mathrm{tot}}}{N}(\eta_i - \eta_j) + \sum_{k=1}^{M_i} \Delta\chi_{ik} - \sum_{k=1}^{M_j} \Delta\chi_{jk} \right) \\
&\quad + \frac{1}{2} \left(\sum_{l=1}^{N} J_{il} \left(\frac{q_{\mathrm{tot}}}{N} + \sum_{m=1}^{M_l} p_{lm} \right) - \sum_{l=1}^{N} J_{li} \left(\frac{q_{\mathrm{tot}}}{N} + \sum_{k=1}^{M_i} p_{ik} \right) \right) + p_{ij}\zeta_{ij},
\end{aligned}
$$

using the identity of the Coulomb-matrix elements ($J_{ij} = J_{ji}$), the terms involving the atomic hardness η_x are incorporated into the diagonals of the J_{xy} matrix, excluding the contribution from the total charge q_{tot}. Note that the q_{tot} terms in the two sums cancel. The first term in Eq. (3.67) is the difference in electronegativity between atoms i and j sharing the bond plus the correction; the second term represents the difference in electrostatic potentials at the atoms, and the third term accounts for the interaction between i and j times the charge transfer. This results in a coupled set of equations, written in matrix form as

$$\mathbf{M}\,\mathbf{P} = \mathbf{R},$$

where $\mathbf{M}$ is a square matrix of dimension equal to the number of bonds, $\mathbf{P}$ is the vector of the charge transfers for all bonds, and $\mathbf{R}$ is the right-hand side of the equations. The matrix elements are given by

$$M_{ij,kl} = J_{ik} - J_{il} - J_{jk} + J_{jl} + \delta_{ij,kl}\Delta\eta_{ij}$$

where $\delta_{ij,kl}$ is one if bond ij is identical to kl and zero otherwise. The right-hand side is defined by the electronegativity terms according to

$$R_{ij} = \chi_j - \chi_i + \sum_{k=1}^{M_j} \Delta\chi_{jk} - \sum_{k=1}^{M_i} \Delta\chi_{ik} + \frac{q_{\mathrm{tot}}}{N} \left(\sum_{l=1}^{N} J_{jl} - \sum_{l=1}^{N} J_{il} \right).$$

The charges of the shells (and virtual sites) are treated as constant in this algorithm, meaning that q_{tot} becomes the sum of the charges of the shells and virtual sites and the total charge of the compound. During force field training, all of these charges can be modified alongside the SQE parameters. As noted in the paper describing the ACT software [1], the SQE algorithm may not be flexible enough to reproduce optimal charge distributions, and other algorithms [112, 117] may need to be implemented in future versions of the software.

3.7 Force Field Training Targets

A number of different targets can be used for training force fields in the ACT. Below, we mention the most important ones including, where needed, the technical details necessary to appreciate the methods. For information on obtaining or generating training data we refer to Section *Training Data*.

Computing interaction energies and components of interaction energies is a crucial part of force field development. In the ACT we have hitherto used data from symmetry-adapted perturbation theory (SAPT) calculations [118, 119]. It is not entirely trivial to match the energy components from SAPT to force field terms, however.

3.7.1 Algorithm to compute energy components in ACT

The component of the interaction energy of a dimer can be computed from the difference between dimer and monomers A and B [120]:

$$E_x^{inter}(AB) = E_x^{total}(AB) - (E_x(A) + E_x(B)) \tag{3.67}$$

where x is exchange or dispersion and $total$ indicates that the energy includes both the intra- and intermolecular interactions. To compute the electrostatics or induction energy of a dimer in the gas phase, the ACT first computes the relaxed energy of the two monomers A and B, that is, the energy of the shell particles is minimized with respect to their positions, yielding $E_x(A)$ and $E_x(B)$. The rationale for this is that SAPT computes electrostatic energies between unperturbed monomers based on the response/relaxation of monomer Hartree-Fock (HF) orbitals in the electric field of the interacting partner. Then, the energies of the dimer AB are computed in three steps:

1. electrostatics is computed with shells located in the relaxed monomer positions, yielding $E_{elec}^{total}(AB)$,
2. the shells of compound A are allowed to relax (further) in the electric field from compound B, while shells of compound B remain at their monomer positions, and vice versa, yielding the second order relaxation $E_{induc}^{inter(2)}$ (see ref. [66] for details),
3. the shells are allowed to relax completely, yielding the total E_{induc} from which the higher order terms, named $E_{induc}^{inter(3)}$ here for convenience, can be derived by subtracting $E_{induc}^{inter(2)}$. According to ref. [66], parameters of models corresponding to the higher other terms, including, potentially, charge transfer, can be trained to the δ_{HF} contribution of the SAPT induction energy. Here, we have added the exponential term proposed by McDaniel and Schmidt for this purpose (section *Induction correction*).

The terms below can be compared directly to SAPT:

$$E_{elec}^{inter}(AB) = E_{elec}^{total}(AB) - (E_{ei}(A) + E_{ei}(B)) \tag{3.68}$$

$$E_{induc}^{inter(2)}(AB) = \left(E_{ei}^{total}\|_A + E_{ei}^{total}\|_B\right) - 2E_{elec}^{total}(AB) \tag{3.69}$$

$$E_{induc}^{inter(3)}(AB) = E_{ei}^{total}(AB) - E_{induc}^{(2)}(AB) - E_{elec}^{total}(AB) \tag{3.70}$$

where ei is short for $elec + induc$ and the notation $\|_X$ indicates that the shells of compound X are kept fixed in the relaxed monomer conformation. If we sum Eqns. (3.68)- (3.70) we recover Eqn. (3.67) where x equals ei. Eqn. (3.68) corresponds to the electrostatics in SAPT.

To summarize, ACT computes the induction term in two parts: a second order term $E_{induc}^{inter(2)}(AB)$ (Eqn. (3.69)) and a third-and-higher-order term $E_{induc}^{inter(3)}(AB)$ (Eqn. (3.70)), the sum of which corresponds to the total induction from SAPT.

McDaniel and Schmidt [66] proposed that Eqn. (3.70) should be equal to the δ_{HF} + δ_{MP2} terms from SAPT while Eqn. (3.69) would correspond to the polarization energy. They then continue to suggest how this can be implemented in a force field:

$$E_{induc} = E_{shell} + E_{pol} + E_{\delta HF}$$

where E_{shell} is Eqn. (3.8) and

$$E_{pol} = A_{ij}^{ind}\exp(-b_{ij}r_{ij})$$

where r_{ij} is the interatomic distance and b_{ij} a constant, and

$$E_{\delta HF} = A_{ij}^{\delta HF} \exp(-b_{ij} r_{ij})$$

where both A_{ij}^{ind} and $A_{ij}^{\delta_{HF}}$ are determined from a negative geometric combination rule

$$A_{ij} = -\sqrt{A_i A_j}$$

meaning these terms are always attractive. These potentials use the b_{ij} that is used in the exchange energy (using a Buckingham potential, Eqn. (3.13)). Whether this is the most appropriate way of splitting terms and reproducing SAPT energies remains to be determined.

In the output the ACT training module *alexandria train_ff* there are two terms related to induction. The term INDUCTIONCORRECTION refers to Eqn. (3.70). If that is present, the term INDUCTION refers to Eqn. (3.69), if not it refers to the sum of the two.

3.7.2 Monomer Energies and Forces

Typically, a series of single-point quantum calculations are done at a user-defined level of theory. These calculations can then be converted to a molprop file. More information to come.

3.7.3 Other Properties

In principle, all the molecular properties mentioned in Section *Predicting Molecular Properties* can be used for training, but it is highly recommended to leave some properties for validation.

3.8 Parameters for Force Field Training

Each potential available in the ACT has its own set of parameters. Those are listed in Table 3.8 for non-bonded parameters, Table 3.9 for bonded parameters. Parameters for virtual sites are given in Table 3.10 and for charge generation algorithms in Table 3.11. In total, well over 80 parameter categories can be trained using the ACT.

Table 3.8: Force field parameters related to non-bonded potentials that can be trained using ACT. Note that the parameter names are case-sensitive and, in some cases, they correspond to a Greek symbol in the equation.

Potential	Equation	Parameters
COULOMB_GAUSSIAN	(3.2)	zeta
COULOMB_SLATER	(3.5)	zeta
POLARIZATION	(3.8)	alpha
MACDANIEL_SCHMIDT	(3.9)	a1dexp bdexp
LJ14_7	(3.10)	sigma epsilon gamma delta
GENERALIZED_BUCKINGHAM	(3.11)	rmin epsilon gamma delta
WANG_BUCKINGHAM	(3.12)	sigma epsilon gamma
BUCKINGHAM	(3.13)	A b C
TANG_TOENNIES	(3.15)	Att btt c6tt c8tt c10tt
TT2	(3.16)	Att2b bExchtt2b c6tt2b c8tt2b c10tt2b
SLATER_ISA	(3.17)	A b
LJ12_6	(3.18)	sigma epsilon
LJ12_6_4	(3.19)	sigma epsilon gamma
BORN_MAYER	(3.20)	A b
Generalized mean	(3.36)	exponent

Table 3.9: Force field parameters related to bonded potentials that can be trained using ACT. Note that the parameter names are case-sensitive and, in some cases, they correspond to a Greek symbol in the equation.

Potential	Equation	Parameters
HARMONIC_BONDS	(3.37)	kb bondlength bondenergy
MORSE_BONDS	(3.38)	beta De bondlength D0
HUA_BONDS	(3.39)	De bondlength b c
HARMONIC_ANGLES	(3.40)	kt angle
LINEAR_ANGLES	(3.41)	klin a
HARMONIC_DIHEDRALS	(3.44)	kimp
FOURIER_DIHEDRALS	(3.45)	c0 c1 c2 c3 c4 c5
PROPER_DIHEDRALS	(3.46)	kp mult phi0

Table 3.10: Parameters related to virtual sites that can be trained using ACT. Note that the parameter names are case-sensitive. Equations will be added later.

Virtual Site	Equation	Parameters
VSITE1		vs1a
VSITE2		vs2a
VSITE2FD		vs2fd_a
VSITE3		vs3a vs3b
VSITE3S		vs3a
VSITE3FD		vs3fd_a vs3fd_b
VSITE3FAD		vs3fad_a vs3fad_b
VSITE3OUT		vs3out_a vs3out_b vs3out_c
VSITE4		vs4a vs4b vs4c
VSITE4S		vs4sa vs4sb
VSITE4S3		vs4s3a

Table 3.11: Parameters related to charge algorithms that can be trained using ACT. Note that the parameter names are case-sensitive. Equations will be added later.

Algorithm	Equation	Parameters
EEM	(3.64)	eta chi
SQE	(3.65)	eta chi delta_eta delta_chi

3.9 Training Data

3.9.1 Using existing data

A recent review from the ACT developers~ [8] discusses the different available quantum chemistry data sets. Some of these can be used in the ACT, for instance the coupled-cluster dimer data set due to Donchev *et al.* [28], the Non-covalent interaction atlas from the Czech group led by Řezáč~ [121] and some more. The SAPT dataset on protein side-chain analogs and backbone analogs by Burns~ [122] can in principle

be used in the ACT as well.

In principle, the ANI-1 dataset~ [123] can be used to provide off-equilibrium energies of small compounds, but in the first version it was limited to compound containing the elements C, H, N, O only.

3.9.2 Alexandria Library

The Alexandria Library contains energies at optimized conformations of about 5000 compounds, thermochemistry, electric multipoles and electrostatic potential at grid points around molecules~ [17, 18]. It is possible to train electrostatic models using data from the library.

3.9.3 Donchev data set

To use the dataset due to Donchev *et al.* [28], we provide a script that reads the comma-separated value file provided by those authors. Please refer to their article for download information. Then refer to the built-in help in the script for more guidance by executing:

```
donchev2molprop -h
```

and investigating the output.

3.9.4 Non-covalent interaction atlas

The Non-covalent interaction atlas~ [121] can be used in a similar manner. At the time of writing it can be href{http://www.nciatlas.org}{found here}. To convert the data to ACT files, start by:

```
ncia2molprop -h
```

and investigate your options.

3.9.5 ACT data

Quantum chemical data used in ACT-related publications will be uploaded to a sharing site.

3.9.6 Generating SAPT data

A recent review described the available data sets available for machine learning or force field training~ [8]. There is however a lack of certain data, or data sets are incomplete. For this reason there is a git repository SaptACT that provides script to run SAPT calculations using the Psi4 software~ [124] including tools to convert the output to ACT compatible inputs (see below). The steps needed to prepare data for training in ACT are as follows:

- Clone the SaptACT repository using *git*
- Prepare selection of compounds, e.g. water, methanol, ethanol, and make sure that monomer structures for these compounds are present in the *xyz/monomers* catalog.
- Generate a dimer selection file using the *gen_dimers.py* script in the SaptACT repository:

  ```
  ./gen_dimers.py -sel water methanol ethanol -o alcohol.dat
  ```

  which will generate a file containing:

```
water#water
water#methanol
water#ethanol
methanol#methanol
methanol#ethanol
ethanol#ethanol
```

where the # symbol separates the monomers. Note that the order on the command line determines the order in the selection file.

- Perform SAPT calculations using Psi4 by generating randomly oriented dimers at a series of distances defined by scaling the shortest distance between any pair of atoms in the generated orientations. In this case, six distances varying from 0.9 to 1.4 times the sum of the van der Waals radii of the nearest atoms will be generated.:

```
./run_calcs.py -dimers alchol.dat -ndist 6 -mindist 0.9 -maxdist 1.4 -
↪norient 10
```

Note, that in the above command 6 x 10 calculations are started for each dimer in *alcohol.dat*, that is 360 calculations in total. You will obviously need a compute cluster to do these calculations, in particular if you select a more accurate SAPT level of theory~ [119] than the default (sapt2+/aug-cc-pvdz).

3.9.7 Generating single molecule data

Single molecules off-equilibrium energies and forces are needed to parameterize the intramolecular potential functions. Scripts are available that will take a structure of a monomer, perform a 50 ps MD simulation in the gas phase at elevated temperature using the GAFF force field~ [125] and the GROMACS software~ [126]. Then, conformations are extracted from the simulation trajectory and these are subjected to quantum chemistry calculations using Psi4~ [124].

3.9.8 Conversion to ACT molprop files

The ACT uses the href{https://en.wikipedia.org/wiki/XML{eXtensible Markup Language} to store both data for training and force field files. Once your SAPT calculations are finished you need to perform the following steps to generate the ACT input:

- Convert the Psi4 outputs to compact and human-readable href{https://en.wikipedia.org/wiki/JSON}{json} files using another script in SaptACT:

```
./generate_json.py
```

which will traverse the output directories, find Psi4 outputs, and if there is not already a *results.json* file, will generate it. Please familiarize yourself with the content of these files.

- When the json files have been generated, you are ready to convert them to a molprop file:

```
./write_molprop.py -o molprop.xml
```

please inspect the output file from this script to verify that your calculations are present.

Both these scripts have more flags that can be useful, please investigate those using the *-h* option.

CHAPTER

FOUR

INTERACTIVE NOTEBOOKS

4.1 Charge Methods

A number of different algorithms are available in ACT.

4.1.1 Electronegativity equalization method

The electronegativity equalization method (EEM) is a second order expansion of the molecular energy E_{EEM} in terms of the partial charges q_i:

$$E_{EEM}(q_1, q_2, ..., q_N) = \sum_{i=1}^{N} \left[\chi_i q_i + \frac{1}{2}\eta_i q_i^2 + \frac{1}{2} \sum_{l \neq i}^{N} q_i q_l J_{il} \right]$$

where N is the number of atoms, χ_i are the atomic electronegativities, η_i the atomic hardness and J_{il} the Coulomb interaction between atoms. The factor 1/2 before the Coulomb matrix is to avoid double counting. In equilibrium, the chemical potential is the same for each atom, χ_{eq}. The EEM can be implemented by taking the derivative of E_{EEM} with respect to q_i and equating it to zero:

$$0 = \frac{\partial E_{EEM}}{\partial q_i} = \chi_{eq} = \chi_i + q_i \eta_i + \frac{1}{2} \sum_{l \neq i}^{N} q_l J_{il}$$

which leads to a set of $N + 1$ equations that is linear in the q_i and that can be solved using linear algebra tools. After taking the derivative with respect to q_i a matrix equation is obtained:

$$\begin{bmatrix} \eta_1 & J_{12} & J_{13} & ... & J_{1N} & -1 \\ J_{21} & \eta_2 & J_{23} & ... & J_{2N} & -1 \\ ... & ... & ... & ... & ... & ... \\ J_{N1} & J_{N2} & J_{N3} & ... & \eta_N & -1 \\ 1 & 1 & 1 & ... & 1 & 0 \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ ... \\ q_N \\ \chi_{eq} \end{bmatrix} = \begin{bmatrix} -\chi_1 \\ -\chi_2 \\ ... \\ -\chi_N \\ q_{tot} \end{bmatrix}$$

Note the last column in the matrix is there to make sure that the electronegativity for all atoms is the same (χ_{eq}), while the last row is there to make sure the total charge q_{tot} is maintained. A good reference for the method is the classic paper by Rappe and Goddard.

In the code below we test the method on a methanol molecule, using parameters from Verstraelen et al. as well as experimental data from Cordero et al..

```
import numpy as np
from scipy.linalg import lstsq
import math
from charge_utils import *
from test_systems import *
```

(continues on next page)

(continued from previous page)

```
def solveEEM(names, coords, qtotal, model, verbose=False):
    # Compute Coulomb
    J = calcJEEM(names, coords, model)
    N = coords.shape[0]

    # Right hand side of the equation
    rhs = np.zeros(N+1,dtype=float)
    chi = get_chi(model)
    for i in range(N):
        if names[i] in chi:
            rhs[i] = -chi[names[i]]
        else:
            print("No chi for %s" % names[i])
            exit(1)
    rhs[N] = qtotal
    q = np.linalg.solve(J, rhs)
    if verbose:
        print("J = \n{}".format(J))
        print("rhs = {}".format(rhs))
        print("q = {}".format(q))
        y = np.dot(J,q)
        print("y = {}".format(y))
    return q

def run_compound(molname, verbose, alexandria=False):
    mol = get_system(molname)
    if not mol:
        print("No test system %s defined" % molname)
    else:
        print("\n%s" % molname)
        q = solveEEM(mol["names"], mol["coords"], mol["qtotal"], verbose,␣
↪alexandria)
        for i in range(mol["coords"].shape[0]):
            print("q[%s] = %g" % (mol["names"][i], q[i]))
        printDipole(q, mol["coords"], mol["atomnr"], False)

verbose = False

for compound in [ "carbon-monoxide", "water", "methanol", "acetate" ]:
    run_compound(compound, "ACM-g", verbose)
```

```
carbon-monoxide
q[c2] = 0.100795
q[o] = -0.100795
Dipole = 0.0580969 Debye

water
q[hw] = 0.130157
q[ow] = -0.260313
```

(continues on next page)

(continued from previous page)

```
q[hw] = 0.130157
Dipole = 0.731574 Debye

methanol
q[oh] = -0.252355
q[hp] = 0.13162
q[c3] = -0.151608
q[h1] = 0.0878756
q[h1] = 0.087886
q[h1] = 0.0965812
Dipole = 1.22594 Debye

acetate
q[c3] = -0.254055
q[cm] = -0.108471
q[om] = -0.322013
q[om] = -0.322018
q[hc] = 0.00241518
q[hc] = 0.00166854
q[hc] = 0.00247338
Dipole = 0.841977 Debye
```

Note the positive charge on O and negative charge on one of the H with the EEM chi and eta values from Verstraelen. These charges are too small since the experimental dipole for methanol is about 1.69 Debye.

4.1.2 Split charge equilibration

The split charge equilibration (SQE) method postulates that partial charges q_i on atoms can only arise from transfer of charge p_{ij} between bonded atoms i and j only, according to:

$$q_i = \frac{q_{tot}}{N} + \sum_{i,j}^{bonds} p_{ij}$$

where q_{tot} is the net charge on the compound and $p_{ij} = -p_{ji}$. Although it is trivial to determine the partial charges q_i from the charge transfer p_{ij}, the reverse is not necessarily true. To exemplify this we write the equations for methanol:

$$\begin{bmatrix} 0 & p_{12} & p_{13} & p_{14} & p_{15} & 0 \\ -p_{12} & 0 & 0 & 0 & 0 & 0 \\ -p_{13} & 0 & 0 & 0 & 0 & 0 \\ -p_{14} & 0 & 0 & 0 & 0 & 0 \\ -p_{15} & 0 & 0 & 0 & 0 & p_{56} \\ 0 & 0 & 0 & 0 & 0 - p_{56} & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \end{bmatrix}$$

With these equations we can express the p_{ij} in terms of the charges after manual Gaussian elimination as follows:

$$\begin{bmatrix} p_{12} \\ p_{13} \\ p_{14} \\ p_{15} \\ p_{56} \end{bmatrix} = \begin{bmatrix} -q_2 \\ -q_3 \\ -q_4 \\ -q_5 - q_6 \\ -q_6 \end{bmatrix}$$

with q_1 by definition $q_{tot} - q_2 - q_3 - q_4 - q_5 - q_6$ (with q_{tot} = 0). However, expressing the p_{ij} in q_i does not guarantee that the charge transfer through space is zero. Therefore it is necessary to do the reverse. As outlined by Chen et al. the problem can be solved by expressing the energy in terms of the charge transfer variables.

Verstraelen and co-workers used the following variant of the molecular energy:

$$E_{SQE} = E_{EEM} + \sum_{i,j}^{M} \left(\tfrac{1}{2}\zeta_{ij} p_{ij}^2 + \Delta\chi_{ij}(q_i - q_j) \right)$$

In this formulation, the energy is not just a function of the charges, but also of the charge transfer over M bonds. ζ_{ij} is the bond hardness and $\Delta\chi_{ij}$ is the electronegativity correction, both of these are bond-specific. We therefore replace the charge variable by charge transfer variables and substitute $J_{ii} = \eta_i$, and for clarity we replace the summation over bonds by a summation over atom pairs, which is the same thing noting that only p_{ij} for chemical bonds are non-zero (note that we have to introduce a factor 1/2 to avoid double counting). If we introduce M_x as the number of bonds for species x we obtain

$$E_{SQE} = \sum_{n=1}^{N} \left[\left(\frac{q_{tot}}{N} + \sum_{m=1}^{M_n} p_{nm} \right) \left(\chi_n + \tfrac{1}{2}\eta_n \left(\frac{q_{tot}}{N} + \sum_{m=1}^{M_n} p_{nm} \right) + \tfrac{1}{2} \sum_{l \neq n}^{N} \left[\frac{q_{tot}}{N} + \sum_{m=1}^{M_l} p_{lm} \right] J_{nl} \right) \right] + \sum_{i,j}^{M} \left[\tfrac{1}{2}\zeta_{ij} p_{ij}^2 + \Delta\chi_{ij} \left(\left[\frac{q_{tot}}{N} + \sum_{m=1}^{M_i} p_{im} \right] - \left[\frac{q_{tot}}{N} + \sum_{m=1}^{M_j} p_{jm} \right] \right) \right]$$

Our task now is to find the p_{ij} that minimize E_{SQE}. Please note that the all summations are over atoms i, j, k, l. Therefore, we take the derivative and equate it to zero:

$$0 = \frac{\partial E_{SQE}}{\partial p_{ij}} = \left(\chi_i - \chi_j + \frac{q_{tot}}{N}(\eta_i - \eta_j) + \sum_{k=1}^{M_i} \Delta\chi_{ik} - \sum_{k=1}^{M_j} \Delta\chi_{jk} \right) + \frac{1}{2} \left(\sum_{l=1}^{N} J_{il} \left(\frac{q_{tot}}{N} + \sum_{m=1}^{M_l} p_{lm} \right) - \sum_{i=1}^{N} J_{li} \left(\frac{q_{tot}}{N} + \sum_{k=1}^{M_i} p_{ik} \right) \right) + p_{ij}\zeta_{ij}$$

using the identity of the Coulomb-matrix elements ($J_{ij} = J_{ji}$) and where the terms involving the atomic hardness η_x are included on the diagonals of the J_{xy} matrix, except for the contribution of the total charge q_{tot}. Note that the q_{tot} terms in the two sums cancel. The first term is the difference in electronegativity between atoms i and j sharing the bond plus the correction, the second term represents the difference in electrostatic potentials at the atoms, the third term is the interaction between i and j times the charge transfer. This leads to a coupled set of equations, written in matrix form as:

$$\mathbf{M}P = R$$

where $\mathbf{M}$ is a square matrix of dimension number of bonds, P is a vector containing the charge transfers for all bonds and R the right hand side of the equations. If the summations are all written out we obtain the result published by Chen et al. for QTPIE and similar algorithms:

$$M_{ij,kl} = J_{ik} - J_{il} - J_{jk} + J_{jl}$$

for off-diagonal elements of the matrix. For the diagonal elements, the bond hardness ζ_{ij} needs to be added. The right hand side is given by the electronegativity terms according to:

$$R_{ij} = \chi_j - \chi_i + \sum_{k=1}^{M_j} \Delta\chi_{jk} - \sum_{k=1}^{M_i} \Delta\chi_{ik} + \frac{q_{tot}}{N} \left(\sum_{l=1}^{N} J_{jl} - \sum_{l=1}^{N} J_{il} \right).$$

The equations are implemented in Python below.

```
import numpy as np
from scipy.linalg import lstsq
```

(continues on next page)

(continued from previous page)

```
import math
from charge_utils import *
from test_systems import *

def calcRHS(names, bonds, JEEM, qtotal, model):
    Natoms = len(names)
    Nbond = len(bonds)
    RHS = np.zeros(Nbond, dtype=float)
    ##### calc corrected chi ####
    chi_corr = []
    chi = get_chi(model)
    for i in range(Natoms):
        chi_corr.append(chi[names[i]])
        for k in range(Nbond):
            ai  = names[bonds[k][0]]
            aj  = names[bonds[k][1]]
            if bonds[k][0] == i:
                chi_corr[i] += getDeltaChi(ai, aj, model)
            if bonds[k][1] == i:
                chi_corr[i] -= getDeltaChi(ai, aj, model)
        for l in range(Natoms):
            chi_corr[i] += JEEM[i][l]*(qtotal/Natoms)
    #### calc EN diff for the bonds ####
    for i in range(Nbond):
        RHS[i] = chi_corr[bonds[i][1]] - chi_corr[bonds[i][0]]
    return RHS

def solveSQE(names, coords, bonds, verbose, qtotal, model):
    JEEM, JSQE = calcJSQE(names, coords, bonds, verbose, model)
    rhs  = calcRHS(names, bonds, JEEM, qtotal, model)
    if verbose:
        print("RHS: {}".format(rhs))
    pij  = np.linalg.solve(JSQE, rhs)
    if verbose:
        print("pij: {}".format(pij))
    q =  np.zeros(coords.shape[0], dtype=float)
    for n in range(len(bonds)):
        bi = bonds[n][0]
        bj = bonds[n][1]
        q[bi] += pij[n]
        q[bj] -= pij[n]
    Natoms = names.shape[0]
    for i in range(Natoms):
        q[i] += qtotal/Natoms
    print("qSQE: {}".format(q))
    return q

def run_compound(molname, verbose, shells):
    mol = get_system(molname)
```

(continues on next page)

(continued from previous page)

```
    print("%s" % molname)
    for bonds in mol["bonds"]:
        q = solveSQE(mol["names"], mol["coords"], bonds, verbose, mol["qtotal
↪"], "ACS-g")
    printDipole(q, mol["coords"], mol["atomnr"], False)

verbose = False
shells = False

for compound in [ "methanol", "water", "acetate", "carbon-monoxide" ]:
    run_compound(compound, verbose, shells)
```

```
methanol
qSQE: [-0.50802387  0.33691938 -0.33742533  0.16436908  0.16438559  0.
↪17977516]
qSQE: [-0.50802387  0.33691938 -0.33742533  0.16436908  0.16438559  0.
↪17977516]
qSQE: [-0.50802387  0.33691938 -0.33742533  0.16436908  0.16438559  0.
↪17977516]
qSQE: [-0.50802387  0.33691938 -0.33742533  0.16436908  0.16438559  0.
↪17977516]
qSQE: [-0.50802387  0.33691938 -0.33742533  0.16436908  0.16438559  0.
↪17977516]
Dipole = 2.07033 Debye
water
qSQE: [ 0.11909703 -0.23819405  0.11909703]
qSQE: [ 0.11909703 -0.23819405  0.11909703]
qSQE: [ 0.11909703 -0.23819405  0.11909703]
Dipole = 0.669411 Debye
acetate
qSQE: [-0.48654202 -0.0231567  -0.24282282 -0.24282009 -0.00136991 -0.00196462
 -0.00132384]
qSQE: [-0.48654202 -0.0231567  -0.24282282 -0.24282009 -0.00136991 -0.00196462
 -0.00132384]
Dipole = 1.41698 Debye
carbon-monoxide
No hardness information for bond c2-o
No deltaChi information for bond c2-o
No deltaChi information for bond c2-o
qSQE: [ 0.77205802 -0.77205802]
No hardness information for bond o-c2
No deltaChi information for bond o-c2
No deltaChi information for bond o-c2
qSQE: [ 0.77205802 -0.77205802]
Dipole = 0.445003 Debye
```

4.2 Atom Typing using RDKit

CHAPTER

FIVE

DEVELOPER MANUAL

Please checkout the Doxygen documentation.

BIBLIOGRAPHY

[1] David van der Spoel, Julián Marrades, Kristian Kříž, A. Najla Hosseini, Alfred T. Nordman, João Paulo Ataide Martins, Marie-Madeleine Walz, Paul J. van Maaren, and Mohammad M. Ghahremanpour. Evolutionary machine learning of physics-based force fields in high-dimensional parameter-space. *Digit. Discovery*, 4:1925–1935, 2025. doi:10.1039/d5dd00178a.

[2] David van der Spoel and Erik Lindahl. Brute-Force Molecular Dynamics Simulations of Villin Headpiece: Comparison with NMR Parameters. *J. Phys. Chem. B*, 107(40):11178–11187, 2003. doi:10.1021/jp034108n.

[3] Oliver F Lange, David van der Spoel, and Bert L de Groot. Scrutinizing Molecular Mechanics Force Fields on the Submicrosecond Timescale with NMR Data. *Biophys. J.*, 99:647–655, 2010. doi:10.1016/j.bpj.2010.04.062.

[4] Haiyang Zhang, Chunhua Yin, Yang Jiang, and David van der Spoel. Force field benchmark of amino acids: i. hydration and diffusion in different water models. *J. Chem. Inf. Model.*, 58:1037–1052, 2018.

[5] Zahedeh Bashardanesh and David van der Spoel. Impact of dispersion coefficient on simulations of proteins and organic liquids. *J. Phys. Chem. B.*, 122:8018–8027, 2018. doi:10.1021/acs.jpcb.8b05770.

[6] David van der Spoel. Systematic design of biomolecular force fields. *Curr. Opin. Struct. Biol.*, 67:18–24, 2021. doi:10.1016/j.sbi.2020.08.006.

[7] David van der Spoel, Jin Zhang, and Haiyang Zhang. Quantitative predictions from molecular simulations using explicit or implicit interactions. *Wiley Interdiscip. Rev.-Comput. Mol. Sci.*, 12:e1560, 2022. doi:{10.1002/wcms.1560}.

[8] Kristian Kříž, Lisa Schmidt, Alfred T. Andersson, Marie-Madeleine Walz, and David van der Spoel. An imbalance in the force: the need for standardised benchmarks for molecular simulation. *J. Chem. Inf. Model.*, 63:412–431, 2023. doi:10.1021/acs.jcim.2c01127.

[9] C Caleman, P J van Maaren, M Hong, J S Hub, L T Costa, and D van der Spoel. Force field benchmark of organic liquids: density, enthalpy of vaporization, heat capacities, surface tension, compressibility, expansion coefficient and dielectric constant. *J. Chem. Theory Comput.*, 8:61–74, 2012. doi:10.1021/ct200731v.

[10] Nina M. Fischer, Paul J. van Maaren, Jonas C. Ditz, Ahmet Yildirim, and David van der Spoel. Properties of liquids in molecular dynamics simulations with explicit long-range Lennard-Jones interactions. *J. Chem. Theory Comput.*, 11:2938–2944, 2015. doi:10.1021/acs.jctc.5b00190.

[11] Jin Zhang, Badamkhatan Tuguldur, and David van der Spoel. Force field benchmark II: Gibbs energy of solvation of organic molecules in organic liquids. *J. Chem. Inf. Model.*, 55:1192–1201, 2015. doi:10.1021/acs.jcim.5b00106.

[12] Jin Zhang, Badamkhatan Tuguldur, and David van der Spoel. Correction to force field benchmark II: Gibbs energy of solvation of organic molecules in organic liquids. *J. Chem. Inf. Model.*, 56:819–820, 2016. doi:10.1021/acs.jcim.6b00081.

[13] David van der Spoel, Mohammad Mehdi Ghahremanpour, and Justin Lemkul. Small molecule thermochemistry: a tool for empirical force field development. *J. Phys. Chem. A*, 122:8982–8988, 2018. doi:10.1021/acs.jpca.8b09867.

[14] Lisa Schmidt, David van der Spoel, and Marie-Madeleine Walz. Probing phase transitions in organic crystals using atomistic md simulations. *ACS Phys Chem Au*, 3:84–93, 2023. doi:10.1021/acsphyschemau.2c00045.

[15] A. Najla Hosseini and David van der Spoel. Simulations of amyloid-forming peptides in the crystal state. *Prot. J.*, 42:192–204, 2023. doi:10.1007/s10930-023-10119-3.

[16] Mohammad Mehdi Ghahremanpour, Paul J. van Maaren, Jonas Ditz, Roland Lindh, and David van der Spoel. Large-scale calculations of gas phase thermochemistry: enthalpy of formation, standard entropy and heat capacity. *J. Chem. Phys.*, 145:114305, 2016. doi:10.1063/1.4962627.

[17] Mohammad Mehdi Ghahremanpour, Paul J. van Maaren, and David van der Spoel. The Alexandria library: a quantum chemical database of molecular properties for force field development. *Sci. Data*, 5:180062, 2018. doi:10.1038/sdata.2018.62.

[18] Mohammad Mehdi Ghahremanpour, Paul J. van Maaren, and David van der Spoel. Alexandria library [data set]. Zenodo. 2017. URL: https://dx.doi.org/10.5281/zenodo.1004711.

[19] Marie Madeleine Walz, Mohammad M. Ghahremanpour, Paul J. van Maaren, and David van der Spoel. Phase-transferable force field for alkali halides. *J. Chem. Theory Comput.*, 14:5933–5948, 2018. doi:10.1021/acs.jctc.8b00507.

[20] Marie-Madeleine Walz and David van der Spoel. Molten alkali halides - temperature dependence of structure, dynamics and thermodynamics. *Phys. Chem. Chem. Phys.*, 21:8516–18524, 2019. doi:10.1039/C9CP03603B.

[21] Marie-Madeleine Walz and David van der Spoel. Systematically improved melting point prediction: a detailed physical simulation model is required. *Chem. Comm.*, 55:12044–12047, 2019. doi:10.1039/C9CC06177K.

[22] Marie-Madeleine Walz and David van der Spoel. Direct link between structure, dynamics and thermodynamics in molten salts. *J. Phys. Chem. C.*, 123:25596–25602, 2019. doi:10.1021/acs.jpcc.9b07756.

[23] Marie-Madeleine Walz and David van der Spoel. Microscopic origin of conductivity in molten salts unraveled by computer simulations. *Commun. Chem.*, 4(9):1–10, 2021. doi:10.1038/s42004-020-00446-2.

[24] David van der Spoel, Henning Henschel, Paul J. van Maaren, Mohammad M. Ghahremanpour, and Luciano T. Costa. A potential for molecular simulation of compounds with linear moieties. *J. Chem. Phys.*, 153(8):084503, 2020. doi:10.1063/5.0015184.

[25] Kristian Kříž, Paul J. van Maaren, and David van der Spoel. Impact of combination rules, level of theory and potential function on the modelling of gas and condensed phase properties of noble gases. *J. Chem. Theory Comput.*, 20:2362–2376, 2024. doi:10.1021/acs.jctc.3c01257.

[26] Kristian Kříž and David van der Spoel. Quantification of anisotropy in exchange and dispersion interactions: a simple model for physics-based force fields. *J. Phys. Chem. Lett.*, 15:9974–9978, 2024. doi:10.1021/acs.jpclett.4c02034.

[27] Paul J. van Maaren and David van der Spoel. Quantitative evaluation of anharmonic bond potentials for molecular simulations. *Digit. Discov.*, 4:824–830, 2025. doi:10.1039/D4DD00344F.

[28] Alexander G. Donchev, Andrew G. Taube, Elizabeth Decolvenaere, Cory Hargus, Robert T. McGibbon, Ka-Hei Law, Brent A. Gregersen, Je-Luen Li, Kim Palmo, Karthik Siva, Michael Bergdorf, John L. Klepeis, and David E. Shaw. Quantum chemical benchmark databases of gold-standard dimer interaction energies. *Sci. Data*, 8(1):55, 2021. doi:10.1038/s41597-021-00833-x.

[29] Peter Eastman, Raimondas Galvelis, Raúl P. Peláez, Charlles R. A. Abreu, Stephen E. Farr, Emilio Gallicchio, Anton Gorenko, Michael M. Henry, Frank Hu, Jing Huang, Andreas Krämer, Julien Michel, Joshua A. Mitchell, Vijay S. Pande, João PGLM Rodrigues, Jaime Rodriguez-Guerra, Andrew C. Simmonett, Sukrit Singh, Jason Swails, Philip Turner, Yuanqing Wang, Ivy Zhang, John D. Chodera, Gianni De Fabritiis, and Thomas E. Markland. Openmm 8: molecular dynamics simulation with machine learning potentials. *J. Phys. Chem. B.*, 128:109–116, 2024. doi:10.1021/acs.jpcb.3c06662.

[30] Mohammad Mehdi Ghahremanpour, Paul J. van Maaren, Carl Caleman, Geoffrey R. Hutchison, and David van der Spoel. Polarizable drude model with s-type gaussian or slater charge density for general molecular mechanics force fields. *J. Chem. Theory Comput.*, 14:5553–5566, 2018. doi:10.1021/acs.jctc.8b00430.

[31] Daniel S D Larsson and David van der Spoel. Screening for the location of RNA using the chloride ion distribution in simulations of virus capsids. *J. Chem. Theory Comput.*, 8:2474–2483, 2012. doi:10.1021/ct3002128.

[32] B G Dick and A W Overhauser. Theory of the dielectric constants of alkali halide crystals. *Phys. Rev.*, 112:90–103, 1958. doi:10.1103/PhysRev.112.90.

[33] P C Jordan, P J van Maaren, J Mavri, D van der Spoel, and H J C Berendsen. Towards Phase Transferable Potential Functions: Methodology and Application to Nitrogen. *J. Chem. Phys.*, 103:2272–2285, 1995. doi:10.1063/1.469703.

[34] P J van Maaren and D van der Spoel. Molecular dynamics simulations of water with a novel shell-model potential. *J. Phys. Chem. B.*, 105:2618–2626, 2001. doi:10.1021/jp003843l.

[35] P. Dauber-Osguthorpe and A.T. Hagler. Biomolecular force fields: where have we been, where are we now, where do we need to go and how do we get there? *J. Comput. Aid. Mol. Des.*, 33:133–203, 2019. doi:10.1007/s10822-018-0111-4.

[36] A.T. Hagler. Force field development phase II: relaxation of physics-based criteria... or inclusion of more rigorous physics into the representation of molecular energetics. *J. Comput. Aided Mol. Des.*, 33:205–264, 2019. doi:10.1007/s10822-018-0134-x.

[37] Zhifeng Jing, Chengwen Liu, Sara Y. Cheng, Rui Qi, Brandon D. Walker, Jean-Philip Piquemal, and Pengyu Ren. Polarizable force fields for biomolecular simulations: recent advances and applications. *Ann. Rev. Biophys.*, 48(1):371–394, 2019. doi:10.1146/annurev-biophys-070317-033349.

[38] A. Najla Hosseini, Kristian Kříž, and David van der Spoel. Beyond partitioning: using force field science to evaluate electrostatics models. *J. Chem. Theory Comput.*, 22: , 2026. doi:10.1021/acs.jctc.6c00039.

[39] Herman J. C. Berendsen. *Simulating the physical world*. Cambridge University Press, Cambridge, 2007.

[40] Frank Jensen. *Introduction to computational chemistry*. John Wiley & Sons Ltd, West Sussex, UK, 2007.

[41] P Calaminici, K Jug, and M Köster. Density funtional calculations of molecular polarizabilities anf hyperpolarizabilities. *J. Chem. Phys.*, 109(18):7756, 1998. doi:10.1063/1.477421.

[42] A. J Stone. *The Theory of Intermolecular Forces*. Oxford University Press, Great Clarendon Street, Oxford, ox2 6dp, UK, 2013.

[43] Henning Henschel, Alfred T. Andersson, Willem Jespers, Mohammad Mehdi Ghahremanpour, and David van der Spoel. Theoretical infrared spectra: quantitative similarity measures and force fields. *J. Chem. Theory Comput.*, 16(5):3307–3315, 2020. doi:10.1021/acs.jctc.0c00126.

[44] M. J. Frisch, G. W. Trucks, H. B. Schlegel, G. E. Scuseria, M. A. Robb, J. R. Cheeseman, G. Scalmani, V. Barone, G. A. Petersson, H. Nakatsuji, X. Li, M. Caricato, A. V. Marenich, J. Bloino, B. G. Janesko, R. Gomperts, B. Mennucci, H. P. Hratchian, J. V. Ortiz, A. F. Izmaylov, J. L. Sonnenberg, D. Williams-Young, F. Ding, F. Lipparini, F. Egidi, J. Goings, B. Peng, A. Petrone, T. Henderson, D. Ranasinghe, V. G. Zakrzewski, J. Gao, N. Rega, G. Zheng, W. Liang, M. Hada, M. Ehara, K. Toyota, R. Fukuda, J. Hasegawa, M. Ishida, T. Nakajima, Y. Honda, O. Kitao, H. Nakai, T. Vreven, K. Throssell, J. A. Montgomery, Jr., J. E. Peralta, F. Ogliaro, M. J. Bearpark, J. J. Heyd, E. N. Brothers, K. N. Kudin, V. N. Staroverov, T. A. Keith, R. Kobayashi, J. Normand, K. Raghavachari, A. P. Rendell, J. C. Burant, S. S. Iyengar, J. Tomasi, M. Cossi, J. M. Millam, M. Klene, C. Adamo, R. Cammi, J. W. Ochterski, R. L. Martin, K. Morokuma, O. Farkas, J. B. Foresman, and D. J. Fox. Gaussian 16 Revision A.03. 2016. Gaussian Inc. Wallingford CT.

[45] A D Becke. Density-functional exchange-energy approximation with correct asymptotic-behavior. *Phys. Rev. A*, 38:3098–3100, 1988. doi:10.1103/PhysRevA.38.3098.

[46] R A Kendall, T H Dunning, Jr., and R J Harrison. Electron affinities of the first-row atoms revisited. Systematic basis sets and wave functions. *J. Chem. Phys.*, 96:6796–6806, 1992. URL: http://dx.doi.org/10.1063/1.462569.

[47] D E Woon and T H Dunning, Jr. Benchmark calculations with correlated molecular wave functions. I. Multireference configuration interaction calculations for the second row diatomic hydrides. *J. Chem. Phys.*, 99:1914–1929, 1993. doi:10.1063/1.464303.

[48] D E Woon and T H Dunning, Jr. Gaussian basis sets for use in correlated molecular calculations. III. The atoms aluminum through argon. *J. Chem. Phys.*, 98:1358–1371, 1993.

[49] William L. Jorgensen, Mohammad M. Ghahremanpour, Anastasia Saar, and Julian Tirado-Rives. OPLS/2020 force field for unsaturated hydrocarbons, alcohols, and ethers. *J. Phys. Chem. B.*, 128:250–262, 2023. doi:10.1021/acs.jpcb.3c06602.

[50] John Rumble. *CRC Handbook of Chemistry and Physics 103rd edition*. CRC Press, Gaitherburg, MD, 2022. URL: http://hbcp.chemnetbase.com/.

[51] C G Gray and K E Gubbins. *Theory of Molecular Fluids*. Volume 9 of The International Series of Monographs on Chemistry. Oxford University Press, Oxford, 1984.

[52] D A McQuarrie. *Statistical Mechanics*. Harper & Row, New York, 1976.

[53] G K Schenter, S M Kathmann, and B C Garrett. Dynamical benchmarks of the nucleation kinetics of water. *J. Phys. Chem. A.*, 116:4275–4280, 2002.

[54] I. Amdur and E. A. Mason. Properties of gases at very high temperatures. *Phys. Fluids*, 1:370–383, 09 1958. doi:10.1063/1.1724353.

[55] W L Jorgensen, J Chandrasekhar, J D Madura, R W Impey, and M L Klein. Comparison of simple potential functions for simulating liquid water. *J. Chem. Phys.*, 79:926–935, 1983. doi:10.1063/1.445869.

[56] Claude Millot, Jean-Christophe Soetens, MarÃlia T. C. Martins Costa, Matthew P. Hodges, and Anthony J. Stone. Revised anisotropic site potentials for the water dimer and calculated properties. *J. Phys. Chem. A.*, 102:754–770, 01 1998. doi:10.1021/jp972578+.

[57] A. J. Stone, A. Dullweber, O. Engkvist, E. Fraschini, M. P. Hodges, A. W. Meredith, D. R. Nutt, P. L. A. Popelier, and D. J. Wales. *Orient: a program for studying interactions between molecules*. 2018. version 5.0. arXiv:https://gitlab.com/anthonyjs/orient.git.

[58] Brian J. Kirby and Pavel Jungwirth. Charge scaling manifesto: a way of reconciling the inherently macroscopic and microscopic natures of molecular simulations. *J. Phys. Chem. Lett.*, 10:7531–7536, 2019. doi:10.1021/acs.jpclett.9b02652.

[59] S W Rick, S J Stuart, and B J Berne. Dynamical fluctuating charge force fields: Application to liquid water. *J. Chem. Phys.*, 101:6141–6156, 1994. doi:10.1063/1.468398.

[60] R Hentschke, E M Aydt, B Fodi, and E Schöckelmann. *Molekulares Modellieren mit Kraftfeldern*. Bergische Universität Wuppertal, Wuppertal, Germany, 2004. URL: http://constanze.materials.uni-wuppertal.de.

[61] Anders Öhrn, Jose M. Hermida-Ramon, and Gunnar Karlström. Method for slater-type density fitting for intermolecular electrostatic interactions with charge overlap. I. The model. *J. Chem. Theory Comput.*, 12(5):2298–2311, 2016. doi:10.1021/acs.jctc.5b01155.

[62] I I Guseinov. Analytical evaluation of two-centre coulomb, hybrid and one-electron integrals for slater-type orbitals. *J. Phys. B: Atom. Molec. Phys.*, 3(11):1399, 1970. doi:10.1088/0022-3700/3/11/001.

[63] D van der Spoel and P J van Maaren. The origin of layer structure artifacts in simulations of liquid water. *J. Chem. Theory Comput.*, 2:1–11, 2006. doi:10.1021/ct0502256.

[64] T Darden, D York, and L Pedersen. Particle mesh Ewald: An N-log(N) method for Ewald sums in large systems. *J. Chem. Phys.*, 98:10089–10092, 1993. doi:10.1063/1.464397.

[65] U Essmann, L Perera, M L Berkowitz, T Darden, H Lee, and L G Pedersen. A smooth particle mesh Ewald method. *J. Chem. Phys.*, 103:8577–8592, 1995. doi:10.1063/1.470117.

[66] Jesse G. McDaniel and J.R. Schmidt. Physically-motivated force fields from symmetry-adapted perturbation theory. *J. Phys. Chem. A.*, 117(10):2053–2066, 2013. doi:10.1021/jp3108182.

[67] Thomas A Halgren. The representation of van der Waals (vdW) interactions in molecular mechanics force fields: potential form, combination rules, and vdW parameters. *J. Amer. Chem. Soc.*, 114:7827–7843, 1992. doi:10.1021/ja00046a032.

[68] Jasper C Werhahn, Evangelos Miliordos, and Sotiris S Xantheas. A new variation of the buckingham exponential-6 potential with a tunable, singularity-free short-range repulsion and an adjustable long-range attraction. *Chem. Phys. Lett.*, 619:133–138, 2015. doi:10.1016/j.cplett.2014.11.051.

[69] Lee-Ping Wang, Jiahao Chen, and Troy Van Voorhis. Systematic Parametrization of Polarizable Force Fields from Quantum Chemistry Data. *J. Chem. Theory Comput.*, 9(1):452–460, 2013. doi:10.1021/ct300826t.

[70] R A Buckingham. The Classical Equation of State of Gaseous Helium, Neon and Argon. *Proc. R. Soc. London Ser. A*, 168:264–283, 1938. doi:10.1098/rspa.1938.0173.

[71] Edward A. Mason. Transport Properties of Gases Obeying a Modified Buckingham (Exp-Six) Potential. *J. Chem. Phys.*, 22:169–186, 2004. doi:10.1063/1.1740026.

[72] K T Tang and J P Toennies. An improved simple-model for the vanderwaals potential based on universal damping functions for the dispersion coefficients. *J. Chem. Phys.*, 80:3726–3741, 1984. doi:10.1063/1.447150.

[73] K. T. Tang and J. P. Toennies. The van der Waals potentials between all the rare gas atoms from He to Rn. *J. Chem. Phys.*, 118(11):4976–4983, 03 2003. URL: https://doi.org/10.1063/1.1543944, arXiv:https://pubs.aip.org/aip/jcp/article-pdf/118/11/4976/19209973/4976_1_online.pdf, doi:10.1063/1.1543944.

[74] Xiaowei Sheng, J. Peter Toennies, and K. T. Tang. Conformal analytical potential for all the rare gas dimers over the full range of internuclear distances. *Phys. Rev. Lett.*, 125:253402, 2020. doi:10.1103/PhysRevLett.125.253402.

[75] Mary J. Van Vleet, Alston J. Misquitta, Anthony J. Stone, and J. R. Schmidt. Beyond born-mayer: improved models for short-range repulsion in ab initio force fields. *J. Chem. Theory Comput.*, 12(8):3851–3870, 2016. doi:10.1021/acs.jctc.6b00209.

[76] Mary J. Van Vleet, Alston J. Misquitta, and J. R. Schmidt. New angles on standard force fields: toward a general approach for treating atomic-level anisotropy. *J. Chem. Theory Comput.*, 14(2):739–758, 2018. PMID: 29266931. doi:10.1021/acs.jctc.7b00851.

[77] J E Jones. On the determination of molecular fields. -II. From the equation of state of a gas. *Proc. Royal Soc. Lond. Ser. A*, 106:463–477, 1924. doi:10.1098/rspa.1924.0082.

[78] Christian L. Wennberg, Teemu Murtola, Berk Hess, and Erik Lindahl. Lennard-Jones lattice summation in bilayer simulations has critical effects on surface tension and lipid properties. *J. Chem. Theory Comput.*, 9:3527–3537, 2013. doi:10.1021/ct400140n.

[79] Chang Lyoul Kong and Manoj R Chakrabarty. Combining rules for intermolecular potential parameters. iii. application to the exp 6 potential. *J. Phys. Chem.*, 77(22):2668–2670, 1973. doi:10.1021/j100640a019.

[80] D Berthelot. Sur le mélange des gaz. *C. R. Hebd. Seances Acad. Sci.*, 126:1703–1855, 1898.

[81] H. A. Lorentz. Ueber die anwendung des satzes vom virial in der kinetischen theorie der gase. *Annalen der Physik*, 248(1):127–136, 1881. doi:10.1002/andp.18812480110.

[82] W Hogervorst. Transport and equilibrium properties of simple gases and forces between like and unlike atoms. *Physica*, 51(1):77–89, 1971. doi:10.1016/0031-8914(71)90138-8.

[83] Li Yang, Lei Sun, and Wei-Qiao Deng. Combination Rules for Morse-Based van der Waals Force Fields. *J. Phys. Chem. A*, 122:1672–1677, 2018. doi:10.1021/acs.jpca.7b11252.

[84] P M Morse. Diatomic molecules according to the wave mechanics. II. Vibrational levels. *Phys. Rev.*, 34:57–64, 1929. doi:10.1103/PhysRev.34.57.

[85] Edward A. Mason. Forces between Unlike Molecules and the Properties of Gaseous Mixtures. *J. Chem. Phys.*, 23:49–56, 1955. doi:10.1063/1.1740561.

[86] M Waldman and A T Hagler. New combining rules for rare gas van der waals parameters. *J. Comput. Chem.*, 14:1077–1084, 1993. doi:10.1002/jcc.540140909.

[87] Rui Qi, Qiantao Wang, and Pengyu Ren. General van der Waals potential for common organic molecules. *Bioorg. Med. Chem.*, 24:4911–4919, 2016. doi:10.1016/j.bmc.2016.07.062.

[88] Uwe Hohm. A continuous description of different means with application to mixing rules. *ACS Omega*, 11:10641–10648, 2026. doi:10.1021/acsomega.5c12377.

[89] W Hua. 4-parameter exactly solvable potential for diatomic-molecules. *Phys. Rev. A*, 42(5):2524–2529, SEP 1 1990. doi:10.1103/PhysRevA.42.2524.

[90] Wei Hua. Four-parameter potential and its bound-state matrix elements. *J. Phys. B: Atom., Molec. Opt. Phys.*, 23:2521, aug 1990. doi:10.1088/0953-4075/23/15/019.

[91] J P Ryckaert and A Bellemans. Molecular Dynamics of Liquid n-Butane near its Boiling Point. *Chem. Phys. Lett.*, 30:123–125, 1975. doi:10.1016/0009-2614(75)85513-8.

[92] David van der Spoel, Erik Lindahl, Berk Hess, Gerrit Groenhof, Alan E Mark, and Herman J C Berendsen. GROMACS: Fast, Flexible and Free. *J. Comput. Chem.*, 26:1701–1718, 2005. doi:10.1002/jcc.20291.

[93] Peter Eastman and Vijay S Pande. Efficient Nonbonded Interactions for Molecular Dynamics on a Graphics Processing Unit. *J. Comput. Chem.*, 31:1268–1272, 2010. doi:10.1002/jcc.21413.

[94] David van der Spoel and A. Najla Hosseini. Point + Gaussian charge model for electrostatic interactions derived by machine learning. *Phys. Chem. Chem. Phys.*, 27:13817–13820, 2025. doi:10.1039/D5CP01254F.

[95] M W Mahoney and W L Jorgensen. A five-site model for liquid water and the reproduction of the density anomaly by rigid, nonpolarizable potential functions. *J. Chem. Phys.*, 112:8910–8922, 2000. doi:10.1063/1.481505.

[96] W C Swope, H C Andersen, P H Berens, and K R Wilson. A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules: Application to small water clusters. *J. Chem. Phys.*, 76:637–649, 1982. doi:10.1063/1.442716.

[97] J-P Ryckaert, G Ciccotti, and H J C Berendsen. Numerical integration of the cartesian equations of motion of a system with constraints: molecular dynamics of n-alkanes. *J. Comput. Phys.*, 23:327–341, 1977. doi:10.1016/0021-9991(77)90098-5.

[98] H C Andersen. Rattle: A "velocity" version of the Shake algorithm for molecular dynamics calculations. *J. Comput. Phys.*, 52:24–34, 1983. doi:10.1016/0021-9991(83)90014-1.

[99] V Kräutler, W F van Gunsteren, and P H Hünenberger. A fast SHAKE algorithm to solve distance constraint equations for small molecules in molecular dynamics simulations. *J. Comput. Chem.*, 22:501–508, 2001. doi:10.1002/1096-987X(20010415)22:5<501::AID-JCC1021>3.0.CO;2-V.

[100] Shuyi Qin. Sensitivity analysis in high-dimensional space. Master's thesis, Uppsala University, Dept. of Information Technology, 2021. URL: https://uu.diva-portal.org/smash/record.jsf?pid=diva2%3A1614100&dswid=6669.

[101] Steven F. van Dijk. *Genetic Algorithms for Map Labeling*. PhD thesis, Utrecht University, Dept. of Mathematics and Computer Science, 2001. URL: https://dspace.library.uu.nl/bitstream/handle/1874/864/full.pdf?sequence=1.

[102] Peter J. Huber. Robust estimation of a location parameter. *Ann. Math. Statist.*, 35:73–101, 1964.

[103] Mark DelloStritto and Michael L. Klein. Improved loss functions for machine-learned atomic potentials. *J. Chem. Phys.*, 163(13):134108, 10 2025. URL: https://doi.org/10.1063/5.0280032, arXiv:https://pubs.aip.org/aip/jcp/article-pdf/doi/10.1063/5.0280032/20721431/134108_1_5.0280032.pdf, doi:10.1063/5.0280032.

[104] G Lamoureux, A D MacKerell, and B Roux. A simple polarizable model of water based on classical Drude oscillators. *J. Chem. Phys.*, 119:5185–5197, 2003. doi:10.1063/1.1598191.

[105] Greg Landrum and RDKit contributors. Rdkit/rdkit: 2025_09_4 (q3 2025) release. 2025. doi:10.5281/zenodo.18098214.

[106] Daylight Chemical Information Systems, Inc. Smarts — a language for describing molecular patterns. Accessed: 2026-01-26. URL: https://www.daylight.com/dayhtml/doc/theory/theory.smarts.html.

[107] Brent H Besler, Kenneth M Merz Jr., and Peter A Kollman. Atomic charges derived from semiempirical methods. *J. Comput. Chem.*, 11:431–439, 1990. doi:10.1002/jcc.540110404.

[108] Toon Verstraelen, Veronique Van Speybroeck, and Michel Waroquier. The electronegativity equalization method and the split charge equilibration applied to organic systems: Parametrization, validation, and comparison. *J. Chem. Phys.*, 131:044127, 2009. doi:10.1063/1.3187034.

[109] A K Rappé and W A Goddard III. Charge Equillibration for Molecular Dynamics Simulations. *J. Phys. Chem.*, 95:3358–3363, 1991. doi:10.1021/j100161a070.

[110] Razvan A. Nistor, Jeliazko G. Polihronov, Martin H. Müser, and Nicholas J. Mosey. A generalization of the charge equilibration method for nonmetallic materials. *J. Chem. Phys.*, 125(9):094108, 2006. doi:10.1063/1.2346671.

[111] Chunhua Yin, Ziheng Cui, Yang Jiang, David van der Spoel, and Haiyang Zhang. Role of host-guest charge transfer in cyclodextrin complexation: a computational study. *J. Phys. Chem. C.*, 123:17745–17756, 2019. doi:10.1021/acs.jpcc.9b05399.

[112] Frank Jensen. Unifying charge-flow polarization models. *J. Chem. Theory Comput.*, 19:4047–4073, 2023. doi:10.1021/acs.jctc.3c00341.

[113] Peter Itskowitz and Max L Berkowitz. Chemical potential equalization principle: direct approach from density functional theory. *J. Phys. Chem. A*, 101(31):5687–5691, 1997. doi:10.1021/jp963962u.

[114] W J Mortier, S K Ghosh, and S Shankar. Electronegativity-equalization method for the calculation of atomic charges in molecules. *J. Amer. Chem. Soc.*, 108:4315–4320, 1986. doi:10.1021/ja00275a013.

[115] Riccardo Chelli, Piero Procacci, Roberto Righini, and Salvatore Califano. Electrical response in chemical potential equalization schemes. *J. Chem. Phys.*, 111(18):8569–8575, 1999. doi:10.1063/1.480198.

[116] Jiahao Chen, Dirk Hundertmark, and Todd J. Martínez. A unified theoretical framework for fluctuating-charge models in atom-space and in bond-space. *J. Chem. Phys.*, 129(21):214113, 2008. doi:10.1063/1.3021400.

[117] T. Verstraelen, P. W. Ayers, V. Van Speybroeck, and M. Waroquier. Acks2: atom-condensed kohn-sham dft approximated to second order. *J. Chem. Phys.*, 138:074108, 2013. doi:10.1063/1.4791569.

[118] Bogumil Jeziorski, Robert Moszynski, and Krzysztof Szalewicz. Perturbation theory approach to intermolecular potential energy surfaces of van der waals complexes. *Chem. Rev.*, 94(7):1887–1930, 1994. doi:10.1021/cr00031a008.

[119] Trent M. Parker, Lori A. Burns, Robert M. Parrish, Alden G. Ryno, and C. David Sherrill. Levels of symmetry adapted perturbation theory (sapt). I. efficiency and performance for interaction energies. *J. Chem. Phys.*, 140(9):094106, 2014. doi:10.1063/1.4867135.

[120] Grzegorz Chałasiński and Małgorzata M Szcześniak. State of the art and challenges of the ab initio theory of intermolecular interactions. *Chem. Rev.*, 100(11):4227–4252, 2000. doi:10.1021/cr990048z.

[121] Jan Řezáč. Non-covalent interactions atlas benchmark data sets: hydrogen bonding. *J. Chem. Theory Comput.*, 16:2355–2368, 2020. doi:10.1021/acs.jctc.9b01265.

[122] Lori A. Burns, John C. Faver, Zheng Zheng, Michael S. Marshall, Daniel G. A. Smith, Kenno Vanommeslaeghe, Alexander D. MacKerell, Kenneth M. Merz, and C. David Sherrill. The BioFragment Database (BFDb): An open-data platform for computational chemistry analysis of noncovalent interactions. *J. Chem. Phys.*, 147:161727, 2017. doi:10.1063/1.5001028.

[123] Justin S. Smith, Olexandr Isayev, and Adrian E. Roitberg. ANI-1, a data set of 20 million calculated off-equilibrium conformations for organic molecules. *Sci. Data*, 4:170193, 2017. doi:10.1038/sdata.2017.193.

[124] Justin M. Turney, Andrew C. Simmonett, Robert M. Parrish, Edward G. Hohenstein, Francesco A. Evangelista, Justin T. Fermann, Benjamin J. Mintz, Lori A. Burns, Jeremiah J. Wilke, Micah L. Abrams, Nicholas J. Russ, Matthew L. Leininger, Curtis L. Janssen, Edward T. Seidl, Wesley D. Allen, Henry F. Schaefer, Rollin A. King, Edward F. Valeev, C. David Sherrill, and T. Daniel Crawford. Psi4: an open-source ab initio electronic structure program. *Wiley Interdiscip. Rev. Comput. Mol. Sci.*, 2:556–565, 2011. doi:10.1002/wcms.93.

[125] J Wang, R M Wolf, J W Caldwell, P A Kollman, and D A Case. Development and testing of a general AMBER force field. *J. Comput. Chem.*, 25:1157–1174, 2004. doi:10.1002/jcc.20035.

[126] Sander Pronk, Szilárd Páll, Roland Schulz, Per Larsson, Pär Bjelkmar, Rossen Apostolov, Michael R Shirts, Jeremy C Smith, Peter M Kasson, David van der Spoel, Berk Hess, and Erik Lindahl. GROMACS 4.5: a high-throughput and highly parallel open source molecular simulation toolkit. *Bioinformatics*, 29:845–854, April 2013. doi:10.1093/bioinformatics/btt055.